A Tour of C++

요약 정리

목차

책 정보 및 정리 범위 **3**

전체 학습 내용 개요 **4**

장별 요약 및 상세 정리 **7**

제1장. 기초 7

제2장. 사용자 정의 타입 16

제3장. 모듈화 21

제4장. 클래스 31

제5장. 필수적인 연산 40

제6장. 템플릿 49

제7장. 컨셉과 제네릭 프로그래밍 57

제8장. 라이브러리의 개요 65

제9장. 문자열과 정규 표현식 68

제10장. 입력과 출력 75

제11장. 컨테이너 85

제12장. 알고리즘 92

제13장. 유틸리티 100

제14장. 수치 111

제15장. 동시성 117

제16장. 역사와 호환성 127

핵심 문법 및 코드 모음 **137**

주요 개념 복습 **146**

개념 색인 **149**

책 정보 및 정리 범위

책 정보

책 제목: A Tour of C++

책 제목은 사용자의 확인에 따라 반영했다. 제공된 PDF에는 표제지가 없어 제목의 확인 경로를 원문 정보와 구별한다.

저자: [원문에서 확인할 수 없음]

제공된 351쪽 스캔 PDF에서 실제로 확인한 1~16장의 내용을 원래 장·절 순서로 재구성한 학습 교재다. 원본 책 전체의 완전한 요약이라고 표시하지 않는다. 원문만을 내용 출처로 삼았으며, 원문 속 인용 자료나 인터넷 자료로 설명을 보충하지 않았다.

제공된 PDF의 전체 쪽수: 351

확인 가능한 범위: 원문 목차 5쪽과 인쇄 19~370쪽 중 제공된 346쪽. 중간 누락과 마지막 미제공 부분은 아래에 구별한다.

원문 확인 범위와 제한

- 표지·표제지·판권지·지은이 소개가 제공되지 않았다. 책 제목은 사용자가 확인한 A Tour of C++를 반영했다. 저자명은 원문에서 확인할 수 없어 기재하지 않았다.
- 인쇄 90·140·288·289·326·346쪽은 제공 파일에 없다. 장 경계에 있는 쪽도 빈 페이지라고 추정하지 않았다.
- 인쇄 288~289쪽은 13.4 특수 컨테이너 설명 중간에서 누락됐다. 해당 부분의 코드·정의·결과를 임의 복원하지 않았다.
- 인쇄 371쪽 이후는 제공되지 않았다. 16.4 참고 자료의 나머지, 16.5 조언 본문(목차상 376쪽), 원본 찾아보기(목차상 378쪽)를 읽거나 요약한 것으로 표시하지 않는다.
- 제공 PDF 1~12쪽은 원문 인쇄 359~370쪽이다. 이 부분을 인쇄 347~358쪽 뒤에 배치해 원래 16장의 흐름을 복원했다.
- OCR은 독서 보조로 사용했다. 수록한 대표 코드는 원문 이미지와 대조했으며, 의심되는 원문 표기와 판독 한계는 해당 위치에 표시했다. 나머지 모든 문자에 대해 완전한 전사 정확성을 보증하는 것은 아니다.
- 원문의 C++11·14·17 관련 설명과 C++20·모듈·컨셉·Ranges TS 등에 대한 당시의 예정·실험적 상태를 그대로 유지했다. 현재 표준이나 최신 자료로 수정하지 않았다.
- 중요한 원문 코드 중 학습 목적에 맞는 대표 예제를 선별했다. 발췌와 원문의 생략 표시는 구별해 알렸으며, 모든 코드를 전재한 자료는 아니다.
- 실행 결과가 제시되지 않은 예제에는 [원문에 실행 결과 미제시]를 표시했다. 코드를 실행·수정해 새 결과를 생성하지 않았다.

출처의 "인쇄"는 스캔 이미지에서 확인한 원본 쪽수이고 "제공 PDF"는 첨부 파일의 첫 쪽을 1로 세는 위치다. 이 학습 교재의 목차·쪽번호·색인은 새 교재의 실제 PDF 쪽수를 사용한다. 원문의 번호와 교재 번호는 서로 다르다.

전체 학습 내용 개요

제1장. 기초

출처: 책 19-48쪽 / 원문 PDF 18-47쪽

- 컴파일과 링크, 함수, 타입·초기화, 스코프·수명, 상수, 포인터·배열·참조를 통해 C++의 기본 계산 모델을 살펴본다.
- 기본 연산이 하드웨어와 대응하는 방식과 초기화·대입의 차이를 연결한다.

제2장. 사용자 정의 타입

출처: 책 49-60쪽 / 원문 PDF 48-59쪽

- struct·class·union·enum으로 표현과 연산을 갖춘 타입을 만든다.
- Vector 예제를 데이터 구조→인터페이스 분리→생성자에 의한 초기화로 발전시킨다.

제3장. 모듈화

출처: 책 61-89쪽 / 원문 PDF 60-88쪽

- 선언과 정의를 나누고, 헤더·모듈·네임스페이스로 프로그램 구성 요소의 경계를 표현한다.
- 예외·불변 조건·계약·정적 어써션으로 에러를 다루고, 인자·반환 값으로 정보를 전달한다.

제4장. 클래스

출처: 책 91-118쪽 / 원문 PDF 89-116쪽

- 구체 타입, 추상 타입, 클래스 계층 구조라는 세 가지 기본 형태를 비교한다.
- complex와 Vector에서 생성·소멸과 자원 관리를 배우고, Container·Shape에서 가상 함수·상속·다형성을 살펴본다.

제5장. 필수적인 연산

출처: 책 119-139쪽 / 원문 PDF 117-137쪽

- 생성·소멸·복사·이동을 관련 연산의 집합으로 설계한다.
- 기본 멤버별 복사의 한계와 자원 소유권 이전을 이해하고, 비교·컨테이너·입출력·리터럴·교환·해시의 관례를 정리한다.

제6장. 템플릿

출처: 책 141-162쪽 / 원문 PDF 138-159쪽

- 타입·값을 파라미터화한 클래스와 함수
- 함수 객체·람다로 알고리즘 동작 지정
- 인자 추론·별칭·컴파일 시간 선택

제7장. 컨셉과 제네릭 프로그래밍

출처: 책 163-182쪽 / 원문 PDF 160-179쪽

- 템플릿 요구 사항을 컨셉으로 드러내기
- 의미 있는 일반화와 리프팅
- 파라미터 팩·접힘·인자 포워딩·컴파일 모델

제8장. 라이브러리의 개요

출처: 책 183-188쪽 / 원문 PDF 180-185쪽

- 기본 언어 위에서 라이브러리가 맡는 역할
- 표준 라이브러리의 기능 분류
- 헤더 포함과 std 이름 접근

제9장. 문자열과 정규 표현식

출처: 책 189-208쪽 / 원문 PDF 186-205쪽

- 소유하는 문자열과 비소유 읽기 뷰
- 문자열 연산과 표현·수명·구간 검사
- 정규 표현식의 검색·표기법·부분 매치·반복자

제10장. 입력과 출력

출처: 책 209-232쪽 / 원문 PDF 206-229쪽

- 스트림의 타입 변환·상태·자원 관리
- 기본·사용자 정의 타입의 입출력과 형식화
- 파일·문자열 스트림과 이식성 있는 파일 시스템

제11장. 컨테이너

출처: 책 233-252쪽 / 원문 PDF 230-249쪽

- 컨테이너는 객체를 저장하는 자원 핸들이며 시퀀스를 정의한다.
- vector의 크기·용량·요소 값 의미와 구간 검사를 이해한다.
- list·map·unordered_map을 저장 구조와 조회 조건으로 구분한다.

제12장. 알고리즘

출처: 책 253-274쪽 / 원문 PDF 250-271쪽

- 반복자로 표현한 구간에 공통 알고리즘을 적용한다.
- 컨테이너·알고리즘 분리, 검색 실패, 출력 공간 조건을 확인한다.
- 술어·스트림 반복자·컨셉·실행 정책으로 알고리즘 사용을 확장한다.

제13장. 유틸리티

출처: 책 275-287, 290-314쪽 / 원문 PDF 272-309쪽

- RAII·스마트 포인터·이동·포워딩으로 자원과 소유권을 표현한다.
- 비소유 구간·유사 컨테이너·선택지 타입을 구별한다.
- 할당자·시간·함수 어댑션·타입 함수의 역할과 원문 조건을 익힌다.

제14장. 수치

출처: 책 315-325쪽 / 원문 PDF 310-320쪽

- 표준 수학 함수와 에러 표현, 시퀀스 수치 알고리즘을 다룬다.
- 복소수·난수·수치 배열·수치 타입 속성을 연결한다.

제15장. 동시성

출처: 책 327-345쪽 / 원문 PDF 321-339쪽

- 태스크·스레드와 인자·결과 전달을 다룬다.
- 공유 데이터의 경합·락·데드락·이벤트 대기 조건을 이해한다.
- future·promise·packaged_task·async로 태스크 통신을 표현한다.

제16장. 역사와 호환성

출처: 책 347-370쪽 / 원문 PDF 1-12, 340-351쪽

- 실제 시스템 문제를 해결하면서 언어와 라이브러리가 함께 발전한 과정.
- C++11·14·17의 언어 기능과 라이브러리 추가 사항, 제거·지원 중단의 구별.
- C/C++의 역사적 관계와 스타일·void*·링크 호환성.

장	원문 범위
제1장. 기초	출처: 책 19-48쪽 / 원문 PDF 18-47쪽
제2장. 사용자 정의 타입	출처: 책 49-60쪽 / 원문 PDF 48-59쪽
제3장. 모듈화	출처: 책 61-89쪽 / 원문 PDF 60-88쪽
제4장. 클래스	출처: 책 91-118쪽 / 원문 PDF 89-116쪽
제5장. 필수적인 연산	출처: 책 119-139쪽 / 원문 PDF 117-137쪽
제6장. 템플릿	출처: 책 141-162쪽 / 원문 PDF 138-159쪽
제7장. 컨셉과 제네릭 프로그래밍	출처: 책 163-182쪽 / 원문 PDF 160-179쪽
제8장. 라이브러리의 개요	출처: 책 183-188쪽 / 원문 PDF 180-185쪽
제9장. 문자열과 정규 표현식	출처: 책 189-208쪽 / 원문 PDF 186-205쪽
제10장. 입력과 출력	출처: 책 209-232쪽 / 원문 PDF 206-229쪽
제11장. 컨테이너	출처: 책 233-252쪽 / 원문 PDF 230-249쪽
제12장. 알고리즘	출처: 책 253-274쪽 / 원문 PDF 250-271쪽
제13장. 유틸리티	출처: 책 275-287, 290-314쪽 / 원문 PDF 272-309쪽
제14장. 수치	출처: 책 315-325쪽 / 원문 PDF 310-320쪽
제15장. 동시성	출처: 책 327-345쪽 / 원문 PDF 321-339쪽
제16장. 역사와 호환성	출처: 책 347-370쪽 / 원문 PDF 1-12, 340-351쪽

장별 요약 및 상세 정리

제1장. 기초

출처: 책 19-48쪽 / 원문 PDF 18-47쪽

학습 내용 개요

- 컴파일과 링크, 함수, 타입·초기화, 스코프·수명, 상수, 포인터·배열·참조를 통해 C++의 기본 계산 모델을 살펴본다.
- 기본 연산이 하드웨어와 대응하는 방식과 초기화·대입의 차이를 연결한다.

설명의 흐름

1. 소스 파일→컴파일→목적 파일→링크→실행 파일
2. 함수와 타입으로 작업·데이터에 이름 부여→초기화와 수명 관리→선택·반복→메모리상의 동작

1.1 소개

출처: 책 20쪽 / 원문 PDF 19쪽

이 장은 C++의 표기법, 메모리 모델과 계산 모델, 코드를 조직해 프로그램을 만드는 기본 방법을 소개한다. C에서 흔히 볼 수 있는 스타일을 뒷받침하는 기능을 포함하며, 원문은 이 스타일을 절차적 프로그래밍이라고 부른다.

1.2 프로그램

출처: 책 20-22쪽 / 원문 PDF 19-21쪽

C++는 컴파일되는 언어다. 컴파일러가 소스 텍스트에서 목적 파일을 만들고, 링커가 목적 파일들을 병합해 실행 가능한 프로그램을 만든다. 실행 파일은 특정 시스템·하드웨어 조합에 맞게 만들어진다. 책에서 말하는 이식성은 주로 같은 소스를 여러 시스템에서 컴파일하고 실행할 수 있다는 뜻이다.

ISO C++는 내장 타입·루프 등을 포함하는 핵심 언어 기능과 컨테이너·입출력 등을 포함하는 표준 라이브러리 구성 요소를 정의한다. 표준 라이브러리는 보조적인 머신 코드를 제외하면 C++ 자체로 구현할 수 있다. C++는 정적 타입 언어이므로 컴파일러는 객체·값·이름·표현식을 사용하는 곳에서 그 타입을 알아야 한다. 적용할 수 있는 연산은 객체의 타입으로 정해진다.

1.2.1 Hello, World!

출처: 책 21-23쪽 / 원문 PDF 20-22쪽

모든 C++ 프로그램에는 전역 함수 main()이 하나 있어야 하며, main() 호출로 프로그램이 시작된다. 중괄호는 함수 몸체를 묶고 //는 줄 끝까지 이어지는 주석이다. main()의 int 반환 값은 시스템에 전달되며, 반환 구문이 없으면 성공을 나타내는 값이 전달된다. 원문은 0이 아닌 반환 값을 실패로 설명하면서, 운영체제·실행 환경마다 이 값의 활용 여부는 다르다고 덧붙인다.

Hello, World! 출력

코드 출처: 책 22쪽 / 원문 PDF 21쪽

C++ 원문

```
#include <iostream>

int main()
{
    std::cout << "Hello, World!\n";
}
```

코드의 의미

입출력 선언을 포함하고 main()에서 문자열 리터럴을 표준 출력에 쓴다.

동작 과정

1. main() 호출 후 std::cout에 문자열을 쓰고 개행한다.

원문 실행 결과

Hello, World! 다음에 새로운 행이 추가된다.

#include <iostream>은 표준 스트림 입출력 기능의 선언을 포함한다. std::cout은 표준 출력 스트림이며, std::는 표준 라이브러리 네임스페이스에서 이름을 찾도록 한정한다. 문자열 리터럴은 큰따옴표로 둘러싼 문자들의 열이며 ₩n은 개행 문자다. 실행 가능한 코드는 기본적으로 함수 안에 있고 main()에서 직접 또는 간접 호출된다.

1.3 함수

출처: 책 23-26쪽 / 원문 PDF 22-25쪽

함수 정의는 어떤 작업을 어떻게 수행할지 명시한다. 함수를 호출하기 전에 선언해야 하며, 선언에는 이름·반환 타입·인자의 개수와 타입이 들어간다. 반환 타입은 이름 앞에, 인자 타입들은 이름 뒤의 괄호 안에 둔다. void는 반환 값이 없음을 뜻한다. 인자의 타입을 검사하고 필요한 암묵적 변환을 수행하므로, 요구 타입과 맞지 않는 인자는 컴파일 시점에 거부될 수 있다.

함수의 타입은 반환 타입과 인자 타입으로 이뤄진다. 멤버 함수의 타입에는 클래스 이름도 포함된다. 선언에 붙인 인자 이름은 독자를 돕지만, 정의가 아닌 선언에서는 컴파일러가 무시한다. 원문은 의미 있는 작업을 짧은 함수로 나누고 이름을 붙이면 의존성을 드러내고 코드의 길이·복잡도에 따른 에러를 줄일 수 있다고 설명한다.

인자 타입에 따른 함수 오버로딩

코드 출처: 책 25쪽 / 원문 PDF 24쪽

C++ 원문

```
void print(int);       // 정수를 인자로 받음
void print(double);    // 부동소수점 수를 인자로 받음
void print(string);    // 문자열을 인자로 받음

void user()
{
    print(42);          // print(int) 호출
    print(9.65);        // print(double) 호출
    print("Barcelona"); // print(string) 호출
}
```

코드의 의미

같은 이름의 함수 중 인자 타입에 가장 적합한 함수를 선택한다.

동작 과정

1. 각 호출에 선택되는 함수는 원문 주석에 명시되어 있다.

원문 실행 결과

[원문에 실행 결과 미제시]

같은 이름에 인자 타입이 다른 함수들을 정의하는 것을 함수 오버로딩이라고 한다. 컴파일러는 호출마다 가장 적절한 함수를 선택한다. 둘 이상을 호출할 수 있고 어느 쪽도 더 적합하지 않으면 모호한 호출 에러가 발생한다. 오버로딩한 함수들은 모두 주어진 인자를 출력하는 print()처럼 같은 의미를 구현해야 한다.

1.4 타입과 변수, 산술 연산

출처: 책 26-27쪽 / 원문 PDF 25-26쪽

선언은 프로그램에 어떤 실체를 드러내고 그 타입을 지정한다. 타입은 객체가 취할 수 있는 값과 연산의 집합이다. 객체는 어떤 타입의 값을 갖는 메모리이며, 값은 타입에 따라 해석되는 비트의 집합이다. 변수는 이름이 붙은 객체다.

원문에 제시된 기본 타입은 bool, char, int, double, unsigned다. 기본 타입은 하드웨어 기능에 직접 대응하고 고정된 크기를 갖지만, 실제 크기는 구현에 따라 다르다. char는 한 문자를 저장하는 크기이며 다른 타입의 크기는 char 크기의 배수다. sizeof로 크기를 확인한다. 수 리터럴은 소수점이나 지수가 있으면 부동소수점 수로 인식한다. 정수의 기본은 10진수이며 0b·0x·0 접두사는 각각 2진·16진·8진을 나타낸다. 작은따옴표는 긴 리터럴의 자릿수 구분자로 사용할 수 있다.

1.4.1 산술 연산

출처: 책 27-29쪽 / 원문 PDF 26-28쪽

기본 타입의 적절한 조합에는 +, -, *, /, 정수 나머지 %와 단항 +·-를 사용할 수 있다. ==·!=·<·>·<=·>=는 비교를 수행한다. 비트별 논리 연산 &, |, ^, ~와 논리 연산 &&, ||, !를 구분한다. 비트별 연산은 각 비트에 적용되며, &&와 ||는 연산 항의 값에 따라 true 또는 false를 반환한다.

대입과 산술 연산에서는 기본 타입 간의 유효한 변환이 수행된다. 일반 산술 변환은 주어진 연산 항에서 얻을 수 있는 최대 정밀도를 목표로 한다. double과 int의 덧셈은 배정도 부동소수점 연산으로 수행되지만, 그 결과를 int에 저장하면 정보가 버려질 수 있다. =는 대입이고 ==는 상등 비교다. +=, -=, *=, /=, %= 및 증가·감소 연산은 자주 쓰는 갱신을 간결하게 표현한다. 원문은 표현식 평가와 대입 방향을 설명하고 함수 인자의 평가 순서는 정해져 있지 않다고 명시한다.

1.4.2 초기화

출처: 책 29-31쪽 / 원문 PDF 28-30쪽

객체는 사용 전에 값을 받아야 한다. = 형식과 중괄호 초깃값 목록 등을 사용할 수 있다. 원문은 명명된 타입에 일반적인 {} 형식을 주로 사용하도록 권한다. {}는 정보 손실 위험이 있는 축소 변환을 막는다. 반면 =를 이용한 기본 타입 변환에서는 double→int나 int→char와 같은 암묵적 축소 변환이 허용될 수 있다.

초기화와 축소 변환

코드 출처: 책 30쪽 / 원문 PDF 29쪽

C++ 원문

```
int i1 = 7.8;   // i1은 7이 됨(당황했는가?)
int i2 {7.8};   // 에러: 부동소수점 수를 정수로 변환
```

코드의 의미

= 초기화에서 정보가 잘릴 수 있는 경우와 {}에서 이를 거부하는 경우를 비교한다.

원문 실행 결과

원문은 i1이 7이 되고 i2 초기화는 에러라고 명시한다.

상수는 초기화하지 않은 채 존재할 수 없다. 변수도 적절한 초깃값을 얻기 전에는 정의하지 않는 것이 원문의 권고다. auto는 초깃값에서 타입을 유추해 반복되는 타입명을 줄인다. 특히 정확한 타입명이 길거나 알기 어려운 제네릭 코드에 유용하다. 스코프가 커서 독자에게 타입을 명확히 보이려 하거나 범위·정밀도를 지정하려는 경우처럼 이유가 있을 때는 구체적인 타입을 명시한다.

1.5 스코프와 수명

출처: 책 31-32쪽 / 원문 PDF 30-31쪽

선언은 이름을 스코프에 드러낸다. 함수·람다 안의 지역 이름은 선언 지점부터 해당 블록 끝까지 유효하다. 클래스 정의 안에서 함수·람다 밖에 둔 이름, 또는 enum class의 이름은 멤버 이름이며 해당 선언의 스코프를 갖는다. 네임스페이스 안에서 함수·람다·클래스·enum class 밖에 정의한 이름은 네임스페이스 멤버다. 어떤 구성 요소에도 속하지 않는 이름은 전역 네임스페이스의 전역 이름으로 본다.

이름의 유효 범위와 객체가 살아 있는 기간을 함께 구별한다. 객체는 사용 전 생성·초기화되어야 하며 일반적으로 포함된 스코프 끝에서 소멸한다. 네임스페이스 객체는 프로그램 종료 시 소멸하고, 멤버는 자신을 포함하는 객체의 소멸 시점에 따른다. new로 생성한 객체는 delete로 소멸할 때까지 유효하다. 임시 객체나 new로 생성한 객체에는 이름이 없을 수도 있다.

1.6 상수

출처: 책 32-34쪽 / 원문 PDF 31-33쪽

const는 값을 바꾸지 않겠다는 약속을 나타내며 컴파일러가 이를 강제한다. 포인터나 참조로 전달된 데이터를 변경하지 않는다는 사실을 인터페이스에 표시할 수 있다. const 값은 실행 시간에 계산될 수 있다. constexpr는 값을 컴파일 시간에 평가한다는 뜻이며, constexpr 상수의 값은 반드시 컴파일 시간에 계산된다.

constexpr 함수와 상수 표현식

코드 출처: 책 33쪽 / 원문 PDF 32쪽

같은 페이지의 선행 선언 예제는 생략하고 square와 max1~max3 블록을 발췌했다.

C++ 원문

```
constexpr double square(double x) { return x*x; }

constexpr double max1 = 1.4*square(17);  // OK 1.4*square(17)는 상수 표현식
constexpr double max2 = 1.4*square(var); // 에러: var은 상수 표현식이 아님
const double max3 = 1.4*square(var);     // OK, 실행 시간에 평가할 수 있음
```

코드의 의미

상수 인자를 사용하는 경우와 변수를 사용하는 경우의 상수 표현식 성립 여부를 구분한다.

원문 실행 결과

[원문에 실행 결과 미제시]

원문 주의 사항

- var는 같은 페이지 앞 예제에서 int var = 17로 선언된 비상수다.

상수 표현식 안에서 호출할 함수도 constexpr로 정의해야 한다. constexpr 함수에 상수가 아닌 인자를 주는 것은 허용되지만 그 호출 결과는 상수 표현식이 아니다. 따라서 같은 연산을 상수 버전·일반 버전으로 중복 정의할 필요가 없다. 원문은 constexpr 함수가 단순하고 부수 효과가 없으며 인자로 전달된 정보만 사용해야 한다고 설명한다. 지역 변수와 루프는 허용되지만 지역이 아닌 변수를 변경할 수 없다.

지역 변수와 루프를 사용하는 constexpr

코드 출처: 책 34쪽 / 원문 PDF 33쪽

C++ 원문

```
constexpr double nth(double x, int n) // 0<=n이라고 가정
{
    double res = 1;
    int i = 0;
    while (i<n) { // while 루프: 조건이 true일 동안 반복(1.7.1절)
        res*=x;
        ++i;
    }
    return res;
}
```

코드의 의미

constexpr 함수 안에서도 지역 변수와 루프를 사용할 수 있음을 보여 준다.

동작 과정

1. i<n인 동안 res에 x를 곱하고 i를 증가시킨다.

2. 반복 뒤 res를 반환한다.

원문 실행 결과

[원문에 실행 결과 미제시]

원문 주의 사항

- 0<=n이라는 전제를 원문 그대로 유지한다.

원문에서 상수 표현식이 필요한 사례는 배열의 경계, case 라벨, 템플릿 값 인자, constexpr로 선언한 상수다. 컴파일 시간 평가는 성능과 관계되며, 상태가 변하지 않는 객체라는 불변성 자체도 설계의 고려 사항이다.

1.7 포인터, 배열, 참조

출처: 책 34-37쪽 / 원문 PDF 33-36쪽

배열은 연속된 영역에 할당되는 동일 타입의 시퀀스다. 배열의 아래쪽 경계는 0이고, 크기는 상수 표현식이어야 한다. 포인터 변수는 적절한 타입 객체가 있는 주소를 저장한다. 선언에서 T*는 T를 가리키는 포인터, 표현식의 단항 &는 주소, 접두사 *는 가리키는 객체의 내용을 뜻한다.

일반 for는 초기화·조건 검사·증가를 명시한다. 구간 for는 시퀀스의 요소들을 처음부터 끝까지 탐색한다. auto x는 요소를 복사하고, auto& x는 요소 자체를 참조한다. 참조는 접근할 때 *를 쓰지 않으며 일단 초기화되면 다른 객체를 참조하도록 바꿀 수 없다. 참조 인자는 원본을 복사하지 않고 변경할 때 사용하며, const 참조는 복사 없이 변경도 방지할 때 사용한다.

구간 for와 요소 참조

코드 출처: 책 36쪽 / 원문 PDF 35쪽

C++ 원문

```
void increment()
{
    int v[] = {0,1,2,3,4,5,6,7,8,9};

    for (auto& x : v) // v에 포함된 모든 x에 1을 더함
        ++x;
    // ...
}
```

코드의 의미

요소를 복사하지 않고 참조해 원래 배열의 요소를 변경한다.

원문 실행 결과

[원문에 실행 결과 미제시]

원문 주의 사항

- // ...는 원문 자체에 있는 생략 표시다.

1.7.1 널 포인터

출처: 책 37-39쪽 / 원문 PDF 36-38쪽

유효하게 역참조하려면 포인터가 실제 객체를 가리켜야 한다. 가리킬 객체가 없거나 가용한 객체가 없음을 나타낼 때 nullptr를 저장한다. 모든 포인터 타입에 같은 nullptr 값을 사용하며, nullptr는 정수가 아니다. 원문은 0·NULL 대신 nullptr를 사용해 정수와 포인터의 혼동을 줄이라고 권한다.

널 검사 후 C 스타일 문자열 탐색

코드 출처: 책 38쪽 / 원문 PDF 37쪽

C++ 원문

```
int count_x(const char* p, char x)
// p[]에 존재하는 x와 동일한 값의 수를 센다.
// p는 0으로 끝나는 char의 배열을 가리킨다고 가정(그렇지 않으면 아무것도 가리키지 않음)
{
    if (p==nullptr)
        return 0;
    int count = 0;
    for (; *p!=0; ++p)
        if (*p==x)
            ++count;
    return count;
}
```

코드의 의미

0으로 끝나는 배열의 문자 중 x와 같은 값의 수를 센다.

동작 과정

1. 널 포인터이면 0을 반환한다.
2. 그렇지 않으면 끝의 0을 만날 때까지 포인터를 증가시킨다.
3. 각 문자와 x를 비교해 count를 늘리고 반환한다.

원문 실행 결과

[원문에 실행 결과 미제시]

원문 주의 사항

- p는 0으로 끝나는 char 배열을 가리키거나 아무것도 가리키지 않는다는 전제가 있다.
- 문자열 리터럴의 문자는 변경 불가하므로 인자를 const char*로 받는다.

수치 값의 조건 테스트는 0과의 비교, 포인터의 조건 테스트는 nullptr와의 비교에 해당한다. while는 조건이 false가 될 때까지 반복한다. 참조에는 널 참조라는 개념이 없으며 반드시 유효한 객체를 참조해야 한다.

1.8 조건 테스트

출처: 책 39-42쪽 / 원문 PDF 38-41쪽

if와 switch는 선택을, while와 for는 반복을 표현한다. 입력 연산자 >>는 표준 입력 스트림 cin에서 값을 읽으며, 오른쪽 연산 항 타입이 받아들일 입력을 결정한다. 변수는 구문을 둘 수 있는 곳에서 선언할 수 있으므로 원문은 실제로 필요한 위치에 정의한다.

switch는 값을 서로 다른 상수 case 라벨들과 비교한다. 어느 라벨과도 맞지 않으면 default가 선택되며, default도 없으면 아무 동작을 하지 않는다. 함수를 반환하지 않고 switch 다음으로 진행하려면 break를 사용한다. 여러 case를 같은 행동에 연결할 수도 있다.

if 초기화 절과 조건

코드 출처: 책 42쪽 / 원문 PDF 41쪽

C++ 원문

```
void do_something(vector<int>& v)
{
    if (auto n = v.size(); n!=0) {
        // ... n!=0 이면 여기를 실행...
    }
    // ...
}
```

코드의 의미

n을 v.size()로 초기화한 뒤 n!=0을 테스트하고 이름의 유효 범위를 if에 제한한다.

원문 실행 결과

[원문에 실행 결과 미제시]

원문 주의 사항

- 본문은 원문 자체에서 // ...로 생략되어 있다.

if의 초기화 절에서 선언한 이름은 다른 브랜치에서도 유효하다. for에서처럼 변수의 스코프를 제한해 가독성을 높이고 에러를 줄이려는 목적이다. 0 또는 nullptr와 비교할 때는 조건 값을 직접 테스트하는 간단한 형식도 사용할 수 있다.

1.9 하드웨어 대응

출처: 책 42-43쪽 / 원문 PDF 41-42쪽

C++의 기본 연산은 하드웨어 기능에 직접 대응하며 대부분 하나의 기계어로 처리된다. 구현은 메모리를 일련의 위치로 바라보고, 타입이 있는 객체를 그 위치에 저장하며, 포인터로 위치를 가리킨다. 포인터는 머신 주소로 표현된다. 배열은 메모리상의 연속된 객체들을 추상화한 것이다. 원문은 이 기본 머신 모델이 수학이 아니라 컴퓨터 하드웨어에 기반한다고 설명한다.

1.9.1 대입

출처: 책 43-45쪽 / 원문 PDF 42-44쪽

내장 타입의 대입은 머신의 복사 연산이다. x=y 뒤에 x와 y는 같은 값을 갖지만 서로 독립된 객체다. 포인터 대입 p=q도 주소 값을 복사하므로 두 포인터 자체는 독립적이지만 같은 객체를 가리키게 된다. 반면 참조에 대입하면 참조 대상을 바꾸는 대신 이미 참조하고 있는 객체에 값을 쓴다. 원문은 모든 내장 타입에서 x=y의 결과로 x==y가 성립하며, 대입·상등 비교를 제공하는 잘 설계된 사용자 정의 타입에도 같은 기대를 적용한다.

1.9.2 초기화

출처: 책 45-46쪽 / 원문 PDF 44-45쪽

초기화는 초기화되지 않은 메모리를 유효한 객체로 만드는 일이고, 대입은 이미 값을 가진 객체에 새 값을 쓰는 일이다. 초기화되지 않은 변수를 읽거나 쓰는 효과는 일반적으로 예측할 수 없다. 참조는 초기화 없이 만들 수 없으며, int& r = x도 값 복사가 아니라 r을 x에 바인딩하는 초기화다. string·vector처럼 해제할 자원을 소유하는 타입에서도 두 동작의 구별이 중요하다. 함수 인자 전달과 반환 값의 기본 동작은 초기화와 유사하다.

1.10 조언

출처: 책 46-48쪽 / 원문 PDF 45-47쪽

- 의미 있는 연산을 짧은 함수로 묶고 한 함수는 하나의 논리적 연산을 수행하게 한다. 비슷한 연산은 같은 의미로 오버로딩한다.
- 변수는 반드시 초기화하고, 초깃값이 생길 때까지 선언을 미루며, 스코프를 작게 유지한다. 축소 변환과 복잡한 표현식을 피한다.
- 마법의 수 대신 기호 상수를 쓰고 변경 불가한 데이터를 선호한다. 컴파일 시간 평가가 필요한 함수에는 constexpr를 쓴다.
- 명명된 타입은 {} 초기화를 주로 사용하고 auto로 타입 이름 반복을 줄인다. 선언 하나에는 이름 하나를 둔다.
- unsigned는 비트 조작에 사용하고 포인터는 단순하게 사용한다. 0·NULL 대신 nullptr를 쓴다.
- 이름의 범위와 용도에 맞게 길이를 정하고 비슷한 이름·모두 대문자인 이름을 피한다. 코드가 이미 보여 주는 사실보다 의도를 주석에 쓰고 들여쓰기를 일관되게 한다.

중요 내용 요약

- 타입은 값·연산의 집합이고 변수는 명명된 객체다.
- const의 변경 금지와 constexpr의 컴파일 시간 평가를 구분한다.
- 포인터는 주소를 저장하고, 참조는 초기화한 객체에 바인딩된다.
- 초기화는 객체를 유효하게 만들며 대입은 기존 객체에 값을 쓴다.

혼동하기 쉬운 내용

const와 constexpr

출처: 책 32-34쪽 / 원문 PDF 31-33쪽

구분	const	constexpr
의미	값을 바꾸지 않는다는 약속	컴파일 시간에 평가
계산 시점	실행 시간 계산 가능	상수의 값은 컴파일 시간 계산
함수 인자와의 관계	const 참조로 변경 방지	비상수 인자를 주면 결과는 상수 표현식이 아님

포인터와 참조의 대입

출처: 책 44-46쪽 / 원문 PDF 43-45쪽

구분	포인터	참조
대입 대상	저장한 주소 값	이미 참조하는 객체
재지정	다른 객체 주소를 저장 가능	초기화 뒤 참조 대상을 바꾸지 못함
없음의 표현	nullptr 사용	유효한 객체를 반드시 참조

장 마무리 복습

- 컴파일과 링크의 역할, 소스 이식성의 의미를 다시 연결한다.

- 함수 선언의 인자·반환 타입과 오버로딩 선택을 함께 복습한다.
- 스코프는 이름의 가시성과, 수명은 객체의 생성·소멸과 연결한다.
- 배열의 연속성, 포인터의 주소, 참조의 바인딩을 구별한다.

제2장. 사용자 정의 타입

출처: 책 49-60쪽 / 원문 PDF 48-59쪽

학습 내용 개요

- struct·class·union·enum으로 표현과 연산을 갖춘 타입을 만든다.
- Vector 예제를 데이터 구조→인터페이스 분리→생성자에 의한 초기화로 발전시킨다.

설명의 흐름

1. 내장 타입의 한계→관련 데이터 조직화→표현과 연산 결합→대안 값 저장→명명된 상수 집합

2.1 소개

출처: 책 49-50쪽 / 원문 PDF 48-49쪽

기본 타입, const 제한자, 선언 연산에서 만들 수 있는 타입은 내장 타입이다. 원문은 내장 타입이 하드웨어를 직접적·효율적으로 반영하도록 의도적으로 로우레벨에 가깝게 설계되었다고 설명한다. C++는 내장 타입과 연산에 추상화 메커니즘을 조합해 프로그래머가 필요한 하이레벨 기능을 만들도록 한다.

이렇게 만든 사용자 정의 타입은 적절한 표현 방식과 연산을 갖추며, 클래스와 열거형을 예로 들 수 있다. 내장 타입이나 다른 사용자 정의 타입을 기반으로 만들 수 있다. 잘 설계한 사용자 정의 타입은 사용하기 쉽고 에러 위험이 적으며 효율적일 수 있다. 표준 라이브러리도 주로 사용자 정의 타입으로 구성된다.

2.2 구조체

출처: 책 50-52쪽 / 원문 PDF 49-51쪽

새로운 타입을 만드는 첫 단계는 필요한 구성 요소를 데이터 구조로 조직화하는 것이다. struct Vector의 첫 버전은 요소의 개수 sz와 요소들을 가리키는 double* elem을 묶는다. Vector 변수를 선언하는 것만으로는 elem이 사용할 배열을 가리킨다는 보장이 없으므로 vector_init()이 별도로 필요하다.

Vector의 데이터 표현

코드 출처: 책 50쪽 / 원문 PDF 49쪽

C++ 원문

```
struct Vector {
    int sz;       // 요소의 개수
    double* elem; // 요소를 가리키는 포인터
};
```

코드의 의미

요소 개수와 배열을 가리키는 포인터를 하나의 타입으로 묶는다.

원문 실행 결과

[원문에 실행 결과 미제시]

구조체를 초기화하는 별도 함수

코드 출처: 책 51쪽 / 원문 PDF 50쪽

C++ 원문

```
void vector_init(Vector& v, int s)
{
    v.elem = new double[s]; // s개의 double로 이뤄진 배열 할당
    v.sz = s;
}
```

코드의 의미

새 배열을 할당하고 Vector의 포인터와 크기를 설정한다.

동작 과정

1. 배열을 할당해 주소를 v.elem에 저장한다.

2. v.sz에 s를 저장한다.

원문 실행 결과

[원문에 실행 결과 미제시]

원문 주의 사항

- 이 버전의 초기화·해제 책임은 아직 클래스에 결합되어 있지 않다.

new는 자유 저장 영역, 동적 메모리 또는 힙이라고 부르는 곳에 메모리를 할당한다. 그 객체는 생성된 스코프와 무관하게 delete로 소멸할 때까지 유효하다. Vector&는 const가 아닌 참조이므로 vector_init()은 인자로 받은 원본 Vector를 변경할 수 있다. 이름·참조를 통해 struct 멤버에 접근할 때는 .을 쓰고, 포인터를 통해서는 ->를 쓴다.

원문의 read_and_sum()은 Vector를 초기화하고 입력을 요소에 저장한 뒤 합을 계산한다. 그러나 사용자는 elem과 sz라는 표현의 세부 사항을 알아야 한다. 원문은 이 Vector를 언어 기능·설계 기법을 설명하기 위한 점진적 예제로 쓰며, 표준 vector나 string을 재발명하기보다 표준 라이브러리 구성 요소를 활용하라고 권한다.

2.3 클래스

출처: 책 53-55쪽 / 원문 PDF 52-54쪽

사용자 정의 타입을 일관되게 사용하고 표현을 나중에 개선하려면, 모두가 쓰는 인터페이스와 외부에서 접근할 수 없는 데이터를 사용하는 구현을 분리한다. class가 이를 지원한다. 클래스는 데이터·함수·타입을 멤버로 가질 수 있고 public 멤버가 인터페이스를 정의한다. private 멤버에 대한 접근은 인터페이스를 통해 이뤄진다.

인터페이스와 표현을 분리한 Vector

코드 출처: 책 53쪽 / 원문 PDF 52쪽

C++ 원문

```
class Vector {
public:
    Vector(int s) :elem{new double[s]}, sz{s} { } // Vector 생성
    double& operator[](int i) { return elem[i]; } // 요소 접근: 첨자 연산
    int size() { return sz; }
private:
    double* elem; // 요소를 가리키는 포인터
    int sz;       // 요소의 개수
};
```

코드의 의미

생성·요소 접근·크기 조회를 public 인터페이스로 제공하고 표현은 private에 둔다.

원문 실행 결과

[원문에 실행 결과 미제시]

원문 주의 사항

- 원문 이 단계의 예제에는 에러 처리와 소멸자가 아직 없다.

Vector와 이름이 같은 멤버 함수는 생성자다. 객체를 만들 때 생성자가 반드시 사용되므로 별도 초기화 함수 호출을 잊는 실수를 막는다. Vector(int)는 생성 시 요소 개수를 정수로 주어야 함을 드러낸다. 멤버 초깃값 목록의 elem{new double[s]}는 할당된 배열의 주소로 elem을, sz{s}는 s로 sz를 초기화한다.

operator[]는 적절한 요소의 double&를 반환해 읽기와 쓰기를 제공하고, size()는 요소 개수를 돌려준다. Vector 객체 자체는 elem과 sz를 갖는 고정 크기의 핸들이며, 실제 요소는 다른 곳에 저장된다. 따라서 객체마다 요소 개수가 달라도 핸들 자체의 크기는 같다. 원문은 이러한 핸들 기법이 개수가 변하는 데이터에 흔히 쓰인다고 설명한다.

이 단계의 Vector에는 아직 에러 처리와 할당한 배열의 해제가 없다. 원문은 이를 각각 3.5절과 4.2.2절에서 보완한다. struct와 class에는 근본적 차이가 없으며, struct는 멤버의 기본 접근이 public인 class다. 따라서 struct에도 생성자를 포함한 멤버 함수를 둘 수 있다.

2.4 공용체

출처: 책 55-57쪽 / 원문 PDF 54-56쪽

union은 모든 멤버를 같은 메모리 주소에 할당하는 구조체이며, 원문은 공용체의 크기를 가장 큰 멤버의 크기로 설명한다. 일반적으로 실제 값을 갖는 멤버는 하나뿐이다. 원문의 Entry 예에서는 Node*와 int가 동시에 사용되지 않아 별도 멤버로 두면 공간이 낭비된다. 둘을 union Value에 넣으면 메모리를 절약한다.

Node*와 int의 저장 공간 공유

코드 출처: 책 56쪽 / 원문 PDF 55쪽

C++ 원문

```
union Value {
    Node* p;
    int i;
};
```

코드의 의미

동시에 쓰지 않는 두 종류의 값을 같은 메모리에 저장한다.

원문 실행 결과

[원문에 실행 결과 미제시]

원문 주의 사항

- 어느 멤버가 유효한지는 프로그래머가 관리해야 한다.

언어는 union에 현재 어느 종류의 값이 있는지 확인하지 않으므로 프로그래머가 관리해야 한다. Entry의 Type t가 ptr이면 v.p, num이면 v.i를 쓰는 관계를 지켜야 한다. 타입 필드와 실제 저장된 타입의 관계를 직접 관리하면 에러가 발생하기 쉽다. 원문은 공용체와 타입 필드를 캡슐화하고 멤버 함수로만 접근하게 하는 태그된 공용체의 추상화를 권한다.

variant로 표현하는 대안 값

코드 출처: 책 57쪽 / 원문 PDF 56쪽

C++ 원문

```
struct Entry {
    string name;
    variant<Node*,int> v;
};

void f(Entry* pe)
{
    if (holds_alternative<int>(pe->v)) // *pe가 int를 포함하는가?(13.5.1절 참고)
        cout << get<int>(pe->v); // int 가져오기
    // ...
}
```

코드의 의미

variant가 int를 포함하는지 확인한 뒤 그 값을 가져온다.

동작 과정

1. holds_alternative<int>가 참인 경우에 get<int>를 사용한다.

원문 실행 결과

[원문에 실행 결과 미제시]

원문 주의 사항

- // ...는 원문에서 생략한 부분이다.

표준 라이브러리 variant는 대안적인 여러 타입 중 하나의 값을 제공하며 대부분의 상황에서 union을 대체할 수 있다. variant<Node*,int>는 Node* 또는 int를 저장한다. holds_alternative<int>()로 int 포함 여부를 확인

하고 get<int>()로 값을 가져오는 원문 예는 별도의 타입 필드를 수작업으로 관리하는 경우보다 간단하고 안전하다는 설명에 연결된다.

2.5 열거형

출처: 책 57-59쪽 / 원문 PDF 56-58쪽

열거형은 여러 정수 값의 집합을 명명된 상수들로 표현하는 사용자 정의 타입이다. enum class는 열거자의 스코프를 해당 열거형으로 제한하므로 Color::red와 Traffic_light::red를 혼동하지 않고 같은 red 이름을 사용할 수 있다. 두 열거형은 독립된 타입이며 서로의 값이나 정수와 암묵적으로 혼용할 수 없다.

서로 독립된 enum class

코드 출처: 책 57쪽 / 원문 PDF 56쪽

C++ 원문

```
enum class Color { red, blue , green };
enum class Traffic_light { green, yellow, red };

Color col = Color::red;
Traffic_light light = Traffic_light::red;
```

코드의 의미

열거자의 스코프를 열거형으로 제한해 같은 이름을 구분한다.

원문 실행 결과

[원문에 실행 결과 미제시]

원문은 기반 타입에서 열거형을 초기화할 필요가 있으면 Color{5}나 Color y {6}처럼 명시적 초기화를 할 수 있다고 설명한다. 기본적으로 enum class는 대입·초기화·비교를 정의하며, 사용자 정의 타입이므로 연산을 추가할 수 있다. Traffic_light의 전치 증가 연산은 green→yellow→red→green 순서의 변화를 정의한다.

열거형의 전치 증가 연산 정의

코드 출처: 책 59쪽 / 원문 PDF 58쪽

C++ 원문

```
Traffic_light& operator++(Traffic_light& t) // 전치 증가: ++
{
    switch (t) {
    case Traffic_light::green: return t=Traffic_light::yellow;
    case Traffic_light::yellow: return t=Traffic_light::red;
    case Traffic_light::red: return t=Traffic_light::green;
    }
}

Traffic_light next = ++light; // 다음 값은 Traffic_light::green
```

코드의 의미

사용자 정의 타입인 열거형에 신호의 전환을 나타내는 연산을 정의한다.

원문 실행 결과

원문 주석은 다음 값이 Traffic_light::green이라고 명시한다.

원문 주의 사항

- light의 선행 값은 앞 페이지 예제의 Traffic_light::red다.

class 없이 enum을 쓰면 열거자 이름은 enum 이름과 같은 스코프에 속하고, 열거자 값은 정수로 암묵 변환된다. 원문의 enum Color { red, green, blue }; int col = green;에서 col은 1이다. 기본 열거자 값은 0에서 시작해 하나씩 증가한다. 원문은 예측하지 못한 에러를 줄이기 위해 enum class를 선호하라고 조언한다.

2.6 조언

출처: 책 59-60쪽 / 원문 PDF 58-59쪽

- 내장 타입이 너무 로우레벨이면 잘 정의된 사용자 정의 타입을 쓰고, 관련 데이터는 struct나 class로 조직한다.
- class로 인터페이스와 구현을 분리하고 생성자로 초기화를 보장·단순화한다.
- 벌거벗은 union을 피하고 공용체와 타입 필드를 클래스로 감싼다.
- 명명된 상수 집합에는 열거형을 사용한다. enum보다 enum class를 선호하고 안전하고 쉽게 쓸 수 있도록 필요한 연산을 정의한다.

중요 내용 요약

- 생성자는 객체 생성 시 사용되므로 별도 초기화 호출 누락을 막는다.
- 핸들의 크기와 핸들이 참조하는 데이터의 개수는 구별한다.
- union의 저장 공간 공유에는 활성 타입 관리가 필요하며, variant는 대안적인 타입 중 하나의 값을 제공한다.
- enum class는 이름의 스코프와 독립된 타입을 제공한다.

혼동하기 쉬운 내용

원문에서 비교한 데이터 표현

출처: 책 53-59쪽 / 원문 PDF 52-58쪽

구분	원문의 핵심 차이
struct / class	struct는 기본 멤버 접근이 public인 class이며 근본적 차이는 없다.
직접 union / variant	union의 저장 타입은 프로그래머가 관리한다. variant는 대안 타입 중 하나를 제공한다.
enum / enum class	enum은 이름이 바깥 스코프에 있고 정수로 암묵 변환된다. enum class는 스코프와 타입을 제한한다.

장 마무리 복습

- 관련 데이터의 구조화에서 표현·인터페이스 분리로 이어지는 Vector의 발전을 복습한다.
- 생성자, 초깃값 목록, operator[], size()가 각각 맡는 역할을 연결한다.
- 타입 필드와 union의 활성 값 사이 관계를 캡슐화해야 하는 이유를 확인한다.
- Color와 Traffic_light가 같은 red 이름을 가져도 독립된 타입이라는 점을 재확인한다.

제3장. 모듈화

출처: 책 61-89쪽 / 원문 PDF 60-88쪽

학습 내용 개요

- 선언과 정의를 나누고, 헤더·모듈·네임스페이스로 프로그램 구성 요소의 경계를 표현한다.

- 예외·불변 조건·계약·정적 어써션으로 에러를 다루고, 인자·반환 값으로 정보를 전달한다.

설명의 흐름

1. 인터페이스/구현 분리→분할 컴파일→모듈과 네임스페이스→에러 전략→함수 경계의 정보 전달

3.1 소개

출처: 책 61-63쪽 / 원문 PDF 60-62쪽

독립적으로 개발된 함수·타입·계층 구조·템플릿을 관리하려면 상호작용을 명확히 정의해야 한다. 첫 단계는 인터페이스와 구현을 분리하는 것이다. C++는 선언으로 인터페이스를 표현하고 함수·타입을 사용하는 데 필요한 사항을 명시한다. 함수 몸체인 정의는 다른 곳에 둘 수 있다.

클래스 밖의 멤버 정의에서는 Vector::처럼 클래스 이름으로 한정한다. 라이브러리도 같은 언어 기능으로 만든 다른 코드이므로 기본 원리는 같다. 원문은 구성 요소를 여러 번 선언할 수 있지만 정의는 한 번만 가능하다고 설명한다.

3.2 분할 컴파일

출처: 책 63-66쪽 / 원문 PDF 62-65쪽

사용자 코드는 타입·함수의 선언만 있으면 컴파일할 수 있고, 정의는 다른 파일에서 따로 컴파일할 수 있다. 분할 컴파일은 논리적으로 구별되는 부분을 강제로 나누고 컴파일 시간을 줄이는 데 도움이 된다. 모듈의 인터페이스 선언은 보통 헤더에 두고, 사용자와 구현 파일이 모두 같은 헤더를 포함해 컴파일러가 일관성을 확인하게 한다.

분할 컴파일을 위한 헤더

코드 출처: 책 64쪽 / 원문 PDF 63쪽

C++ 원문

```
// Vector.h:

class Vector {
public:
    Vector(int s);
    double& operator[](int i);
    int size();
private:
    double* elem; // elem은 sz개의 double을 포함하는 배열을 가리킴
    int sz;
};
```

코드의 의미

사용자 코드와 구현 코드가 함께 보는 인터페이스 선언을 담는다.

원문 실행 결과

[원문에 실행 결과 미제시]

헤더를 사용하는 독립 사용자 파일

코드 출처: 책 64쪽 / 원문 PDF 63쪽

C++ 원문

```
// user.cpp:

#include "Vector.h" // Vector의 인터페이스 가져오기
#include <cmath> // sqrt()를 포함하는 표준 라이브러리 수학 함수 인터페이스 가져오기

double sqrt_sum(Vector& v)
{
    double sum = 0;
    for (int i=0; i!=v.size(); ++i)
        sum+=std::sqrt(v[i]); // 제곱근의 합
    return sum;
}
```

코드의 의미

정의가 다른 파일에 있어도 선언을 바탕으로 Vector를 사용하는 코드를 컴파일한다.

동작 과정

1. 각 요소의 제곱근을 누적해 반환한다.

원문 실행 결과

[원문에 실행 결과 미제시]

Vector.h는 인터페이스를 공유하는 지점이며 user.cpp와 Vector.cpp는 독립적으로 컴파일한다. 원문은 프로그램을 의존성이 잘 정의된 모듈의 집합으로 생각하고 파일을 통한 물리적 모듈화를 활용하라고 권한다. 독립적으로 컴파일하는 .cpp 파일과 그 파일이 #include하는 헤더를 포함한 단위를 변환 단위라고 부른다. 한 프로그램은 수천 개의 변환 단위로 구성될 수 있다.

3.3 모듈(C++20)

출처: 책 66-68쪽 / 원문 PDF 65-67쪽

#include는 사용하는 변환 단위마다 헤더 텍스트를 다시 처리한다. 또한 먼저 포함한 헤더의 선언·매크로가 뒤 헤더의 의미에 영향을 줄 수 있어 포함 순서가 중요해지는 문제가 있다. 원문 시점의 module은 아직 ISO C++가 아니라 ISO 기술 명세에 포함된 기능이며, 세부 사항이 바뀔 수 있고 제품 수준에서 사용하려면 더 시간이 필요할 수 있다고 설명된다. 이 요약은 그 상태 설명을 그대로 유지한다.

모듈 선언과 노출할 클래스

코드 출처: 책 66-67쪽 / 원문 PDF 65-66쪽

이 모듈 예제의 앞부분을 발췌했다. 뒤의 멤버 함수 정의·비멤버 size 정의와 사용자 파일은 생략했다.

C++ 원문

```
// Vector.cpp 파일:
module; // module을 정의

// ... Vector 구현에 필요한 내용 작성 ...

export module Vector; // "Vector" 모듈 정의

export class Vector {
public:
    Vector(int s);
    double& operator[](int i);
    int size();
private:
    double* elem; // elem은 sz개의 double을 포함하는 배열을 가리킴
    int sz;
};
```

코드의 의미

Vector 모듈에서 사용자에게 클래스 인터페이스를 노출한다.

원문 실행 결과

[원문에 실행 결과 미제시]

원문 주의 사항

- 원문은 당시 모듈을 ISO 기술 명세에 포함된 기능으로 설명한다.

export는 모듈 사용자에게 보여 줄 인터페이스를 노출하고, 필요한 곳에서 import로 가져온다. 원문은 import와 #include를 함께 써 기존 코드에서 점진적으로 옮기는 방법을 보인다. 모듈은 사용하는 모든 변환 단위마다 다시 컴파일하지 않고 한 번 컴파일한다. 두 모듈의 import 순서는 의미에 영향을 주지 않으며 import는 전이되지 않는다. 즉 모듈 내부에서 가져온 이름을 그 모듈의 사용자가 암묵적으로 접근·조작할 수 있는 것은 아니다.

3.4 네임스페이스

출처: 책 68-70쪽 / 원문 PDF 67-69쪽

네임스페이스는 일부 선언을 묶고 독립적으로 개발한 이름들이 충돌하지 않도록 한다. 다른 네임스페이스의 이름은 std::cout 또는 My_code::main처럼 이름을 한정해 접근한다. 프로그램의 실제 main()은 전역 네임스페이스에 둔다.

using std::swap과 같은 using 선언은 특정 이름을 현재 스코프에서 선언한 것처럼 사용하게 한다. using namespace std와 같은 using 지시자는 네임스페이스의 모든 이름을 사용할 수 있게 한다. 후자는 이름을 선택적으로 가져오는 것이 아니므로 신중히 사용한다. 원문은 애플리케이션 전반에서 쓰는 기반 라이브러리나 기존 프로그램을 개선하는 과정에서의 사용을 설명하고, 헤더에는 using 지시자를 넣지 말라고 조언한다.

3.5 에러 처리

출처: 책 71쪽 / 원문 PDF 70쪽

에러 처리의 가장 중요한 도구는 타입 시스템이다. 내장 타입과 제어 구문만 사용하기보다 적절한 사용자 정의 타입과 알고리즘으로 높은 수준의 개념을 표현하면 실수 가능성을 낮추고 컴파일러가 에러를 찾을 가능성을 높인다. 추상화는 런타임 에러를 탐지하는 지점과 처리하는 지점도 분리한다. 라이브러리를 많이 쓰는 큰 프로그램일수록 일관된 전략이 중요하므로 설계 초기에 에러 처리 방식을 고려한다.

3.5.1 예외

출처: 책 71-73쪽 / 원문 PDF 70-72쪽

Vector 작성자는 범위 밖 접근 시 사용자가 무엇을 해야 할지 알 수 없고, 사용자는 잘못된 접근을 항상 감지할 수 없다. 따라서 구현자가 문제를 탐지해 알리고 사용자가 조치를 취하게 한다. 예제의 operator[]는 인덱스가 범위 밖이면 out_of_range를 던진다.

범위 검사와 예외 핸들러

코드 출처: 책 72쪽 / 원문 PDF 71쪽

C++ 원문

```
double& Vector::operator[](int i)
{
    if (i<0 || size()<=i)
        throw out_of_range{"Vector::operator[]"};
    return elem[i];
}

void f(Vector& v)
{
    // ...
    try { // 여기서 발생한 예외를 아래에 정의한 핸들러가 처리
        v[v.size()] = 7; // v의 끝을 넘어서 접근 시도
    }
    catch (out_of_range& err) { // out_of_range 예외
        // ... 스코프 밖 에러 처리...
        cerr << err.what() << '\n';
    }
    // ...
}
```

코드의 의미

범위 밖 접근을 구현자가 감지하고 사용자가 예외를 처리한다.

동작 과정

1. i가 음수이거나 size() 이상이면 throw한다.
2. v[v.size()] 대입 시도는 실패하고 catch에 진입한다.
3. what()으로 예외 메시지를 출력한다.

원문 실행 결과

[원문에 실행 결과 미제시]

원문 주의 사항

- // ...는 원문의 생략 표시다.

throw는 해당 타입의 핸들러에 제어를 넘긴다. 구현은 호출 스택을 거슬러 올라가며 필요한 스코프·함수를 종료하고 그 과정에서 소멸자를 호출한다. try에 처리할 코드를 놓고 catch에서 예외를 잡는다. 예외는 복사를 피하려고 참조로 잡으며 what()은 던질 때 설정한 메시지를 제공한다. 모든 함수에서 try를 반복하기보다 생성자에서 자원을 획득하고 소멸자에서 해제하는 RAII를 이용하는 것이 원문의 권고다.

예외를 던지지 않는 함수는 noexcept로 선언할 수 있다. 그런 함수에서 예외가 빠져나오면 std::terminate()를 호출해 프로그램을 즉시 종료한다. 선언만으로 내부 호출이 성공한다고 보장되는 것은 아니라는 점을 원문의 user() 예제가 보여 준다.

3.5.2 불변 조건

출처: 책 73-76쪽 / 원문 PDF 72-75쪽

함수의 사전 조건은 작동의 기본 가정이다. Vector 인덱싱은 인덱스가 [0:size()) 안에 있어야 한다는 전제를 검사한다. [a:b)는 a를 포함하고 b는 포함하지 않는 반개구간이다. 클래스 불변 조건은 멤버 함수가 의지할 수 있도록 클래스 수준에서 보장되어야 하는 조건이다. Vector에서는 elem이 sz개의 double을 갖는 배열을 가리킨다는 관계가 이에 해당한다.

생성자에서 유효한 크기 확인

코드 출처: 책 74쪽 / 원문 PDF 73쪽

C++ 원문

```
Vector::Vector(int s)
{
    if (s<0)
        throw length_error{"Vector 생성자: 크기가 음수임"};
    elem = new double[s];
    sz = s;
}
```

코드의 의미

불변 조건을 세우기 전에 음수 크기를 거부한다.

동작 과정

1. s<0이면 length_error를 던진다.

2. 그렇지 않으면 배열을 할당하고 크기를 저장한다.

원문 실행 결과

[원문에 실행 결과 미제시]

원문 주의 사항

- new의 할당 실패는 std::bad_alloc으로 보고된다.

- [원문 확인 필요] 이어지는 원문 설명의 "양수가 아님"과 코드의 s<0을 임의로 일치시키지 않았다.

생성자는 불변 조건을 성립시키고, 멤버 함수는 종료할 때 불변 조건을 보존해야 한다. s<0이면 length_error를 던지고, new가 메모리를 할당하지 못하면 std::bad_alloc을 던진다. 사용자 정의 예외 타입으로 탐지 지점에서 처리 지점까지 필요한 정보를 전달할 수도 있다. 함수를 완수하지 못하면 최소한의 지역 정리를 하고 throw;로 다시 던지거나, 복구가 불가능하면 종료하는 예를 원문이 제시한다.

불변 조건은 무엇을 원하는지 명확하게 하고 더 구체적인 설계를 강제하며, 디버깅·테스팅을 거쳐 올바른 코드를 유지하게 돕는다. 생성자·소멸자 기반 자원 관리도 이 개념에 기초한다. 원문의 생성자 조건은 s<0인데

이어지는 설명은 "양수가 아님"이라고 표현한다. [원문 확인 필요] 이 요약에서는 코드의 음수 검사 조건을 바꾸지 않았다.

3.5.3 여러 가지 에러 처리 방식

출처: 책 76-78쪽 / 원문 PDF 75-77쪽

에러를 지역적으로 처리할 수 없다면 호출자에게 실패를 알려야 한다. 원문은 예외, 실패를 나타내는 값, 프로그램 종료를 비교한다. C++의 예외는 정상적인 값을 대신 반환하는 기법이 아니라 요청받은 작업을 완수하지 못했음을 보고하도록 설계되었고 생성자·소멸자와 통합된다.

- 에러 코드: 실패가 흔하고 예상 가능하며 직접 호출자가 처리할 것으로 기대할 수 있는 경우. 파일 열기 실패가 예다.
- 예외: 에러가 드물어 확인을 잊기 쉬운 경우, 직접 호출자가 처리하기 어려운 경우, 새 에러 종류가 늘 수 있는 경우, 생성자처럼 반환 경로가 없는 경우, 값과 에러를 함께 반환하면 경로가 복잡한 경우.
- 예외가 적합한 다른 경우: 여러 호출을 거슬러 전달해야 하거나 복구가 여러 호출 결과에 의존하는 경우, 콜백에서 실패한 경우, 작업 취소가 필요한 경우.
- 종료: 복구할 수 없거나, 심각한 에러를 스레드·프로세스·컴퓨터 재시작으로 처리하도록 설계한 시스템.

원문은 에러 코드와 예외 중 어느 하나를 완전히 배제하지 말고, 판단이 어려우면 예외를 사용하라고 권한다. 예외가 항상 느리다는 믿음도 부정하며, 복잡하거나 드문 실패를 처리하고 반복적인 에러 코드 확인을 피할 때 빠를 수도 있다고 설명한다. 간결한 예외 처리에는 RAII가 필수적이다.

3.5.4 계약

출처: 책 79쪽 / 원문 PDF 78쪽

원문 시점에는 사전 조건·불변 조건을 선택적으로 실행 시간에 검사하는 일반적인 표준 방법이 없다고 설명한다. C++20에 계약 메커니즘이 제안되어 테스트에서는 엄격히 검사하고 배포에서는 최소 검사하는 용도가 소개되지만, 원문은 당시 임시 방편을 사용해야 한다고 명시한다. 예로 RANGE_CHECK 매크로에 따라 범위 검사를 수행하는 코드를 제시한다.

표준 라이브러리의 assert()는 주어진 실행 시간 조건을 확인하는 디버그용 매크로다. 디버그 모드에서 조건이 성립하지 않으면 종료하며, 디버그 모드가 아니면 검사하지 않는다. 원문은 유연성은 적지만 충분한 경우가 많다고 설명한다.

3.5.5 정적 어써션

출처: 책 80-81쪽 / 원문 PDF 79-80쪽

실행 시간 에러는 예외로 보고하되, 컴파일 시간에 찾을 수 있는 에러는 그때 찾는 것이 좋다. 기대 조건을 표현하는 구문을 어써션이라고 하며 static_assert는 상수 표현식으로 나타낼 수 있는 속성을 검사한다. static_assert(A,S)에서 A가 true가 아니면 S를 컴파일러 에러 메시지로 출력한다. S를 생략하면 기본 메시지를 사용한다.

컴파일 시간 속성 검사

코드 출처: 책 80쪽 / 원문 PDF 79쪽

C++ 원문

```
static_assert(4<=sizeof(int), "정수가 너무 작음"); // 정수 크기 확인
```

코드의 의미

현재 시스템에서 int 크기가 최소 4바이트라는 기대를 컴파일 시간에 확인한다.

원문 실행 결과

조건이 성립하지 않으면 원문의 메시지 "정수가 너무 작음"을 컴파일러 에러로 보고한다.

원문의 속력 예제는 함수 인자 speed는 상수 표현식이 아니므로 static_assert에 쓸 수 없고 constexpr local_max는 사용할 수 있음을 보여 준다. 제네릭 코드의 타입 파라미터에 대한 가정을 확인하는 용도로도 유용하다.

3.6 함수 인자와 반환 값

출처: 책 81-82쪽 / 원문 PDF 80-81쪽

함수 호출은 프로그램의 두 부분 사이에 정보를 전달하는 주된 방법이다. 작업에 필요한 정보는 인자로, 결과는 반환 값으로 전달한다. 전역 변수는 에러의 주요 원인이므로 피하고, 공유 상태는 클래스 멤버처럼 함께 잘 정의된 추상 개념을 구현하는 함수들로 제한한다. 객체의 복사·공유 여부, 변경 허용 여부, 빈 객체를 남기는 이동 가능성이 경계 설계의 고려 사항이다. 기본은 복사지만 암묵적 이동으로 최적화될 수 있다.

3.6.1 인자 전달

출처: 책 82-83쪽 / 원문 PDF 81-82쪽

값에 의한 전달은 복사본을 사용하고, 참조에 의한 전달은 호출자 환경의 원본 객체를 참조한다. 원문의 test()는 값 인자 v를 수정해도 원본은 그대로이고, 참조 인자 rv를 수정하면 원본이 바뀜을 출력으로 보여 준다. 작은 객체는 값으로, 복사 비용이 큰 객체는 참조로 전달한다. "작다"는 기준은 아키텍처에 따라 달라지며, 원문은 대략 포인터 두세 개 크기 이하를 제시한다.

성능상 참조를 사용하되 변경이 필요 없으면 const 참조를 선호한다. 기본 함수 인자는 자주 쓰는 기본값을 선언에 지정한다. 원문의 print(int value, int base =10)는 base를 생략하면 10을 사용하며, 기본값을 넘기는 별도 오버로드보다 문법이 간단하다.

3.6.2 값 반환

출처: 책 84-86쪽 / 원문 PDF 83-85쪽

반환의 기본 동작도 복사이며 작은 값에는 적합하다. 참조 반환은 함수 호출과 독립적으로 존재하는 객체에 호출자가 접근할 때 사용한다. Vector 요소는 인덱싱 호출 뒤에도 존재하므로 참조로 반환할 수 있다. 반면 지역 변수는 함수가 끝나면 사라지므로 그 포인터·참조를 반환하면 안 된다.

큰 Matrix 결과도 값을 반환하는 인터페이스를 유지하면서 이동 생성자로 효율적으로 전달할 수 있다. new로 만든 Matrix*를 반환하고 사용자가 delete하게 하는 예전 스타일은 직접 메모리 관리와 해제 누락 위험을 키운다. 작업을 완료하지 못하면 예외를 던진다. auto 반환 타입 추론은 함수 템플릿·람다에서 유용하지만 구현 변경이 반환 타입 변경으로 이어질 수 있어 안정적인 인터페이스를 위해 주의한다.

3.6.3 구조화된 바인딩

출처: 책 86-88쪽 / 원문 PDF 85-87쪽

함수는 하나의 값만 반환하지만 그 값이 여러 멤버를 가진 객체일 수 있다. Entry 읽기 예제는 문자열·정수를 하나의 객체로 반환한다. 구조화된 바인딩은 그 객체의 멤버에 지역 이름을 부여한다. auto [n,v]의 타입은 반환 객체에서 유추하며, 일반 auto처럼 const와 &를 붙일 수 있다.

여러 멤버를 가진 반환 객체

코드 출처: 책 86쪽 / 원문 PDF 85쪽

C++ 원문

```
struct Entry {
    string name;
    int value;
};

Entry read_entry(istream& is) // 원시적인 읽기 함수(10.5절에서 더 나은 버전을 볼 수 있음)
{
    string s;
    int i;
    is >> s >> i;
    return {s,i};
}

auto e = read_entry(cin);
cout << "{ " << e.name << " , " << e.value << " }\n";
```

코드의 의미

문자열·정수를 Entry 한 값으로 반환하고 멤버를 통해 사용한다.

동작 과정

1. 문자열과 정수를 읽어 {s,i}로 Entry 반환 값을 생성한다.

원문 실행 결과

[원문에 실행 결과 미제시]

구조화된 바인딩으로 멤버 사용

코드 출처: 책 87쪽 / 원문 PDF 86쪽

앞 페이지의 Entry·read_entry 정의에 이어지는 사용 블록을 발췌했다.

C++ 원문

```
auto [n,v] = read_entry(is);
cout << "{ " << n << " , " << v << " }\n";
```

코드의 의미

반환된 Entry의 멤버에 지역 이름 n과 v를 부여한다.

원문 실행 결과

[원문에 실행 결과 미제시]

private 데이터가 없는 클래스의 단순한 경우 바인딩 이름의 수는 비정적 데이터 멤버 수와 같아야 하고 이름들은 각 멤버에 대응한다. 원문은 멤버를 직접 지칭할 때와 객체 코드 품질 차이는 없지만 아이디어 표현에 유용하다고 설명한다. 접근 함수를 가진 complex의 구조화된 바인딩 예도 제시하지만 그 구현 기법은 책의 범위 밖이라고 명시한다.

3.7 조언

출처: 책 88-89쪽 / 원문 PDF 87-88쪽

- 선언과 정의를 구별하고 인터페이스를 헤더로 표현한다. 구현 파일도 해당 헤더를 포함한다. 지원되는 경우 모듈을 선호한다.

- 네임스페이스로 논리적 구조를 표현하고 헤더에 using 지시자를 넣지 않는다.
- 설계 초기부터 에러 전략을 정한다. 생성자는 불변 조건을 지키거나 예외를 던지며, 에러는 적절한 지점에서 처리한다.
- 모든 함수에서 모든 예외를 잡으려 하지 말고 명시적 try 반복보다 RAII를 이용한다. 던지지 않는 함수는 noexcept로 선언한다.
- 컴파일 시간에 확인할 수 있는 것은 그때 확인한다. 작은 것은 값으로, 큰 것은 참조로 전달하고 변경이 필요 없으면 const 참조를 선호한다.
- 결과는 함수 반환 값으로 전달하며 반환 타입 추론·구조화된 바인딩을 과용하지 않는다. 명명된 반환 타입은 문서화에 유리할 수 있다.

중요 내용 요약

- 선언은 사용 계약이고 정의는 구현이며, 분할 컴파일은 둘을 물리적으로 분리한다.
- 사전 조건은 함수의 전제이고 클래스 불변 조건은 모든 멤버가 의지하는 관계다.
- 예외·에러 코드·종료는 실패의 성격과 처리 위치에 따라 선택한다.
- 지역 객체의 포인터·참조 반환을 피하고 결과는 값으로 반환한다.

혼동하기 쉬운 내용

헤더와 원문 시점의 모듈

출처: 책 66-68쪽 / 원문 PDF 65-67쪽

기준	#include 헤더	모듈
처리	변환 단위마다 헤더 텍스트 처리	모듈 한 번 컴파일
순서	선언·매크로가 포함 순서에 영향	import 순서가 의미에 영향 없음
가시성	포함한 선언·매크로가 뒤 코드에 영향	import는 전이되지 않음

함수 경계의 전달 방법

출처: 책 82-86쪽 / 원문 PDF 81-85쪽

방법	원문의 사용 기준
값 인자	작고 복사 비용이 낮은 객체
참조 인자	원본 접근 또는 큰 객체 복사 회피
const 참조	복사를 피하고 원본을 변경하지 않음
값 반환	결과 전달의 기본 방법, 큰 값은 이동 가능
참조 반환	호출과 독립적으로 존재하는 객체 접근

장 마무리 복습

- 인터페이스를 공유하면서 구현을 독립 컴파일하는 파일 관계를 복습한다.
- 예외를 탐지하는 함수와 처리하는 함수의 역할을 분리해 본다.
- 생성자·불변 조건·RAII·예외 처리가 연결되는 흐름을 재확인한다.

- 값·참조·const 참조·이동과 구조화된 바인딩이 함수 경계에서 맡는 역할을 정리한다.

제4장. 클래스

출처: 책 91-118쪽 / 원문 PDF 89-116쪽

학습 내용 개요

- 구체 타입, 추상 타입, 클래스 계층 구조라는 세 가지 기본 형태를 비교한다.
- complex와 Vector에서 생성·소멸과 자원 관리를 배우고, Container·Shape에서 가상 함수·상속·다형성을 살펴본다.

설명의 흐름

1. 내장 타입처럼 쓰는 구체 클래스→생성자/소멸자의 자원 관리→인터페이스만 보이는 추상 클래스→가상 호출→계층 구조와 소유권

4.1 소개

출처: 책 91-93쪽 / 원문 PDF 89-91쪽

클래스는 프로그램에서 어떤 개념을 표현하는 사용자 정의 타입이다. 원문은 유용한 아이디어를 클래스라는 코드로 직접 표현하면 내장 타입만으로 구현한 경우보다 이해하고 디버깅하기 쉽다고 설명한다. 이 장은 구체 클래스, 추상 클래스, 계층 구조에 속하는 클래스를 다룬다. 뒤의 장에서는 생애 주기 연산, 템플릿, 제네릭 프로그래밍으로 이어진다.

4.2 구체 타입

출처: 책 93-94쪽 / 원문 PDF 91-92쪽

구체 타입의 기본 생각은 내장 타입처럼 작동하도록 하는 것이다. complex는 자신만의 산술 연산을 가진다는 점을 제외하면 int처럼 쓰고, vector·string은 개선된 배열처럼 쓴다. 타입 정의의 일부로 메모리 표현이 포함되므로, 객체를 지역 메모리나 다른 객체 안에 둘 수 있고, 포인터·참조 없이 직접 접근하고, 완전히 초기화하며, 복사·이동할 수 있다.

표현이 private이더라도 각 객체 안에 실제로 존재하므로 표현을 바꾸면 다시 컴파일해야 한다. 이는 명확성과 효율성을 얻는 대가다. 중요한 데이터를 자유 저장 영역에 두고 객체에는 접근용 포인터만 두면 유연성을 높일 수 있다. 원문은 vector·string을 잘 설계된 인터페이스를 가진 자원 핸들로 설명한다.

4.2.1 산술 타입

출처: 책 94-97쪽 / 원문 PDF 92-95쪽

원문의 complex는 실수부 re와 허수부 im이라는 두 double을 표현으로 사용한다. 두 스칼라, 한 스칼라, 인자 없음의 세 생성자가 각각 일반 복소수·실수에서의 복소수·{0,0} 기본값을 만든다. 인자 없이 호출할 수 있는 생성자를 기본 생성자라고 한다. 이를 두면 초기화되지 않는 경우를 막을 수 있다.

구체 산술 타입 complex

코드 출처: 책 94-95쪽 / 원문 PDF 92-93쪽

C++ 원문

```
class complex {
    double re, im; // 메모리 표현: double 두 개
public:
    complex(double r, double i) :re{r}, im{i} {} // 두 스칼라를 이용해 complex 생성
    complex(double r) :re{r}, im{0} {} // 스칼라 하나를 이용해 complex 생성
    complex() :re{0}, im{0} {} // 기본 complex: {0,0}

    double real() const { return re; }
    void real(double d) { re=d; }
    double imag() const { return im; }
    void imag(double d) { im=d; }

    complex& operator+=(complex z)
    {
        re+=z.re; // re와 im에 더하기
        im+=z.im;
        return *this; // 결과 반환
    }
    complex& operator-=(complex z)
    {
        re-=z.re;
        im-=z.im;
        return *this;
    }

    complex& operator*=(complex); // 클래스 밖의 어딘가에서 정의
    complex& operator/=(complex); // 클래스 밖의 어딘가에서 정의
};
```

코드의 의미

두 double의 표현과 관례적인 생성·접근·복합 대입 연산을 결합한다.

동작 과정

1. 생성자는 실수부·허수부를 초기화한다.
2. const 접근자는 값을 읽고 비const 접근자는 값을 쓴다.
3. +=와 -=는 두 성분을 변경하고 자기 객체 참조를 반환한다.

원문 실행 결과

[원문에 실행 결과 미제시]

클래스 안에 정의한 함수는 기본적으로 인라인이며, 선언 앞의 inline으로 그 의도를 명시할 수도 있다. 원문은 단순한 연산이 함수 호출 없이 구현될 수 있도록 하여 효율성을 얻는다고 설명한다. const 멤버 함수는 호출한 객체를 변경하지 않는다. const·비const 객체 모두에서 호출할 수 있지만, 비const 멤버 함수는 비const 객체에서만 호출한다.

표현에 직접 접근할 필요가 없는 산술 연산은 클래스 밖에 정의할 수 있다. 원문의 비멤버 +·-·*·/는 값으로 받은 인자 복사본을 변경해 반환하므로 호출자의 인자를 바꾸지 않는다. 상등 비교는 실수부·허수부를 모두 비교하고 !=는 !(a==b)로 정의한다. 연산자는 관례대로 신중하게 사용한다. 언어가 정한 문법을 바꿀 수 없으므로 단항 /를 새로 만들거나 int의 +를 다른 뜻으로 재정의할 수 없다.

4.2.2 컨테이너

출처: 책 97-99쪽 / 원문 PDF 95-97쪽

컨테이너는 요소의 컬렉션을 저장하는 객체다. 앞의 Vector도 컨테이너지만 new로 할당한 요소를 해제하지 않았다. 생성자에서 얻은 자원을 해제하는 소멸자가 이를 보완한다. 소멸자의 이름은 클래스 이름 앞에 ~를 붙인다. delete는 개별 객체를, delete[]는 배열을 해제한다.

생성자·소멸자로 배열을 관리하는 Vector

코드 출처: 책 98쪽 / 원문 PDF 96쪽

C++ 원문

```
class Vector {
public:
    Vector(int s) :elem{new double[s]}, sz{s} // 생성자: 자원 획득
    {
        for (int i=0; i!=s; ++i) // 요소 초기화
            elem[i]=0;
    }

    ~Vector() { delete[] elem; } // 소멸자 : 자원 해제

    double& operator[](int i);
    int size() const;
private:
    double* elem; // elem은 double요소 sz개를 가리킴
    int sz;
};
```

코드의 의미

배열의 획득·해제를 Vector 생애에 연결한다.

동작 과정

1. 생성 시 배열을 할당하고 모든 요소를 0으로 초기화한다.
2. 소멸 시 delete[]로 배열을 해제한다.

원문 실행 결과

[원문에 실행 결과 미제시]

원문 주의 사항

- 원문은 이 예제를 단순화하기 위해 에러 처리를 생략했다고 설명한다.

Vector 사용자는 해제 코드를 직접 쓰지 않아도 된다. 지역 Vector는 내장 타입처럼 명명·스코프·할당·생애 주기 규칙에 따르고 스코프를 벗어나면 소멸된다. 원문은 예제를 단순화하려고 이 위치에서 에러 처리 코드는 생략했다고 밝힌다. 따라서 앞 절의 검사가 이 코드에 그대로 들어 있다고 가정하지 않는다.

생성자에서 자원을 획득하고 소멸자에서 해제하는 기법은 RAII, 즉 자원 획득이 곧 초기화다. 객체가 실제 데이터를 소유하는 핸들이 되어 생성·소멸과 자원 수명을 결합한다. 원문은 애플리케이션에 드러난 벌거벗은 new·delete를 피하면 에러와 메모리 누수에 덜 취약한 코드를 만들 수 있다고 설명한다.

4.2.3 컨테이너 초기화

출처: 책 100-102쪽 / 원문 PDF 98-100쪽

컨테이너는 요소의 저장을 위해 존재하므로 편리한 입력 방법이 필요하다. 초깃값 목록 생성자는 요소 목록으로 바로 초기화하며, push_back()은 끝에 하나를 추가하고 크기를 1 늘린다. 요소 개수를 미리 모를 때 push_back()이 유용하다. 원문의 read()는 입력이 파일 끝이나 형식 에러로 끝날 때까지 읽은 double을 계속 추가한다. for를 쓴 이유는 임시 d의 스코프를 루프 안으로 제한하기 위해서다.

초깃값 목록 생성자

코드 출처: 책 101쪽 / 원문 PDF 99쪽

C++ 원문

```
Vector::Vector(std::initializer_list<double> lst) // 초깃값 목록 초기화
    :elem{new double[lst.size()]}, sz{static_cast<int>(lst.size())}
{
    copy(lst.begin(),lst.end(),elem); // lst를 elem에 복사(12.6절)
}
```

코드의 의미

목록 크기만큼 저장 공간을 만들고 그 요소들로 초기화한다.

원문 실행 결과

[원문에 실행 결과 미제시]

원문 주의 사항

- static_cast는 변환되는 실제 값을 검사하지 않는다.

std::initializer_list는 {} 목록을 받아 초깃값 목록 생성자에 전달할 때 사용하는 표준 라이브러리 타입이다. 생성자는 목록 크기만큼 배열을 할당한 후 copy()로 요소를 옮긴다. 표준 라이브러리의 크기 타입은 unsigned 정수여서 이 예제는 static_cast<int>로 변환한다. static_cast는 실제 값을 검사하지 않으므로 올바르게 쓰는 책임은 프로그래머에게 있다.

원문은 명시적 타입 변환을 가능하면 피하고, 검사 없는 캐스트는 시스템의 낮은 수준에 제한하라고 권한다. 객체를 바이트 시퀀스로 취급하는 reinterpret_cast와 const를 제거하는 const_cast도 언급한다. 높은 수준에서는 타입 시스템과 잘 설계된 라이브러리로 검사 없는 변환을 피한다.

4.3 추상 타입

출처: 책 102-106쪽 / 원문 PDF 100-104쪽

구체 타입이 정의 안에 표현을 포함하는 것과 달리, 추상 타입은 사용자에게 상세 구현을 완전히 감춘다. 인터페이스와 메모리 표현을 분리하고 포인터·참조로 접근한다. 순수 가상 함수를 가진 추상 클래스 자체의 객체는 만들 수 없다.

순수 가상 인터페이스 Container

코드 출처: 책 102-103쪽 / 원문 PDF 100-101쪽

C++ 원문

```
class Container {
public:
    virtual double& operator[](int) = 0; // 순수 가상 함수
    virtual int size() const = 0; // const 멤버 함수(4.2.1절)
    virtual ~Container() {} // 소멸자(4.2.2절)
};
```

코드의 의미

실제 저장 방식과 분리된 요소 접근·크기 조회 인터페이스를 정의한다.

원문 실행 결과

[원문에 실행 결과 미제시]

원문 주의 사항

- 순수 가상 함수가 있으므로 Container 자체의 객체를 만들 수 없다.

구체 구현을 모르는 사용자 함수

코드 출처: 책 103쪽 / 원문 PDF 101쪽

C++ 원문

```
void use(Container& c)
{
    const int sz = c.size();
    for (int i=0; i!=sz; ++i)
        cout << c[i] << '\n';
}
```

코드의 의미

Container 인터페이스만으로 모든 요소를 순회해 출력한다.

원문 실행 결과

[원문에 실행 결과 미제시]

virtual은 파생 클래스에서 재정의할 수 있는 함수를 표시한다. =0은 순수 가상 함수로서 실제 구현 클래스가 그 연산을 구현해야 함을 나타낸다. Container는 operator[]와 size()라는 인터페이스만 제공하며, use(Container&)는 구체 구현을 모른 채 그 인터페이스를 호출한다. 여러 클래스에 대한 인터페이스를 제공하는 타입을 원문은 다형 타입이라고 부른다.

Vector를 이용한 추상 인터페이스 구현

코드 출처: 책 104쪽 / 원문 PDF 102쪽

C++ 원문

```
class Vector_container :public Container { // Vector_container가 Container를 구현
public:
    Vector_container(int s) : v(s) { } // 요소들의 벡터
    ~Vector_container() {}

    double& operator[](int i) override { return v[i]; }
    int size() const override { return v.size(); }
private:
    Vector v;
};
```

코드의 의미

구체 Vector를 멤버로 사용해 Container의 가상 연산을 구현한다.

동작 과정

1. operator[]와 size()는 내부 Vector에 작업을 위임한다.
2. 소멸 시 멤버 v의 소멸자가 암묵 호출된다.

원문 실행 결과

[원문에 실행 결과 미제시]

:public Container는 Container로부터 파생된 서브타입임을 뜻한다. Container는 기반·슈퍼클래스, Vector_container는 파생·서브클래스다. 파생 클래스는 기반 클래스 멤버를 상속한다. override는 어떤 가상 함수를 재정의한다는 의도를 명시해 이름이나 타입을 잘못 쓴 실수를 컴파일러가 찾게 한다.

Container에는 초기화할 데이터가 없어 생성자를 따로 두지 않지만 가상 소멸자가 있다. 인터페이스 포인터로 소멸할 때 구체 구현의 자원을 알 수 없으므로 올바른 파생 소멸자를 부르기 위해서다. Vector_container 소멸 시 그 멤버 Vector의 소멸자도 암묵 호출된다.

원문은 list<double>을 표현으로 쓰는 List_container도 구현해 같은 use()를 적용한다. list의 인덱싱은 vector보다 크게 느려 일반적인 인덱스 컨테이너 구현에는 쓰지 않지만 대안 구현을 보이는 예다. 구현 변경이나 새 파생 클래스 추가에도 use(Container&)를 다시 컴파일할 필요가 없다는 유연성을 얻으며, 그 대가로 객체에는 포인터·참조로 접근한다.

4.4 가상 함수

출처: 책 107-108쪽 / 원문 PDF 105-106쪽

c[i]가 어느 구현을 호출할지는 실행 시간의 실제 객체에 달려 있다. 따라서 객체에는 적절한 함수를 찾을 정보가 필요하다. 원문은 일반적인 구현 기법으로 가상 함수 테이블(vtbl)을 설명한다. 컴파일러는 가상 함수 이름을 함수 포인터 테이블의 인덱스로 변환하고, 가상 함수가 있는 각 클래스는 자기 함수들의 테이블을 갖는다.

호출자는 구체 객체의 크기나 데이터 모양을 몰라도 테이블 포인터 위치와 함수 인덱스로 접근할 수 있다. 원문은 가상 호출을 일반 호출에 가까운 효율로 구현할 수 있다고 설명하고, 추가 저장 공간은 가상 함수가 있는 객체당 포인터 하나와 클래스당 테이블 하나라고 제시한다.

4.5 클래스 계층 구조

출처: 책 108-111쪽 / 원문 PDF 106-109쪽

계층 구조는 파생 관계로 클래스 집합을 조직하며 "원은 도형의 일종" 같은 관계를 표현한다. 원문은 Shape 아래 Circle·Triangle, Circle 아래 Smiley를 둔다. Shape는 중심·이동·그리기·회전과 가상 소멸자의 추상 인터페이스를 제공한다. rotate_all()은 실제 도형 종류를 몰라도 Shape*를 통해 같은 rotate() 호출을 적용한다.

Circle은 중심 Point와 반지름으로 구체 속성을 구현하고, Smiley는 Circle을 기반으로 눈·입을 추가한다. Smiley::draw()는 Circle::draw()로 얼굴을 그리고 eyes와 mouth의 draw()를 호출한다. 새로운 파생 클래스는 데이터·연산을 추가할 수 있지만 유연성만큼 잘못된 설계의 가능성도 있다.

추상 기반 클래스 포인터로 파생 객체를 소멸시키려면 기반 소멸자는 가상이어야 한다. 가상 호출이 올바른 파생 소멸자를 선택하고, 그 소멸자는 멤버·기반 소멸자를 암묵 호출한다. 원문의 Smiley 소멸자는 소유한 눈·입을 해제한 후 기반 소멸로 이어진다.

4.5.1 계층 구조의 장점

출처: 책 112-114쪽 / 원문 PDF 110-112쪽

인터페이스 상속은 기반 객체를 사용하는 곳에서 파생 객체를 대신 쓸 수 있게 한다. Container·Shape처럼 기반이 파생의 인터페이스가 되며 종종 추상 클래스다. 구현 상속은 기반의 데이터·함수로 파생 구현을 단순화한다. Smiley가 Circle 생성자와 draw()를 이용하는 것이 예다.

원문의 read_shape()는 입력으로 도형 종류를 읽고 알맞은 Circle·Triangle·Smiley를 생성해 Shape*로 돌려준다. user()는 vector<Shape*>에 넣어 종류를 모른 채 그리기·회전을 수행한다. 이 사용자 코드는 새 도형이 추가되어도 한 번 컴파일해 사용할 수 있다. 예제에서는 user()만 포인터를 보관하므로 모든 도형을 해제할 책임도 user()에 있다.

delete와 Shape의 가상 소멸자가 최하위 파생 클래스부터 소멸자를 호출해 자원을 반납한다. 생성은 기반에서 파생으로, 소멸은 파생에서 기반으로 진행된다. 원문은 해당 예제가 특히 에러 처리 면에서 단순화되어 있음을 명시한다.

4.5.2 계층 구조 탐색

출처: 책 114-115쪽 / 원문 PDF 112-113쪽

공통 Shape 인터페이스 밖에 있는 Smiley::wink() 같은 기능을 사용하려면 실제 타입을 확인할 수 있다. 포인터에 대한 dynamic_cast는 기대 타입 또는 그 파생 타입을 가리키지 않으면 nullptr를 반환하므로 다른 타입이 합법적인 대안일 때 사용한다.

포인터 dynamic_cast와 대안 처리

코드 출처: 책 114쪽 / 원문 PDF 112쪽

C++ 원문

```
Shape* ps {read_shape(cin)};

if (Smiley* p = dynamic_cast<Smiley*>(ps)) { // ... ps 가 Smiley를 가리키는가? ...
    // Smiley가 맞음. 적절히 사용.
}
else {
    // ... Smiley가 아님. 다른 작업 시도 ...
}
```

코드의 의미

실행 시간 타입이 기대한 Smiley인지 확인하고 다른 타입도 대안으로 처리한다.

원문 실행 결과

[원문에 실행 결과 미제시]

원문 주의 사항

- ps가 기대 타입 또는 그 파생 타입을 가리키지 않으면 nullptr를 반환한다.
- 각 브랜치의 구체 작업은 원문에서도 생략되어 있다.

기대한 타입이 아닌 경우를 실패로 취급해야 하면 참조에 dynamic_cast를 사용할 수 있다. 이 경우 잘못된 타입이면 bad_cast 예외를 던진다. 원문은 타입 검사를 직접 쓰지 않아도 되는 설계가 더 간단·효율적이라고 권하면서, 기반 인터페이스만 받는 시스템에 객체를 보낸 뒤 돌려받을 때처럼 숨겨진 원래 타입을 복원해야 할 수도 있다고 설명한다.

4.5.3 자원 누수 피하기

출처: 책 115-117쪽 / 원문 PDF 113-115쪽

원문의 벌거벗은 포인터 예제에는 Smiley가 mouth 삭제를 잊거나, read_shape() 사용자가 반환 포인터 삭제를 잊거나, 포인터 컨테이너 소유자가 각 객체 삭제를 잊을 위험이 있다. new 다음에 예외를 던지거나 조기 return하면 마지막 delete까지 도달하지 못해 누수가 생긴다.

unique_ptr로 소유권을 표현한 Smiley

코드 출처: 책 116쪽 / 원문 PDF 114쪽

C++ 원문

```
class Smiley : public Circle {
    // ...
private:
    vector<unique_ptr<Shape>> eyes; // 눈 두 개
    unique_ptr<Shape> mouth;
};
```

코드의 의미

눈과 입의 삭제 책임을 unique_ptr 멤버로 옮겨 자동 해제를 이끈다.

원문 실행 결과

[원문에 실행 결과 미제시]

원문 주의 사항

- // ...는 원문의 생략 표시다. 원문 자체가 멤버 표현의 변경 부분만 보여 준다.

삭제 책임이 있다면 표준 unique_ptr로 소유권을 표현한다. Smiley의 eyes를 vector<unique_ptr<Shape>>, mouth를 unique_ptr<Shape>로 바꾸면 멤버들의 소멸이 자원을 반납해 별도 Smiley 소멸자를 정의할 필요가 없다. 원문은 이 방법의 효율이 올바른 벌거벗은 포인터 코드와 동등하다고 설명한다.

read_shape()도 unique_ptr<Shape>를 반환하고 user()는 그 포인터들을 vector에 보관한다. unique_ptr가 스코프를 벗어날 때 소유 객체를 삭제한다. 이 버전을 사용하려면 draw_all()·rotate_all()도 vector<unique_ptr<Shape>>&를 받도록 바꿔야 한다는 원문의 조건을 함께 유지한다.

4.6 조언

출처: 책 117-118쪽 / 원문 PDF 115-116쪽

- 아이디어를 코드로 직접 표현하고, 간단한 개념에는 구체 클래스를 사용한다. 성능이 중요한 경우에도 구체 타입을 고려한다.

- 생성자가 초기화를 맡고 상태를 바꾸지 않는 멤버 함수는 const로 선언한다. 표현에 직접 접근해야 하는 함수는 멤버로, 대칭적 연산은 비멤버로 만든다.
- 자원 획득에는 해제가 짝을 이루게 하고 RAII 자원 핸들을 쓴다. 컨테이너에는 초깃값 목록 생성자를 제공한다.
- 표현과 인터페이스를 완전히 분리하려면 추상 클래스를 사용하고 다형 객체는 포인터·참조로 접근한다.
- 계층 관계를 표현할 때 인터페이스 상속과 구현 상속을 구별한다. 가상 함수가 있는 클래스에는 가상 소멸자를 두고 override로 의도를 드러낸다.
- 불가피한 계층 탐색에는 dynamic_cast를 사용한다. 찾지 못한 타입이 대안이면 포인터, 실패이면 참조를 사용한다. 소유권은 unique_ptr·shared_ptr로 표현해 해제 누락을 피한다.

중요 내용 요약

- 구체 타입은 표현을 정의에 포함해 직접 사용·복사·이동을 지원한다.
- 생성자와 소멸자의 결합이 RAII를 이루며 자원 수명을 객체 수명에 연결한다.
- 추상 클래스는 순수 가상 인터페이스이고 다형 사용자는 구체 구현을 몰라도 된다.
- 기반 포인터로 삭제할 다형 계층에는 가상 소멸자가 필요하다.
- 계층의 인터페이스 상속·구현 상속과 포인터의 비소유·소유 책임을 구별한다.

혼동하기 쉬운 내용

구체 타입과 추상 타입

출처: 책 93-106쪽 / 원문 PDF 91-104쪽

기준	구체 타입	추상 타입
표현	타입 정의에 존재	사용자에게 구현을 완전히 감춤
사용	직접 객체·지역 변수·복사·이동	인터페이스의 포인터·참조
변경 영향	표현 변경 시 다시 컴파일	인터페이스가 유지되면 사용자 재컴파일 없이 구현 변경
예	complex, Vector	Container, Shape

상속의 두 역할

출처: 책 112쪽 / 원문 PDF 110쪽

구분	원문의 의미	예
인터페이스 상속	기반 타입을 쓰는 곳에 파생 객체를 사용	Container, Shape
구현 상속	기반의 데이터·함수로 파생 구현 단순화	Smiley의 Circle 생성자·draw 활용

dynamic_cast의 두 실패 표현

출처: 책 114-115쪽 / 원문 PDF 112-113쪽

대상	타입 불일치	선택 기준
포인터	nullptr 반환	다른 타입이 합법적인 대안
참조	bad_cast 예외	기대 타입이 아니면 실패

장 마무리 복습

- complex의 표현·생성자·const 접근자·비멤버 연산의 관계를 복습한다.
- Vector의 자원 획득부터 스코프 종료·배열 해제까지 흐름을 다시 확인한다.
- Container→Vector_container와 Shape→Circle→Smiley의 관계를 원문 순서로 정리한다.
- 가상 소멸자와 unique_ptr가 각각 동적 타입의 소멸 선택과 소유권에 맡는 역할을 구별한다.

제5장. 필수적인 연산

출처: 책 119-139쪽 / 원문 PDF 117-137쪽

학습 내용 개요

- 생성·소멸·복사·이동을 관련 연산의 집합으로 설계한다.
- 기본 멤버별 복사의 한계와 자원 소유권 이전을 이해하고, 비교·컨테이너·입출력·리터럴·교환·해시의 관례를 정리한다.

설명의 흐름

1. 생애 주기 연산의 일관성→변환과 기본값→복사 의미→이동과 스코프 간 자원 이전→관례적인 연산

5.1 소개

출처: 책 119쪽 / 원문 PDF 117쪽

초기화·대입·복사·이동은 프로그램의 기초 동작이다. 원문은 이러한 연산에 관례적인 의미가 있으며 이를 임의로 무시해서는 안 된다고 설명한다. 타입을 정의할 때는 객체 생성에서 사용·전달·정리까지의 동작이 일관되게 연결되도록 설계한다.

5.1.1 필수적인 연산

출처: 책 120-122쪽 / 원문 PDF 118-120쪽

생성자·소멸자·복사·이동은 서로 관련된 연산이다. 소멸자가 동적 저장 공간이나 락을 해제하는 등 특별한 일을 한다면 복사·이동에도 별도 정의가 필요할 가능성이 크다. 원문은 일반 생성자, 기본 생성자, 복사·이동 생성자, 복사·이동 대입, 소멸자를 한 집합으로 제시한다.

함께 설계할 생애 주기 연산

코드 출처: 책 120쪽 / 원문 PDF 118쪽

C++ 원문

```
class X {
public:
    X(Sometype); // "일반적인 생성자": 객체 생성
    X(); // 기본 생성자
    X(const X&); // 복사 생성자
    X(X&&); // 이동 생성자
    X& operator=(const X&); // 복사 대입: 대상을 정리하고 복사
    X& operator=(X&&); // 이동 대입: 대상을 정리하고 이동
    ~X(); // 소멸자: 정리 작업
    // ...
};
```

코드의 의미

생성·복사·이동·대입·정리를 서로 관련된 연산 집합으로 제시한다.

원문 실행 결과

[원문에 실행 결과 미제시]

원문 주의 사항

- // ...는 원문의 생략 표시다.

객체는 대입 원본, 초깃값, 함수 인자, 함수 반환 값, 예외로서 복사·이동될 수 있다. 대입에는 대입 연산자를 사용하고 다른 경우에는 생성자가 관련되지만, 대상의 저장 공간에 직접 초기화해 생성자 호출을 생략할 수 있다. 이 복사 생략은 객체 전달의 최적화다.

컴파일러는 일반 생성자를 제외한 특수 멤버를 필요할 때 생성하며 =default로 기본 구현을 명시할 수 있다. 원문은 일부 기본 함수를 명시하면 나머지 기본 함수가 생성되지 않는다고 설명하므로, 연산 집합 전체를 함께 고려한다. 포인터가 delete할 자원을 가리킨다면 단순한 멤버별 복사는 적합하지 않다. 반대로 비소유 포인터라면 그 사실도 독자가 알아야 한다.

기반 클래스 복사의 사용 금지

코드 출처: 책 122쪽 / 원문 PDF 120쪽

C++ 원문

```
class Shape {
public:
    Shape(const Shape&) =delete; // 복사 연산 없음
    Shape& operator=(const Shape&) =delete;
    // ...
};

void copy(Shape& s1, const Shape& s2)
{
    s1 = s2; // 에러: Shape의 복사 연산이 제거됨
}
```

코드의 의미

적합하지 않은 복사 연산을 =delete로 금지한다.

원문 실행 결과

원문은 s1 = s2에서 컴파일 시간 에러가 발생한다고 명시한다.

기본 의미가 충분한 클래스는 컴파일러가 멤버의 기본 생성·복사·이동·소멸을 조합하도록 두고, 특별한 의미가 필요하다면 필수 연산을 함께 정의한다. =delete는 어떤 연산의 사용을 금지하며 사용하면 컴파일 시간 에러가 난다. 원문은 멤버별 복사가 적합하지 않은 Shape 기반 클래스를 예로 든다. 특수 멤버 외의 함수에도 =delete를 사용할 수 있다.

5.1.2 변환

출처: 책 122-123쪽 / 원문 PDF 120-121쪽

인자 하나를 받는 생성자는 그 인자 타입에서 클래스 타입으로의 변환을 정의한다. complex에 double을 주어 {값,0}을 만드는 암묵 변환은 유용할 수 있다. 반면 int 7을 Vector로 암묵 변환해 요소 7개짜리 객체를 만드는 것은 원하지 않는 결과일 수 있다.

explicit 생성자로 암묵 변환 제한

코드 출처: 책 123쪽 / 원문 PDF 121쪽

C++ 원문

```
class Vector {
public:
    explicit Vector(int s); // int에서 Vector로 암묵적인 변환 불가
    // ...
};

Vector v1(7); // OK: v1은 7개 요소를 포함
Vector v2 = 7; // 에러: int에서 Vector로 암묵적인 변환 불가
```

코드의 의미

명시적 생성은 허용하고 의도하지 않은 int→Vector 암묵 변환은 거부한다.

원문 실행 결과

원문은 v1 생성은 가능하고 v2는 에러라고 명시한다.

explicit은 명시적 변환만 허용하게 한다. 이때 Vector v1(7)은 가능하지만 Vector v2 = 7은 금지된다. 원문은 특별한 이유가 없다면 한 인자를 받는 생성자를 explicit으로 선언해 Vector 쪽의 관례를 따르라고 권한다.

5.1.3 멤버 초깃값

출처: 책 124쪽 / 원문 PDF 122쪽

데이터 멤버 선언에서 기본 멤버 초깃값을 지정할 수 있다. 원문의 complex는 re와 im을 각각 0으로 지정하고, 두 인자 생성자는 둘 다 대체하며 한 인자 생성자는 re만 지정한다. 생성자가 별도 값을 주지 않은 멤버에는 기본값을 사용한다. 이를 통해 생성자 코드를 줄이고 실수로 멤버를 초기화하지 않는 일을 막는다.

5.2 복사와 이동

출처: 책 124-125쪽 / 원문 PDF 122-123쪽

원문은 사용자 정의 타입과 내장 타입의 기본 복사 의미를 멤버별 복사로 설명한다. 두 double로 표현되는 complex에서는 초기화와 대입 시 두 성분을 복사하면 독립된 같은 값이 되어 적합하다. 그러나 자원 핸들이거나 추상 타입이면 이런 의미가 적절하지 않은 경우가 많다. 클래스 설계 시 복사가 무엇을 뜻하는지 반드시 고려한다.

5.2.1 컨테이너 복사

출처: 책 125-127쪽 / 원문 PDF 123-125쪽

Vector의 elem 포인터와 sz만 복사하면 두 Vector가 같은 요소를 가리키게 된다. 한 벡터의 요소 변경이 다른 벡터에도 보이고, 자원을 각 객체가 책임진다는 의미와 충돌한다. 원문의 bad_copy()는 이 문제를 도식으로 보여 준다. 소멸자가 있는 자원 소유 클래스라면 기본 복사가 적절한지 특히 확인해야 한다.

자기 저장 공간을 갖는 Vector 복사 생성자

코드 출처: 책 126쪽 / 원문 PDF 124쪽

C++ 원문

```
Vector::Vector(const Vector& a) // 복사 생성자
    :elem{new double[a.sz]}, // 요소를 저장할 공간 할당
    sz{a.sz}
{
    for (int i=0; i!=sz; ++i) // 요소를 복사
        elem[i] = a.elem[i];
}
```

코드의 의미

원본과 독립된 배열에 요소를 복사한다.

동작 과정

1. 원본 크기의 배열을 할당한다.
2. 각 요소를 새 배열에 복사한다.

원문 실행 결과

[원문에 실행 결과 미제시]

기존 자원을 교체하는 복사 대입

코드 출처: 책 127쪽 / 원문 PDF 125쪽

C++ 원문

```
Vector& Vector::operator=(const Vector& a) // 복사 대입
{
    double* p = new double[a.sz];
    for (int i=0; i!=a.sz; ++i)
        p[i] = a.elem[i];
    delete[] elem; // 예전 요소 삭제
    elem = p;
    sz = a.sz;
    return *this;
}
```

코드의 의미

기존 배열을 소유한 대상에 복사 결과를 대입한다.

동작 과정

1. 새 배열을 먼저 할당하고 요소를 복사한다.
2. 이전 배열을 해제한 뒤 포인터와 크기를 바꾼다.
3. 대상 객체의 참조를 반환한다.

원문 실행 결과

[원문에 실행 결과 미제시]

복사 생성자는 원본 크기의 공간을 새로 할당하고 각 요소를 복사해, 두 Vector가 각각 자기 요소의 복사본을 갖게 한다. 복사 대입은 대상에 이미 있던 자원을 정리해야 하므로 새 배열 할당·요소 복사 후 이전 배열을 해제하고 포인터·크기를 교체한다. this는 멤버 함수를 호출한 객체를 가리키며 *this는 결과로 대상 객체 자체를 돌려준다.

5.2.2 컨테이너 이동

출처: 책 127-131쪽 / 원문 PDF 125-129쪽

큰 객체의 복사 비용은 클 수 있다. 참조 인자로 입력 복사는 피할 수 있지만 사라질 지역 객체를 참조로 반환할 수는 없다. 원문의 벡터 덧셈은 지역 res를 만들어 반환한다. 덧셈을 연속하면 더 이상 사용하지 않을 중간 결과를 복사하는 비용이 문제다. 원하는 것은 원소의 또 다른 복사본이 아니라 계산 결과를 함수 밖으로 가져오는 것이다.

지역 결과를 값으로 반환하는 벡터 덧셈

코드 출처: 책 127-128쪽 / 원문 PDF 125-126쪽

C++ 원문

```
Vector operator+(const Vector& a, const Vector& b)
{
    if (a.size()!=b.size())
        throw Vector_size_mismatch{};
    Vector res(a.size());

    for (int i=0; i!=a.size(); ++i)
        res[i]=a[i]+b[i];
    return res;
}
```

코드의 의미

같은 크기 벡터의 요소별 합을 지역 결과에 만들고 값으로 반환한다.

동작 과정

1. 크기가 다르면 예외를 던진다.
2. 원소별 합을 res에 저장한 뒤 반환한다.

원문 실행 결과

[원문에 실행 결과 미제시]

원문 주의 사항

- Vector_size_mismatch의 정의는 이 코드 블록에 제시되지 않는다. 임의 정의를 추가하지 않았다.

배열 소유권을 가져오는 이동 생성자

코드 출처: 책 129쪽 / 원문 PDF 127쪽

C++ 원문

```
Vector::Vector(Vector&& a)
    :elem{a.elem}, // 인자로부터 요소를 가져옴
    sz{a.sz}
{
    a.elem = nullptr; // 이제 요소를 포함하지 않음
    a.sz = 0;
}
```

코드의 의미

배열을 새로 복사하지 않고 원본의 핸들을 가져온다.

동작 과정

1. 원본 포인터와 크기로 대상을 초기화한다.

2. 원본 포인터를 nullptr로, 크기를 0으로 바꾼다.

원문 실행 결과

[원문에 실행 결과 미제시]

원문 주의 사항

- 이동된 원본도 문제없이 소멸할 수 있어야 한다.

이동 생성자는 원본 elem과 sz를 가져오고 원본을 nullptr·0 상태로 만든다. &&는 rvalue 참조를 나타내며, 원문은 함수 반환 결과처럼 이동 가능한 값을 표현하는 데 사용한다고 설명한다. 이동은 원본에서 값을 제거하므로 이동 생성자는 const 인자를 받지 않는다. 이동 대입도 비슷한 목적을 갖는다.

이동 후 원본은 소멸자를 문제없이 호출할 수 있어야 하고, 일반적으로 다시 대입할 수도 있어야 한다. 프로그래머가 앞으로 값을 사용하지 않을 것을 알면 std::move로 그 의도를 나타낼 수 있다. move() 자체는 아무것도 이동시키지 않고 이동 가능하게 만드는 rvalue 참조를 반환하는 일종의 타입 변환이다. 원문은 이동 뒤의 값을 사용하지 않는 것이 좋다고 주석으로 명시한다.

객체 초기화에서는 복사 생략으로 실제 이동 생성 호출도 줄어들 수 있다. 반면 기존 객체에 대한 대입의 복사·이동은 암묵적으로 없애기 어려우므로 이동 대입은 성능 면에서 중요한 역할을 한다.

5.3 자원 관리

출처: 책 131-133쪽 / 원문 PDF 129-131쪽

생성자·복사·이동·소멸을 정의하면 객체가 소유하는 자원의 생애를 제어할 수 있다. 이동은 복사할 수 없거나 복사하기 싫은 객체를 낮은 비용으로 다른 스코프로 옮긴다. 원문의 예에서 thread는 복사할 수 없고, double 100만 개의 Vector는 복사를 피하고 싶은 대상이다.

스레드와 큰 Vector의 자원 이전

코드 출처: 책 131쪽 / 원문 PDF 129쪽

C++ 원문

```
std::vector<thread> my_threads;

Vector init(int n)
{
    thread t {heartbeat}; // heartbeat를 동시적으로(각자 다른 스레드로) 실행
    my_threads.push_back(std::move(t)); // t를 my_threads로 이동(13.2.2절)
    // ... 다른 초기화 수행...

    Vector vec(n);
    for (int i=0; i!=vec.size(); ++i)
        vec[i] = 777;
    return vec; // vec를 init() 밖으로 이동
}

auto v = init(1'000'000); // heartbeat 을 시작하고 v를 초기화
```

코드의 의미

복사 불가능한 thread와 큰 Vector를 이동으로 전달하는 원문 예다.

동작 과정

1. 스레드를 생성해 컨테이너로 이동한다.

2. Vector 요소를 777로 초기화하고 값을 반환한다.

원문 실행 결과

[원문에 실행 결과 미제시]

원문 주의 사항

- heartbeat의 정의와 다른 초기화 내용은 원문 코드에 없다.

자원 핸들에는 포인터 자체도 감춰지게 할 수 있으며 추가 부담 없이 단순하고 유지 보수하기 쉬운 코드를 얻는다. 강력한 자원 안전성은 일반적인 의미의 자원 모두에 대해 누수를 막는 것이다. 메모리뿐 아니라 락·소켓·파일·스레드 핸들도 자원이다. 오래 실행되는 시스템에서는 누수뿐 아니라 필요 이상 오래 보유하는 것도 문제다.

원문은 가비지 컬렉션을 전역적 메모리 관리로 설명하고, 더 명확하고 일반적이며 지역적인 관리 방법이 없을 때 최후 수단으로 고려한다는 저자의 입장을 제시한다. 먼저 각 자원의 소유자가 특정 스코프에 존재하고 종료 시 해제하도록 RAII를 체계적으로 사용한다. 이동·스마트 포인터는 스코프 사이 자원 이전을, 공유 포인터는 공유 소유권을 표현한다.

표준 라이브러리는 string·vector·map의 메모리, ifstream·ofstream의 파일, thread의 스레드, lock_guard·unique_lock의 락, unique_ptr·shared_ptr의 객체에 RAII를 적용한다. 그 결과 보통의 코드에서 관리가 두드러지지 않으면서 소유 시간을 줄이는 암묵적 자원 관리가 가능하다.

5.4 관례적인 연산

출처: 책 133쪽 / 원문 PDF 131쪽

많은 프로그래머와 표준 라이브러리는 특정 연산의 관례적인 뜻을 가정한다. 새로운 타입에서도 비교, 크기·범위 접근, 입출력, 리터럴, swap, hash의 의미가 관례를 따르도록 설계한다.

5.4.1 비교

출처: 책 133-134쪽 / 원문 PDF 131-132쪽

상등 비교 ==·!=는 복사 의미와 밀접하다. 복사한 뒤 원본과 복사본은 같아야 한다. ==를 정의하면 !=도 정의하고 a!=b가 !(a==b)를 뜻하게 한다. 순서 비교 연산도 서로 일관된 의미를 가져야 한다. 원문은 두 피연산자를 동등하게 취급하도록 이항 연산자를 해당 클래스와 같은 네임스페이스의 독립 함수로 정의하는 방식을 권한다.

[원문 확인 필요] 인쇄 134쪽 비교 규칙 목록은 첫 항목과 셋째 항목에 모두 !(b<a)를 인쇄한다. 이 요약에서는 외부 지식으로 그 식을 수정하거나 새로운 식으로 대체하지 않고, 비교 연산 간의 일관성이라는 원문 설명을 보존했다.

5.4.2 컨테이너 연산

출처: 책 134-135쪽 / 원문 PDF 132-133쪽

특별한 이유가 없다면 표준 라이브러리 컨테이너 설계를 따른다. 컨테이너를 필수 연산을 갖춘 자원 핸들로 구현하고 size()로 요소 개수를 알게 한다. 알고리즘은 인덱스 대신 반복자 쌍으로 정의된 시퀀스를 사용한다.

begin()은 첫 요소, end()는 마지막 요소의 다음을 가리킨다. 반복자는 포인터처럼 ++로 다음으로 이동하고 *로 값을 읽고 쓴다. sort(v.begin(),v.end())처럼 반복자 쌍을 표준 알고리즘에 전달한다. 구간 for도 같은 begin()·end() 범위를 사용하며 명시적 반복자 루프와 거의 같은 방식이다.

5.4.3 입력과 출력 연산

출처: 책 135-136쪽 / 원문 PDF 133-134쪽

정수 쌍에 대한 <<·>>는 각각 왼쪽·오른쪽 시프트다. iostream에 대해서는 출력·입력 연산으로 사용한다. 원문은 자세한 입출력 설명을 10장으로 연결하며, 이 절에서는 같은 기호의 관례적인 사용 맥락을 구별한다.

5.4.4 사용자 정의 리터럴

출처: 책 136-137쪽 / 원문 PDF 134-135쪽

사용자 정의 타입에도 내장 타입처럼 편리한 리터럴을 제공할 수 있다. 적절한 접미사의 의미를 리터럴 연산자로 정의하면 접미사 앞 리터럴을 필요한 타입으로 변환한다. 원문은 문자열 s, 시간 단위 s, 허수 i의 예를 들고, 사용하려면 적절한 표준 헤더와 네임스페이스를 가져와야 한다고 설명한다.

허수 접미사를 정의하는 리터럴 연산자

코드 출처: 책 137쪽 / 원문 PDF 135쪽

C++ 원문

```
constexpr complex<double> operator""i(long double arg) // 허수
{
    return {0,arg};
}

complex<double> z = 2.7182818+6.283185i;
```

코드의 의미

부동소수점 리터럴에 i 접미사를 붙여 complex<double> 값을 만든다.

동작 과정

1. 리터럴 연산자가 실수부 0, 허수부 arg인 값을 반환한다.

원문 실행 결과

[원문에 실행 결과 미제시]

operator""는 리터럴 연산자를, 그 뒤 i는 접미사를 나타낸다. long double 인자는 이 접미사가 부동소수점 리터럴에 적용됨을, complex<double> 반환 타입은 만들어질 리터럴의 타입을 나타낸다. 원문의 chrono 접미사는 h·min·s·ms·us·ns이고 complex에는 i·il·if가 열거된다. 원문 표의 string_view 행은 string_literals 및 s로 인쇄되어 있다. [원문 확인 필요] 이를 임의의 다른 이름으로 고치지 않았다.

5.4.5 swap()

출처: 책 137-138쪽 / 원문 PDF 135-136쪽

sort()를 비롯한 많은 알고리즘은 두 객체 값을 교환하는 swap()을 사용하며 빠르고 예외를 던지지 않는다고 가정한다. std::swap(a,b)는 세 번의 이동으로 정의된다. 복사 비용이 큰 타입을 정렬·교환해야 한다면 이동 연산 또는 swap(), 혹은 둘 다 제공하는 것이 좋다. 표준 컨테이너와 string은 빠른 이동을 제공한다.

5.4.6 hash<>

출처: 책 138쪽 / 원문 PDF 136쪽

unordered_map<K,V>는 키 타입 K와 값 타입 V를 갖는 해시 테이블이다. 사용자 타입 X를 키로 사용하려면 hash<X>를 정의해야 한다. 표준 라이브러리는 std::string을 포함한 일반적인 타입의 해시를 제공한다. 이 절은 해시 구현을 새로 전개하지 않고 11.5절로 연결한다.

5.5 조언

출처: 책 138-139쪽 / 원문 PDF 136-137쪽

- 생성자·대입·소멸자를 관련 집합으로 설계하고 필수 연산은 모두 정의하거나 모두 기본에 맡긴다. 기본 구현이 충분하면 직접 다시 쓰지 않는다.
- 포인터를 가진 클래스나 소멸자가 있는 클래스는 사용자 정의 또는 제거된 복사·이동이 필요할 수 있다. 부적절한 기본 복사는 재정의하거나 막는다.
- 한 인자 생성자는 기본적으로 explicit으로 두고 합리적인 기본 멤버 값은 선언에 지정한다.
- 컨테이너는 값을 반환하되 이동으로 효율을 확보하고 큰 피연산자는 const 참조로 받는다.
- 어떤 자원도 누수되지 않도록 강력한 자원 안전성을 제공한다. 연산자는 관례에 맞게 오버로드하며 표준 컨테이너 설계를 따른다.

중요 내용 요약

- 기본 멤버별 복사가 자원 소유 의미에 맞는지 먼저 검토한다.
- 복사 생성은 새 객체를 만들고 복사 대입은 대상의 기존 자원도 정리한다.
- 이동은 결과·자원 전달의 비용을 줄이지만 이동된 원본도 소멸 가능한 상태여야 한다.
- std::move 자체는 이동을 수행하지 않는다.
- 자원 관리는 메모리 외의 락·파일·소켓·스레드도 포함한다.

혼동하기 쉬운 내용

복사와 이동

출처: 책 125-131쪽 / 원문 PDF 123-129쪽

기준	Vector 복사	Vector 이동
저장 공간	새 배열을 할당하고 요소 복사	기존 배열 포인터를 가져옴
원본	자기 요소를 계속 가짐	예제에서는 nullptr·크기 0
입력	const Vector&	Vector&&
결과	독립된 같은 값	자원의 소유권 이전

생성자 제어 표기

출처: 책 121-124쪽 / 원문 PDF 119-122쪽

표기	원문의 목적
=default	컴파일러 기본 구현을 명시
=delete	함수 사용을 컴파일 시간에 금지
explicit	한 인자 생성자의 암묵 변환 제한
멤버 기본 초깃값	생성자가 값을 지정하지 않은 멤버 초기화

장 마무리 복습

- 생성·복사·이동·대입·소멸을 한 집합으로 설계하는 이유를 재확인한다.

- 자원 핸들에 멤버별 복사를 적용했을 때의 공유와, 독립 배열 복사의 차이를 복습한다.
- 함수 결과의 값 반환, 이동, 복사 생략을 연결한다.
- 비교·반복자 범위·입출력·리터럴·swap·hash가 기존 코드의 관례와 맞아야 함을 정리한다.

제6장. 템플릿

출처: 책 141-162쪽 / 원문 PDF 138-159쪽

학습 내용 개요

- 타입·값을 파라미터화한 클래스와 함수
- 함수 객체·람다로 알고리즘 동작 지정
- 인자 추론·별칭·컴파일 시간 선택

설명의 흐름

1. 일반적인 타입 표현 → 인자 제한과 추론 → 일반적인 연산 → 템플릿 작성 메커니즘

6.1 소개

출처: 책 141쪽 / 원문 PDF 138쪽

템플릿은 타입이나 값의 집합을 파라미터화한 클래스 또는 함수다. 특정 타입에 종속되지 않는 벡터와 같은 일반적인 개념을 표현하고, 인자를 지정하여 특정 타입이나 함수를 만들어 낸다. 요소를 저장한다는 벡터의 개념과 요소가 double이라는 사실을 분리하는 것이 출발점이다.

6.2 파라미터화된 타입

출처: 책 142-145쪽 / 원문 PDF 139-142쪽

template<typename T>는 앞으로 선언할 클래스의 타입 파라미터를 T로 지정한다. 이는 모든 타입 T에 대해라는 뜻이다. 타입 파라미터 자리에 class를 쓰는 오래된 표기도 같은 의미다. 클래스의 멤버를 밖에서 정의할 때도 template 접두사와 Vector<T>를 이용한다.

타입 파라미터로 일반화한 Vector

코드 출처: 책 142-143쪽 / 원문 PDF 139-140쪽

C++ 원문

```
template<typename T>
class Vector {
private:
    T* elem;    // elem은 T 타입 요소 sz개를 포함하는 배열을 가리킴
    int sz;
public:
    explicit Vector(int s);    // 생성자: 불변 조건을 조성하고, 자원을 획득
    ~Vector() { delete[] elem; } // 소멸자: 자원 해제

    // ... 복사와 이동 연산 ...

    T& operator[](int i);              // const가 아닌 Vector를 위한 연산자
    const T& operator[](int i) const;  // const Vector를 위한 연산자(4.2.1절)
    int size() const { return sz; }
};

template<typename T>
Vector<T>::Vector(int s)
{
    if (s<0)
        throw Negative_size{};
    elem = new T[s];
    sz = s;
}

template<typename T>
const T& Vector<T>::operator[](int i) const
{
    if (i<0 || size()<=i)
        throw out_of_range{"Vector::operator[]"};
    return elem[i];
}
```

코드의 의미

double을 타입 파라미터 T로 바꾸어 서로 다른 요소 타입에 적용할 수 있게 한 클래스와 멤버 정의다.

동작 과정

1. 생성자는 음수 크기를 거부하고 T의 배열을 확보한다.
2. 인덱싱 함수는 인덱스 구간을 확인한 후 해당 요소의 const 참조를 반환한다.

원문 실행 결과

[원문에 실행 결과 미제시]

원문 주의 사항

- 원문의 복사·이동 연산 생략 주석을 보존했다. 클래스 선언과 뒤의 멤버 정의 두 코드 영역을 함께 수록했다.

출처: 책 143-144쪽 / 원문 PDF 140-141쪽

Vector<char>, Vector<string>, Vector<list<int>>는 서로 다른 요소 타입의 벡터다. 중첩된 템플릿 인자 끝의 >>는 입력 연산자가 아니다. 사용자 정의 Vector를 구간 for로 순회하려면 첫 요소를 나타내는 begin()과 마지막 요소 다음을 나타내는 end()를 알맞게 제공해야 한다.

출처: 책 144-145쪽 / 원문 PDF 141-142쪽

템플릿은 컴파일 시간 메커니즘이므로 같은 일을 손으로 작성한 코드에 비해 실행 시간의 추가 부담이 없다. 템플릿과 템플릿 인자의 집합을 인스턴스화 또는 특수화라고 부른다. 사용하는 각 인스턴스의 코드는 컴파일 후반의 인스턴스화 시간에 생성되며 타입 검사를 받는다. 따라서 타입 안전성은 보장되지만 검사 시점이 늦다.

6.2.1 제한된 템플릿 인자(C++20)

출처: 책 145쪽 / 원문 PDF 142쪽

템플릿에 의미가 있으려면 인자가 특정 성질을 만족해야 한다. Vector가 복사 연산을 제공하려면 요소도 복사할 수 있어야 한다. Element와 같은 컨셉은 이 요구 사항을 확인하는 술어다. 컨셉을 지정한 인자는 제한된 인자, 이러한 인자를 사용하는 템플릿은 제한된 템플릿이다. C++20 이전 코드에서는 필요한 조건을 문서화한다고 설명한다.

[원문 확인 필요] 원문 예는 Vector<int>를 OK, Vector<thread>를 표준 스레드 복사 불가에 의한 에러로 표시한다. 다만 int 예의 주석은 "int 는 복사할 수 없음"으로 인쇄되어 있어 본문의 복사 가능 조건 및 OK 표시와 함께 원문 확인이 필요하다. 이 주석을 반대로 고쳐 옮기지 않았다.

6.2.2 값 템플릿 인자

출처: 책 146쪽 / 원문 PDF 143쪽

템플릿은 타입뿐 아니라 값도 인자로 받는다. Buffer의 T는 요소 타입, N은 크기를 나타낸다. 이 방식으로 자유 저장 영역을 사용하지 않는 전역 버퍼와 스택의 지역 버퍼를 만들 수 있다. **값 템플릿 인자는 반드시 상수 표현식이어야 한다.**

값 템플릿 인자를 받는 Buffer

코드 출처: 책 146쪽 / 원문 PDF 143쪽

C++ 원문

```
template<typename T, int N>
struct Buffer {
    using value_type = T;
    constexpr int size() { return N; }
    T[N];
    // ...
};
```

코드의 의미

타입 인자와 크기 값 인자를 함께 받아 동적 메모리 없이 크기를 정하는 버퍼의 원문 예다.

동작 과정

1. value_type과 size()를 통해 템플릿 인자에 읽기 전용으로 접근한다.

원문 실행 결과

[원문에 실행 결과 미제시]

원문 주의 사항

- [원문 확인 필요] 원문 배열 선언은 변수명 없이 T[N];로 인쇄되어 있다. 수정하지 않았다.
- 원문 자체에 // ... 생략이 있다.

6.2.3 템플릿 인자 추론

출처: 책 146-149쪽 / 원문 PDF 143-146쪽

make_pair()는 전달받은 함수 인자로 반환할 pair의 템플릿 인자를 추론한다. C++17에서는 생성자 인자를 통해 클래스 템플릿 인자도 추론할 수 있다. 원문 Vector에서는 초깃값 목록의 요소 타입이나 복사할 Vector의 타입에서 요소 타입을 알아내지만, 크기만 지정하는 생성에서는 요소 타입을 명시해야 한다.

출처: 책 148쪽 / 원문 PDF 145쪽

추론은 중복을 줄이지만 항상 의도한 타입을 만들지는 않는다. C 스타일 문자열 리터럴을 요소로 전달하면 string이 아닌 const char*가 추론될 수 있다. string 리터럴을 원하면 접미사 s를 사용한다. 초깃값 목록의 요소 타입이 서로 일치하지 않으면 유일한 요소 타입을 추론하지 못한다.

출처: 책 148-149쪽 / 원문 PDF 145-146쪽

생성자 인자 한 쌍이 반복자라는 사실과 반복자의 값 타입을 컴파일러가 추론하지 못하는 경우에는 추론 가이드를 지정할 수 있다. 원문 Vector2는 반복자의 value_type에서 요소 타입을 알아내게 한다. 다만 추론 가이드의 효과에는 미묘한 부분이 있어, 가능하다면 가이드가 필요하지 않게 클래스 템플릿을 설계하라고 조언한다.

6.3 파라미터화된 연산

출처: 책 149쪽 / 원문 PDF 146쪽

타입이나 값에 따라 달라지는 연산은 함수 템플릿, 함수 객체, 람다 표현식으로 나타낼 수 있다. 함수 객체는 데이터도 보유하면서 함수처럼 호출되는 객체이고, 람다는 이러한 함수 객체를 간결하게 표기하는 방법이다. 표준 라이브러리는 타입과 알고리즘 모두를 파라미터화하는 데 이 수단들을 사용한다.

6.3.1 함수 템플릿

출처: 책 150쪽 / 원문 PDF 147쪽

sum()의 시퀀스 타입과 누산기 타입은 서로 독립적이다. 호출자가 초기 누산기 값을 0으로 주는지 0.0으로 주는지에 따라 누산기의 타입이 달라지므로, 같은 int 벡터를 int 또는 double 덧셈으로 처리할 수 있다. 원문은 double 누산기로 int의 최댓값보다 큰 합을 처리할 수 있다고 설명한다. 이 sum()은 표준 accumulate()의 단순화다.

시퀀스와 누산기 타입을 분리한 sum

코드 출처: 책 150쪽 / 원문 PDF 147쪽

C++ 원문

```
template<typename Sequence, typename Value>
Value sum(const Sequence& s, Value v)
{
    for (auto x : s)
        v+=x;
    return v;
}
```

코드의 의미

구간 for로 순회 가능한 시퀀스의 값을 호출자가 지정한 타입과 초깃값의 누산기에 더한다.

동작 과정

1. 각 요소를 v에 누적하고 v를 반환한다.
2. Sequence와 Value는 함수 인자에서 추론된다.

원문 실행 결과

[원문에 실행 결과 미제시]

함수 템플릿은 클래스의 멤버가 될 수 있지만 virtual 멤버는 될 수 없다. 컴파일러가 프로그램에 존재하는 모든 인스턴스화를 알 수 없어 가상 함수 테이블을 만들 수 없기 때문이다.

6.3.2 함수 객체

출처: 책 151-152쪽 / 원문 PDF 148-149쪽

operator()를 정의한 객체는 함수처럼 호출할 수 있다. Less_than은 비교 기준을 객체 안에 저장하고 그 값보다 작은지를 판단한다. **술어**는 호출할 수 있고 결과로 true 또는 false를 반환하는 무엇이다. count()는 컨테이너의 각 요소에 술어를 적용해 참인 요소의 개수를 센다.

데이터를 보유하는 함수 객체

코드 출처: 책 151쪽 / 원문 PDF 148쪽

C++ 원문

```
template<typename T>
class Less_than {
    const T val;    // 비교 대상
public:
    Less_than(const T& v) :val{v} { }
    bool operator()(const T& x) const { return x<val; }    // 연산자 호출
};
```

코드의 의미

비교 대상 val을 보유하면서 함수처럼 호출할 수 있는 객체를 정의한다.

동작 과정

1. 생성자가 비교 대상을 저장한다.
2. 호출 연산자는 전달된 x가 val보다 작은지 판단한다.

원문 실행 결과

[원문에 실행 결과 미제시]

출처: 책 152쪽 / 원문 PDF 149쪽

함수 객체는 서로 다른 비교 기준마다 별도 함수를 정의하거나 전역 변수에 기준을 저장하지 않아도 된다. 간단한 함수 객체는 인라인으로 처리하기 쉬워 간접 함수 호출보다 효율적이라고 설명한다. 일반 알고리즘에서 핵심 동작의 의미를 지정하는 이런 객체를 정책 객체라고도 부른다.

6.3.3 람다 표현식

출처: 책 153-157쪽 / 원문 PDF 150-154쪽

출처: 책 153쪽 / 원문 PDF 150쪽

람다는 사용되는 위치에서 함수 객체를 암묵적으로 만든다. 캡처 리스트 [&]는 몸체에서 사용하는 지역 이름을 참조로 접근하고, [&x]는 x만 참조로 캡처한다. 원문의 복사 캡처 표기 설명에는 [=x]가 인쇄되어 있어 [원문 확인 필요]로 남긴다. 캡처할 지역 이름이 없을 때 빈 []를 사용할 수 있다.

같은 위치에서 정의하는 람다

코드 출처: 책 153쪽 / 원문 PDF 150쪽

C++ 원문

```
void f(const Vector<int>& vec, const list<string>& lst, int x, const string& s)
{
    cout << x
        << "보다 작은 값의 개수: " << count(vec, [&](int a){ return a<x; })
        << '\n';
    cout << s
        << "보다 작은 값의 개수: " << count(lst, [&](const string& a){ return a<s; })
        << '\n';
}
```

코드의 의미

별도의 Less_than 정의 대신 람다 표현식을 count의 술어로 전달한다.

동작 과정

1. 첫 람다는 x보다 작은 정수를 판단한다.
2. 두 번째 람다는 s보다 작은 문자열을 판단한다.
3. count는 술어가 참인 요소의 수를 구하는 앞 절의 함수다.

원문 실행 결과

[원문에 실행 결과 미제시]

출처: 책 153-154쪽 / 원문 PDF 150-151쪽

람다는 간단한 동작을 한 곳에서 쓸 때 편리하다. 단순하지 않은 동작은 이름을 부여하여 목적을 명확히 하고 여러 곳에서 사용할 수 있게 한다. for_all()과 람다를 결합하면 컨테이너를 순회하는 작업과 각 요소에서 수행할 작업을 분리할 수 있다. 원문은 unique_ptr<Shape>를 참조로 받으므로 호출이 Shape의 수명에 영향을 주지 않는다고 설명한다.

출처: 책 154-155쪽 / 원문 PDF 151-152쪽

람다의 파라미터를 auto로 선언하면 다양한 타입을 받는 제네릭 람다가 된다. 원문 rotate_and_draw()는 포인터 또는 unique_ptr로 저장한 Shape를 같은 방식으로 회전하고 그린다. 일반 함수 인자에 auto를 사용할 수 없다는 원문의 당시 설명은 현재 언어 지식으로 보충하거나 수정하지 않는다.

출처: 책 155-157쪽 / 원문 PDF 152-154쪽

람다는 복잡한 초기화에도 쓰인다. 먼저 벡터를 선언하고 switch의 각 가지에서 대입하면, 의도한 값 전에 사용될 위험이 있고 초기화 코드가 다른 코드와 섞이거나 경우 하나를 놓치기 쉽다. 값을 반환하는 람다를 초깃값으로 사용하면 이 선택을 초기화 위치에 모을 수 있다. 원문은 개선 예에도 case 하나가 빠졌지만 이를 더 쉽게 알아볼 수 있다고 지적한다.

6.4 템플릿 메커니즘

출처: 책 157쪽 / 원문 PDF 154쪽

좋은 템플릿을 만드는 도구에는 타입에 의존하는 값, 타입·템플릿의 별칭, if constexpr에 의한 컴파일 시간 선택, requires 표현식에 의한 타입·표현식 속성 확인이 있다. constexpr 함수와 static_assert도 함께 사용한다. 이 기능들은 일반적인 추상화를 구성하는 기본 수단이다.

6.4.1 가변 템플릿

출처: 책 157-158쪽 / 원문 PDF 154-155쪽

이 소절은 타입 T에 따라 달라지는 상수나 변수를 정의하는 예를 다룬다. 원문은 점성 계수 viscosity<T>, 3차원 벡터 external_acceleration<T>와 대입 가능성을 나타내는 Assignable<T,T2>를 제시한다. 값은 constexpr 표현식으로 정의할 수 있고 static_assert로 해당 타입의 조건을 검사할 수 있다. 이 아이디어는 7장의 컨셉 정의로 이어진다. 원문 소절 제목을 유지하며 7.4의 가변 개수 인자 템플릿과 구별해 읽는다.

6.4.2 별칭

출처: 책 158-160쪽 / 원문 PDF 155-157쪽

using은 타입이나 템플릿의 다른 이름을 정의한다. size_t의 실제 타입은 구현마다 다르지만 같은 이름으로 접근하면 이식성을 유지할 수 있다. 컨테이너가 요소 타입을 value_type으로 제공하는 관례를 따르면, 다른 코드가 특정 컨테이너 이름 없이 요소 타입을 얻을 수 있다. 템플릿 인자의 일부를 고정하여 남은 인자만 받는 별칭 템플릿도 정의할 수 있다.

일부 인자를 지정한 템플릿 별칭

코드 출처: 책 160쪽 / 원문 PDF 157쪽

C++ 원문

```
template<typename Value>
using String_map = Map<string,Value>;

String_map<int> m;    // m의 타입은 Map<string,int>
```

코드의 의미

Map의 키 타입을 string으로 고정한 새 템플릿 이름을 제공한다.

동작 과정

1. String_map<int>에서 남아 있는 Value를 int로 지정한다.

원문 실행 결과

[원문에 실행 결과 미제시]

6.4.3 컴파일 시간 if

출처: 책 160-161쪽 / 원문 PDF 157-158쪽

C++17의 if constexpr은 타입의 성질에 따라 필요한 구현을 선택한다. 원문은 is_pod<T>를 바탕으로 단순하고 빠른 연산과 느리지만 안전한 연산을 구분한다. 선택된 경우만 인스턴스화되므로 특정 타입에 대한 지역 최적화를 할 수 있다.

컴파일 시간 if로 구현 선택

코드 출처: 책 160쪽 / 원문 PDF 157쪽

C++ 원문

```
template<typename T>
void update(T& target)
{
    // ...
    if constexpr(is_pod<T>::value)
        simple_and_fast(target);    // 단순한 데이터용
    else
        slow_and_safe(target);
    // ...
}
```

코드의 의미

타입 특성에 따라 두 구현 중 하나를 컴파일 시간에 선택한다.

동작 과정

1. is_pod<T>::value를 조건으로 검사한다.

2. if constexpr에서 선택한 경우만 인스턴스화한다.

원문 실행 결과

[원문에 실행 결과 미제시]

원문 주의 사항

- 원문 자체의 // ...를 보존한 부분적 예제다.

if constexpr은 단순한 텍스트 조작이 아니다. 일반적인 문법·타입·스코프 규칙을 지켜야 하며, 조건부로 try의 시작과 끝을 서로 분리하는 원문 bad() 예는 문법 에러다.

6.5 조언

출처: 책 161-162쪽 / 원문 PDF 158-159쪽

- 여러 인자 타입에 적용할 알고리즘과 컨테이너를 템플릿으로 표현한다.
- 간단한 함수 객체를 한 곳에서만 사용한다면 람다를 이용한다.
- 표기를 단순하게 하고 구현을 숨길 때 템플릿 별칭을 사용한다.
- 템플릿의 타입 검사는 컴파일 시간에 수행되지만 인스턴스화까지 늦어질 수 있다.
- 사용되는 스코프에는 템플릿의 선언뿐 아니라 정의도 필요하다. 원문은 템플릿 정의를 사용하는 모든 변환 단위에서 포함하도록 조언한다.

중요 내용 요약

- 템플릿은 일반 개념을 인자로 구체화하는 컴파일 시간 메커니즘이다.
- 값 인자는 상수 표현식이어야 하며, 인자 추론에는 모호함과 의도와 다른 결과가 있을 수 있다.
- 함수 객체는 상태와 호출을 함께 제공하고 람다는 이를 사용 위치에서 표현한다.
- if constexpr도 문법·타입·스코프 규칙을 지킨다.

혼동하기 쉬운 내용

파라미터화된 연산의 표현

출처: 책 149-153쪽 / 원문 PDF 146-150쪽

수단	원문에서의 역할
함수 템플릿	타입·값이 달라져도 공통 연산 수행
함수 객체	호출 연산과 데이터를 함께 전달
람다	사용 위치에서 함수 객체를 간단히 생성

장 마무리 복습

- 타입 파라미터, 값 파라미터, 요구 조건을 구분해 원문 Vector와 Buffer의 목적을 떠올린다.
- 시퀀스 요소 타입과 누산기 타입이 독립적이라는 관계를 sum()으로 복습한다.
- 함수 객체의 데이터와 람다 캡처의 관계를 다시 확인한다.
- 별칭은 표현을 단순화하고 if constexpr은 인스턴스화할 경우를 선택한다.

원문 확인 메모

- 원문 Buffer의 T[N]; 및 복사 캡처 설명의 [=x]는 의심되는 원문 표기를 보존하고 확인 필요로 표시했다.

제7장. 컨셉과 제네릭 프로그래밍

출처: 책 163-182쪽 / 원문 PDF 160-179쪽

학습 내용 개요

- 템플릿 요구 사항을 컨셉으로 드러내기
- 의미 있는 일반화와 리프팅
- 파라미터 팩·접힘·인자 포워딩·컴파일 모델

설명의 흐름

1. 요구 조건 → 오버로딩과 컨셉 정의 → 구체 코드의 일반화 → 가변 인자 처리 → 타입 검사 시점

7.1 소개

출처: 책 163-164쪽 / 원문 PDF 160-161쪽

템플릿은 타입·값·템플릿 자체를 정보 손실 없이 인자로 전달하고, 인스턴스화 때 알게 되는 정보로 최적화하며, 상수 인자로 컴파일 시간 계산도 수행하게 한다. 클래스는 코드와 값을 함께 담을 수 있으므로 템플릿의 파라미터화 대상은 단순한 요소 타입에 그치지 않는다.

제네릭 프로그래밍의 중심은 요구 사항을 만족하는 여러 타입에 공통으로 적용할 알고리즘을 설계·구현·사용하는 것이다. 템플릿은 컴파일 시간에 파라미터화할 수 있는 다형성을 제공하고, 컨셉은 인자에 요구되는 기본 개념을 표현한다.

7.2 컨셉(C++20)

출처: 책 164-165쪽 / 원문 PDF 161-162쪽

sum()은 서로 다른 시퀀스의 자료 구조와 서로 다른 요소 타입을 모두 받는다. 다만 시퀀스는 구간 for를 사용할 수 있는 begin()/end()를 제공해야 하고 요소는 누산할 값에 더할 수 있어야 한다. 이처럼 템플릿 인자가 만족해야 할 요구 사항을 **컨셉**이라고 한다.

원문은 컨셉의 언어 지원이 아직 ISO C++에 포함되지 않았고 ISO 기술 명세에 있으며, 구현체는 있지만 세부 사항의 변경이나 상용 사용까지의 시간이 위험 요소라고 설명한다. 소절 제목의 C++20 표기와 함께 이 당시 상태를 그대로 유지한다.

7.2.1 컨셉의 사용

출처: 책 165-167쪽 / 원문 PDF 162-164쪽

typename은 인자가 타입이어야 한다는 최소 요구만 지정한다. Sequence와 Number라는 의미 있는 제약을 쓰면 의도가 드러나고, 구현을 모두 보지 않아도 인터페이스에서 잘못된 호출을 막을 수 있어 에러 보고가 좋아진다. 그러나 시퀀스와 수라는 두 개별 조건만으로 요소와 누산기의 연산 관계까지 완전히 지정한 것은 아니다.

시퀀스와 수의 관계를 지정하는 요구 절

코드 출처: 책 166쪽 / 원문 PDF 163쪽

C++ 원문

```
template<Sequence Seq, Number Num>
    requires Arithmetic<Value_type<Seq>,Num>
Num sum(Seq s, Num n);
```

코드의 의미

시퀀스의 요소 타입과 누산할 수 타입 사이에 산술 연산이 가능해야 한다는 요구를 인터페이스에 표현한다.

동작 과정

1. Seq는 Sequence, Num은 Number를 만족해야 한다.
2. 추가 requires 절은 Value_type<Seq>와 Num의 관계를 제한한다.

원문 실행 결과

[원문에 실행 결과 미제시]

출처: 책 166쪽 / 원문 PDF 163쪽

Value_type은 시퀀스의 요소 타입이다. Arithmetic<X,Y>는 두 수치 타입 사이의 산술 연산 가능성을 표현한다. 원문은 실제 구현에 +=만 등장해도 이를 지나치게 좁은 요구로 삼으면 유연성이 줄어든다고 설명한다. 나중에 +와 =를 이용하는 구현으로 바꿀 수도 있으므로 일반적인 의미를 가진 컨셉이 더 낫다.

출처: 책 166-167쪽 / 원문 PDF 163-164쪽

부분적인 명세도 의도를 알리고 점진적으로 개발하는 데 유용하지만, 명시하지 못한 요구에 관한 에러는 인스턴스화 시간에 나타날 수 있다. requires 절과 template<Sequence Seq>와 같은 축약 표기를 사용할 수 있다. 컨셉을 쓸 수 없는 환경에서는 명명 관례와 주석으로 요구를 나타낸다고 설명한다.

7.2.2 컨셉 기반 오버로딩

출처: 책 167-168쪽 / 원문 PDF 164-165쪽

템플릿도 인자가 만족하는 속성에 따라 오버로딩할 수 있다. 원문 advance()는 순방향 반복자에는 반복해서 ++를 수행하고, 임의 접근 반복자에는 +=를 수행하는 두 버전을 비교한다. 컴파일러는 인자가 만족하는 가장 강한 요구 사항에 맞춰 선택한다. 이 선택은 컴파일 시간에 이루어져 실행 시간의 부담을 추가하지 않는다.

- 컨셉을 만족하지 못하는 인자가 있는 함수는 후보에서 제외한다.
- 맞는 함수가 하나뿐이면 선택하고, 똑같이 맞는 두 함수를 구분하지 못하면 모호성 에러를 낸다.
- 한 함수의 요구가 다른 함수의 모든 요구와 추가 요구를 포함한다면 더 엄격하게 맞는 함수를 선택한다.
- 선택되는 함수는 모든 인자가 맞아야 하고, 다른 후보보다 각 인자가 적어도 같은 정도로 맞으며, 적어도 하나는 더 잘 맞아야 한다.

7.2.3 유효한 코드

출처: 책 169-170쪽 / 원문 PDF 166-167쪽

요구 사항의 바탕에는 어떤 표현식이 유효한가라는 문제가 있다. requires 표현식은 주어진 표현식 집합이 유효한 코드인지 판단하는 술어다. requires requires에서 첫 단어는 요구 절을, 두 번째 단어는 이 표현식을 시작하므로 중복 자체는 오타가 아니다.

표현식의 유효성 확인

코드 출처: 책 169쪽 / 원문 PDF 166쪽

advance() 예제에서 requires 표현식만 발췌했다. 함수 정의는 생략했다.

C++ 원문

```
requires(Iter p, int i) { p[i]; p+i; }
```

코드의 의미

인덱싱과 더하기 표현식이 유효한 코드인지를 판단하는 requires 표현식이다.

동작 과정

1. 주어진 문장이 유효하면 true, 그렇지 않으면 false인 술어로 평가한다.

원문 실행 결과

[원문에 실행 결과 미제시]

원문 주의 사항

- 이 표현식만으로 +=나 연산의 반환 타입까지 요구한 것은 아니라고 원문이 지적한다.

원문은 표현식을 직접 나열한 advance()에서도 +=와 반환 타입 요구를 빠뜨렸다고 지적한다. 이런 저수준 표현은 매우 유연하지만 의미를 전달하기 어렵다. 보통 코드에는 적절한 이름과 명시된 의미가 있는 컨셉을 사용하고, requires 표현식은 컨셉 정의에 사용하도록 조언한다.

7.2.4 컨셉 정의

출처: 책 170-172쪽 / 원문 PDF 167-169쪽

컨셉은 하나 이상의 타입을 어떻게 사용할 수 있는지 지정하는 컴파일 시간 술어이며 값은 항상 bool이다. 원문 Equality_comparable은 같은 타입의 두 값을 ==와 !=로 비교할 수 있고 그 결과가 bool로 변환될 수 있어야 한다는 요구다. int에 대한 검사는 성공하지만, 구조체에 이러한 연산이 자동으로 정의되지 않으면 검사가 실패한다.

출처: 책 171쪽 / 원문 PDF 168쪽

두 타입을 비교하도록 확장할 때는 양방향의 ==와 !=를 요구한다. 두 번째 템플릿 인자를 지정하지 않을 때 첫 타입을 쓰도록 기본 인자를 줄 수 있다. 원문은 int와 double, 같은 int의 검사는 성공하고 int와 string의 검사는 실패하는 예를 제시한다.

출처: 책 171-172쪽 / 원문 PDF 168-169쪽

Sequence 정의는 값 타입과 반복자 타입을 제공하고, 컨테이너 관례에 맞는 begin()/end()를 제공하며, 반복자가 요소 타입에 맞는 입력 반복자여야 한다는 조건을 결합한다. 기본 언어 개념을 정확히 표현하는 컨셉을 정의하는 일이 가장 어려우므로, 잘 정돈된 라이브러리의 컨셉을 이용하라고 설명한다.

7.3 제네릭 프로그래밍

출처: 책 172쪽 / 원문 PDF 169쪽

구체적이고 효율적인 알고리즘에서 자료 표현에 묶인 부분을 추상화하여 서로 다른 표현에도 적용하는 것이 제네릭 프로그래밍의 중심이다. 컨셉은 기본 연산과 자료 구조에 대한 추상화이자 템플릿 파라미터가 만족해야 하는 요구 사항이다.

7.3.1 컨셉의 사용

출처: 책 172-173쪽 / 원문 PDF 169-170쪽

유용한 컨셉은 정수·부동소수점 수·시퀀스와 같은 해당 분야의 기본 개념을 표현한다. Regular 타입은 기본 생성이 가능하고 생성·대입으로 독립적이면서 같은 값인 복사본을 만들 수 있으며 ==와 !=로 비교할 수 있다. 원문은 과도하게 영리한 프로그래밍 기법으로 기술적 문제를 겪지 않는다는 조건도 덧붙인다. int, vector, string이 예이며, int와 string은 <·<=·>·>=로 비교 가능한 StrictTotallyOrdered의 예로도 설명한다.

출처: 책 173쪽 / 원문 PDF 170쪽

컨셉은 단순한 구문보다 의미에 관한 개념이다. 예를 들어 +가 존재하기만 한다고 충분하지 않으며, +를 나눗셈으로 정의해서는 합리적인 수의 요구를 만족하지 않는다. 언어로 의미론 전체를 표현할 수 없으므로 분야의 지식과 상식이 필요하다. 원문은 의미 없는 Addable·Subtractable 같은 개념을 나열하기보다 분야의 기본 개념에 대응하도록 설계하라고 조언한다.

7.3.2 템플릿을 이용한 추상화

출처: 책 173-175쪽 / 원문 PDF 170-172쪽

좋은 추상화는 실제 구체적 용례에서 시작한다. 모든 미래 필요를 예상하여 추상화하기보다는 하나 이상의 사례에서 불필요한 세부를 제거한다. 원문의 합산은 요소가 int인지, 저장소가 vector인지, 누산기가 double인지, 초깃값이 0인지에 대한 고정을 템플릿 인자와 함수 인자로 풀어낸다.

구체적인 합산을 일반화한 accumulate

코드 출처: 책 174쪽 / 원문 PDF 171쪽

C++ 원문

```
template<typename Iter, typename Val>
Val accumulate(Iter first, Iter last, Val res)
{
    for (auto p = first; p!=last; ++p)
        res += *p;
    return res;
}
```

코드의 의미

구체적인 vector<int>와 double, 0에 고정된 합산을 반복자·누산기 타입·초깃값으로 일반화한다.

동작 과정

1. first와 last로 표현된 시퀀스를 순회한다.
2. 각 요소를 주어진 res에 누적한다.

원문 실행 결과

[원문에 실행 결과 미제시]

출처: 책 174-175쪽 / 원문 PDF 171-172쪽

반복자 한 쌍은 자료 구조를 추상화하고, Val은 누산기 타입을, res는 초깃값을 전달한다. 동일한 성능을 유지하며 구체적인 코드를 일반화하는 과정을 리프팅이라고 한다. 권장되는 개발 순서는 구체 버전을 만들고 디버그·테스트·측정한 후 구체 타입을 템플릿 인자로 교체하는 것이다. begin()/end() 반복을 줄이려면 Range를 받는 인터페이스로 단순화할 수 있고, 더 일반화하려면 +=도 대체할 수 있다.

7.4 가변 템플릿

출처: 책 176-177쪽 / 원문 PDF 173-174쪽

가변 템플릿은 다양한 타입의 인자를 가변 개수로 받는다. ...으로 선언하는 파라미터를 파라미터 팩이라고 하며, Tail은 타입 시퀀스, tail은 그 타입들에 해당하는 값의 시퀀스다. 전통적인 구현은 첫 인자를 처리한 뒤 나머지 팩을 재귀 호출한다.

파라미터 팩을 재귀적으로 처리

코드 출처: 책 176쪽 / 원문 PDF 173쪽

C++ 원문

```
void print()
{
    // 인자가 없으면 아무것도 하지 않음
}

template<typename T, typename ... Tail>
void print(T head, Tail... tail)
{
    // 각 인자에 대해
    cout << head << ' ';
    print(tail...);
}
```

코드의 의미

첫 인자와 나머지 인자를 분리하여 가변 개수의 인자를 출력하는 두 원문 정의다.

동작 과정

1. head를 출력한다.
2. tail에 대해 print를 다시 호출한다.
3. 인자가 없는 print()가 재귀의 마지막을 처리한다.

원문 실행 결과

[원문에 실행 결과 미제시]

마지막에 팩이 비면 인자 없는 print()가 호출되어 종료한다. 빈 인자를 허용하고 싶지 않으면 if constexpr으로 실제 마지막 호출을 생성하지 않는 대안을 쓸 수 있다.

빈 파라미터 팩 호출을 생성하지 않기

코드 출처: 책 177쪽 / 원문 PDF 174쪽

C++ 원문

```
template<typename T, typename ... Tail>
void print(T head, Tail... tail)
{
    cout << head << ' ';
    if constexpr(sizeof...(tail)> 0)
        print(tail...);
}
```

코드의 의미

컴파일 시간 if로 나머지 인자가 있을 때만 다음 호출을 생성하는 대안이다.

동작 과정

1. sizeof...(tail)로 나머지 개수를 확인한다.
2. 개수가 0이면 마지막 print() 호출을 인스턴스화하지 않는다.

원문 실행 결과

[원문에 실행 결과 미제시]

출처: 책 177쪽 / 원문 PDF 174쪽

가변 템플릿의 장점은 원하는 여러 타입을 받을 수 있다는 유연성이다. 반면 재귀 구현을 고치기 어렵고 컴파일 비용이 커질 수 있으며 인터페이스 타입 검사에 복잡한 템플릿 프로그램이 필요할 수 있다. 표준 라이브러리의 중요한 수단이지만 남용할 수 있다고 경고한다.

7.4.1 접힘 표현식

출처: 책 177-180쪽 / 원문 PDF 174-177쪽

C++17의 접힘 표현식은 파라미터 팩의 요소에 연산을 적용하는 일부 가변 템플릿 구현을 간단히 만든다. (v + ... + 0)은 오른쪽의 0에서 시작하는 괄호 묶음으로, (0 + ... + v)는 왼쪽의 0에서 시작하는 괄호 묶음으로 설명된다.

오른쪽 접힘과 원문 결과

코드 출처: 책 178쪽 / 원문 PDF 175쪽

C++ 원문

```
template<Number... T>
int sum(T... v)
{
    return (v + ... + 0);    // v의 모든 요소를 0부터 더함
}

int x = sum(1, 2, 3, 4, 5); // x는 15가 됨
int y = sum('a', 2.4, x);   // y는 114가 됨(2.4는 소수점 아래는 버려지고 'a'의 값은 97)
```

코드의 의미

파라미터 팩을 오른쪽의 초깃값 0에서부터 더하는 접힘 표현식이다.

동작 과정

1. 괄호가 오른쪽에 중첩되는 형태로 해석한다.
2. 원문은 x=15와 y=114를 결과로 명시한다.

원문 실행 결과

원문 명시: x는 15, y는 114.

원문 주의 사항

- 문자 값과 소수 부분 처리에 대한 설명도 원문 결과에 한정하여 보존했다.

왼쪽 접힘

코드 출처: 책 178쪽 / 원문 PDF 175쪽

C++ 원문

```
template<typename ... T>
int sum2(T... v)
{
    return (0 + ... + v);    // v의 모든 요소를 0에 더함
}
```

코드의 의미

초깃값 0을 왼쪽에 두는 접힘의 원문 대안이다.

동작 과정

1. 괄호가 왼쪽에서부터 중첩되는 형태로 해석한다.

원문 실행 결과

[원문에 실행 결과 미제시]

출처: 책 179-180쪽 / 원문 PDF 176-177쪽

접힘은 합산뿐 아니라 <<를 통한 모든 인자의 출력에도 쓸 수 있다. 원문 to_vector는 여러 인자를 공통 요소 타입의 벡터에 담아 이후 다루기 쉽게 하는 예다. 접힘의 일반적인 아이디어는 accumulate에서도 사용되지만, 원문은 당시 C++의 접힘 표현식은 가변 템플릿 구현을 간단히 하는 용도로 제한된다고 설명한다.

7.4.2 인자 포워딩

출처: 책 180쪽 / 원문 PDF 177쪽

가변 템플릿의 중요한 용도는 인터페이스에서 받은 인자를 다른 인터페이스로 그대로 전달하는 것이다. 원문 InputChannel은 실제 전송 방식을 Transport로 파라미터화하고 생성자 인자를 forward()로 Transport 생성자에 전달한다. 작성자가 특정 Transport에 필요한 인자의 종류를 몰라도 객체를 생성할 수 있다. 일반성과 낮은 실행 부담이 중요한 기본 라이브러리에서 널리 쓴다고 설명한다.

7.5 템플릿 컴파일 모델

출처: 책 181쪽 / 원문 PDF 178쪽

컨셉을 사용하면 먼저 템플릿 인자가 컨셉을 만족하는지 확인한다. 이때 확인할 수 없는 항목은 실제 코드를 생성하는 인스턴스화 시간까지 미룬다. 컨셉이 없는 템플릿은 타입 검사를 인스턴스화 시점에 수행하므로 오류가 늦게 발견되고 여러 곳의 정보가 합쳐진 진단이 쓸모없어질 수 있다.

원문은 이처럼 템플릿 인자에 필요한 연산이 있는지 뒤늦게 확인하는 특성을 컴파일 시간 덕 타이핑과 연결한다. 일반적인 객체가 타입에 따라 연산을 갖는 것과 달리 템플릿 계산의 많은 부분은 객체보다 값에 관한 것이며, constexpr 함수의 지역 변수는 예외로 든다.

제한되지 않은 템플릿은 사용하는 스코프에 선언뿐 아니라 정의가 있어야 한다. 그래서 보통 정의는 .cpp보다 헤더에 놓인다. 원문은 모듈을 사용하면 일반 함수와 비슷하게 코드를 정리할 수 있고 텍스트 포함 문제로부터 보호할 수 있다고 설명한다.

7.6 조언

출처: 책 182쪽 / 원문 PDF 179쪽

- 템플릿 인자의 요구 개념을 세심하게 고려하고 컨셉을 설계 도구로 사용한다.
- 의미 있는 동작을 내포하는 컨셉을 사용하고 요구 연산을 완전하게 명시한다.
- 구체 구현을 디버그하고 측정한 뒤 일반화한다.
- 같은 타입의 인자 목록에는 초깃값 목록을 사용하고 가변 템플릿을 남용하지 않는다.
- 템플릿의 정의가 사용되는 스코프에 있어야 한다.

중요 내용 요약

- 컨셉은 인자에 대한 의미 있는 요구를 표현하는 컴파일 시간 술어다.
- 컨셉의 부분 명세는 유용하지만 모든 오류를 일찍 찾는다고 보장하지 않는다.
- 일반화는 실제 용례와 측정한 구체 코드에서 출발한다.
- 가변 템플릿의 유연성과 구현·컴파일 비용을 함께 고려한다.

혼동하기 쉬운 내용

원문이 대비한 검사 시점

출처: 책 181쪽 / 원문 PDF 178쪽

방식	먼저 확인하는 내용	남는 검사
컨셉 사용	템플릿 인자의 컨셉 만족 여부	그 시점에 확인할 수 없는 항목은 인스턴스화 때 검사
컨셉 미사용	컨셉에 의한 인터페이스 검사 없음	타입 검사가 인스턴스화 시간까지 지연

장 마무리 복습

- Sequence·Number·Arithmetic이 각각 개별 성질과 두 타입 사이의 관계를 맡는다.
- 구문적으로 연산이 존재한다는 사실과 의미 있는 컨셉이라는 조건을 구별한다.
- 리프팅에서는 자료 구조·요소 타입·누산기·초깃값의 고정을 분리한다.
- 재귀의 종료 조건과 if constexpr, 접힘의 방향을 원문 코드와 연결한다.

원문 확인 메모

- 컨셉의 표준·기술 명세 상태와 코드 표기는 원문 당시의 설명을 유지했다.

제8장. 라이브러리의 개요

출처: 책 183-188쪽 / 원문 PDF 180-185쪽

학습 내용 개요

- 기본 언어 위에서 라이브러리가 맡는 역할
- 표준 라이브러리의 기능 분류
- 헤더 포함과 std 이름 접근

설명의 흐름

1. 라이브러리 활용 이유 → 기능 구성과 선정 기준 → 헤더·네임스페이스 사용

8.1 소개

출처: 책 183-184쪽 / 원문 PDF 180-181쪽

실제 프로그램은 기본 언어 기능만으로 작성하기보다 라이브러리를 토대로 구성된다. 좋은 라이브러리를 이용하면 기본 언어만으로 작성하기 어려운 작업을 쉽게 만들 수 있다. 이 책은 모든 세부를 완전히 설명하기보다 표준 라이브러리의 유용한 기능과 일반적인 사용법을 익히는 데 초점을 둔다.

표준 라이브러리에는 설계·구현·유지 보수·확장을 위한 많은 노력이 들어간다. 원문은 직접 코드를 다시 만드는 대신 이 기능을 사용하라고 권한다. 한편 GUI·웹·데이터베이스 인터페이스나 산업별 기반 라이브러리도 있지만, 이 책은 표준에서 정의한 C++와 이식성 있는 예제에 집중하므로 이러한 외부 시스템은 다루지 않는다.

8.2 표준 라이브러리 구성 요소

출처: 책 184-186쪽 / 원문 PDF 181-183쪽

표준 라이브러리는 널리 쓰는 기본 자료 구조와 그 위에서 동작하는 알고리즘을 일반적인 형태로 제공한다. 기본적인 사용 예는 작업 자체의 복잡성에 비해 쉽게 배울 수 있고, 단순한 대안에 비해 큰 추가 부담이 없으며, 초보자와 전문가 모두에게 유용해야 한다는 선정 기준을 설명한다.

- 언어 실행 지원과 C 표준 라이브러리: 메모리 할당, 실행 시간 타입 정보, 타입 시스템 위배를 줄이기 위한 일부 수정.
- 문자열·문자열 뷰·정규 표현식: 여러 문자 세트, 지역화, 읽기 전용 부분 문자열, 패턴 매칭.
- 입출력·파일 시스템: 타입·스트림·버퍼링·로케일을 확장할 수 있는 입출력과 이식성 있는 파일 조작.
- 컨테이너·알고리즘: vector·map과 find·sort·merge 등의 확장 가능한 STL 프레임워크.
- 수치·동시성·병렬 계산: 수학 함수, 복소수, 수치 벡터, 난수와 thread·mutex, 일부 병렬 알고리즘.
- 일반적 유틸리티: 타입 특성, pair·variant·optional, 시간, 스마트 포인터와 자원 관리, array·bitset·tuple, 단위를 나타내는 접미사.

8.3 표준 라이브러리 헤더와 네임스페이스

출처: 책 186-188쪽 / 원문 PDF 183-185쪽

표준 기능은 표준 헤더가 제공하며 std 네임스페이스에 정의된다. 사용하려는 기능의 헤더를 포함하고 std:: 등의 방법으로 이름에 접근해야 한다. 원문의 많은 예제는 단순하게 보이도록 헤더와 std::를 생략하므로, 그러한 코드 조각이 독립적인 완전한 프로그램이라는 뜻은 아니다.

기능을 제공하는 표준 헤더

코드 출처: 책 186쪽 / 원문 PDF 183쪽

C++ 원문

```
#include<string>
#include<list>
```

코드의 의미

표준 string과 list를 제공하는 원문 헤더다.

동작 과정

1. 해당 표준 헤더로 각 기능을 사용할 수 있게 한다.

원문 실행 결과

[원문에 실행 결과 미제시]

std 이름을 직접 한정하기

코드 출처: 책 186쪽 / 원문 PDF 183쪽

같은 원문 영역의 첫 선언을 발췌했다. 이어지는 slogans 목록 선언은 생략했다.

C++ 원문

```
std::string sheep {"Four legs Good; two legs Baaad!"};
```

코드의 의미

std 네임스페이스의 타입 이름을 명시적으로 사용하는 원문 예다.

동작 과정

1. std::string으로 문자열을 선언한다.

원문 실행 결과

[원문에 실행 결과 미제시]

출처: 책 187쪽 / 원문 PDF 184쪽

using namespace std를 쓰면 std:: 없이 이름을 사용할 수 있다. 하지만 한 네임스페이스의 모든 이름을 전역으로 가져오는 것은 일반적으로 좋지 않다고 명시한다. 책에서는 표준 라이브러리의 사용법 학습을 위해 이 방식을 쓰기도 한다.

기능	원문에 나열된 헤더
알고리즘·컨테이너	<algorithm>, <array>, <forward_list>, <list>, <map>, <set>, <unordered_map>, <vector>
문자·입출력	<string>, <regex>, <ios>, <iostream>, <fstream>, <sstream>
수치·시간·동시성	<cmath>, <complex>, <random>, <chrono>, <thread>, <future>
자원·지원	<memory>, <stdexcept>, <utility>, <variant>, <filesystem>

출처: 책 188쪽 / 원문 PDF 185쪽

C 라이브러리의 <stdlib.h> 같은 헤더도 제공한다. 이름 앞에 c를 붙이고 .h를 제거한 <cstdlib> 같은 버전은 이름을 std 네임스페이스에 선언한다. 헤더 목록은 선별된 일부이며 표준 라이브러리 전체 목록이 아니다.

8.4 조언

출처: 책 188쪽 / 원문 PDF 185쪽

- 라이브러리를 활용하고, 선택할 수 있다면 표준 라이브러리를 사용한다.
- 표준 라이브러리가 모든 문제를 해결하는 것은 아니다.
- 필요한 기능을 제공하는 헤더를 포함한다.
- 표준 라이브러리 기능은 std 네임스페이스에 정의된다.

중요 내용 요약

- 라이브러리의 기본 사용법을 익히고 적절한 표준 기능을 선택한다.
- 헤더 포함과 이름의 접근은 서로 연결되지만 각각 필요한 사항이다.
- 책의 코드에서 헤더·std::를 생략한 편집 방식을 완전한 프로그램으로 혼동하지 않는다.

장 마무리 복습

- 컨테이너와 알고리즘을 비롯한 기본 기능은 표준 라이브러리가 제공한다.
- 표준 헤더와 std 네임스페이스를 함께 확인한다.
- C 헤더와 c 접두사 버전의 이름 제공 방식을 구분한다.

제9장. 문자열과 정규 표현식

출처: 책 189-208쪽 / 원문 PDF 186-205쪽

학습 내용 개요

- 소유하는 문자열과 비소유 읽기 뷰
- 문자열 연산과 표현·수명·구간 검사
- 정규 표현식의 검색·표기법·부분 매치·반복자

설명의 흐름

1. string 저장과 조작 → string_view의 인터페이스·수명 → regex로 패턴 기술·검색·순회

9.1 소개

출처: 책 189-190쪽 / 원문 PDF 186-187쪽

텍스트 조작을 위해 표준 라이브러리는 문자 시퀀스를 저장·조작하는 string, 저장 방식에 독립적으로 문자를 읽는 string_view, 패턴을 기술하고 찾는 regex를 제공한다. string과 regex는 여러 문자 타입에 적용할 수 있으며 원문은 유니코드를 그 예로 든다.

9.2 문자열

출처: 책 190-192쪽 / 원문 PDF 187-189쪽

string은 문자 시퀀스를 저장하고 조작하는 Regular 타입이다. +는 문자열 연결을 뜻하며 string·문자열 리터럴·C 스타일 문자열·문자 하나를 함께 연결할 수 있다. 끝에 추가하는 경우 +=는 의도를 명료하게 나타내고 간결하며 때로는 더 효율적이어서 원문이 권장한다. 이동 생성자가 있으므로 긴 string도 값으로 효율적으로 반환할 수 있다.

출처: 책 191쪽 / 원문 PDF 188쪽

string은 변경할 수 있고 대입·인덱싱·부분 문자열 연산을 제공한다. substr()은 지정한 위치부터 길이만큼 별도 string을 복사한다. replace()는 지정한 부분을 교체하며, 원래 부분과 새 문자열의 길이가 같을 필요는 없다.

부분 문자열 복사와 교체

코드 출처: 책 191쪽 / 원문 PDF 188쪽

C++ 원문

```
string name = "Niels Stroustrup";

void m3()
{
    string s = name.substr(6,10); // s = "Stroustrup"
    name.replace(0,5,"nicholas"); // name은 "nicholas Stroustrup"
    name[0] = toupper(name[0]);   // name은 "Nicholas Stroustrup"
}
```

코드의 의미

부분 문자열을 별도 string으로 복사하고 원래 문자열의 일부를 다른 길이의 문자열로 교체한다.

동작 과정

1. substr의 첫 인자는 0부터 세는 위치, 두 번째는 길이다.
2. replace로 Niels를 nicholas로 바꾼다.

3. 첫 문자를 대문자로 바꾼다.

원문 실행 결과

원문 명시: s는 "Stroustrup", name의 마지막 값은 "Nicholas Stroustrup".

출처: 책 191-192쪽 / 원문 PDF 188-189쪽

비교에는 ==와 !=, 사전 순서 비교에는 <·<=·>·>=를 사용한다. []와 at()을 이용한 접근, 반복자 순회, 입력과 스트리밍도 제공한다. C 스타일의 0으로 끝나는 문자 배열이 필요할 때는 c_str()로 읽기 전용 문자 포인터를 얻는다. 원문은 필요할 때에만 이 표현을 사용하라고 조언한다.

출처: 책 192쪽 / 원문 PDF 189쪽

문자열 리터럴을 std::string으로 사용하려면 접미사 s를 쓴다. 접미사가 없는 리터럴과의 타입 차이를 주의한다. 원문은 std::literals::string_literals 네임스페이스가 필요하다고 설명한다.

9.2.1 string 구현

출처: 책 193-194쪽 / 원문 PDF 190-191쪽

일반적으로 사용하는 string은 직접 처음 만든 구현보다 표준 구현을 선택하라고 설명한다. 흔히 사용하는 짧은 문자열 최적화는 짧은 값을 객체 자체에 두고 긴 값에만 자유 저장소를 이용한다. 문자열 길이가 변하면 표현도 적절히 바뀌며, 짧다고 보는 길이는 구현에 따라 달라진다.

실제 성능은 실행 환경의 영향을 받는다. 특히 다중 스레드 환경의 메모리 할당 비용과 서로 다른 길이 문자열에 의한 메모리 단편화는 짧은 문자열 최적화를 사용하는 이유다. 문자 타입을 일반화한 basic_string<Char>가 바탕이며, string은 basic_string<char>의 별칭이다. 원문 Jchar 예처럼 다른 문자 타입에도 일반적인 문자열 연산을 적용할 수 있다.

9.3 문자열 뷰

출처: 책 194-196쪽 / 원문 PDF 191-193쪽

string_view는 연속된 문자 시퀀스를 나타내는 포인터와 길이의 쌍으로 볼 수 있다. 문자는 string·C 스타일 문자열 등 서로 다른 방식으로 저장될 수 있다. 뷰 자체는 문자를 소유하지 않으므로 포인터·참조 및 STL 반복자와 유사하다.

서로 다른 저장 형태를 받는 문자열 뷰

코드 출처: 책 195쪽 / 원문 PDF 192쪽

C++ 원문

```
string cat(string_view sv1, string_view sv2)
{
    string res(sv1.length()+sv2.length());
    char* p = &res[0];
    for (char c : sv1)    // 복사를 수행하는 한 가지 방법
        *p++ = c;
    copy(sv2.begin(),sv2.end(),p); // 다른 방법
    return res;
}
```

코드의 의미

두 읽기 전용 문자 시퀀스를 받는 인터페이스를 보여 주는 원문의 cat 정의다.

동작 과정

1. sv1을 순회하며 복사한다.

2. sv2는 copy로 복사한다.

3. 만든 res를 반환한다.

원문 실행 결과

[원문에 실행 결과 미제시]

원문 주의 사항

- [원문 확인 필요] string res(sv1.length()+sv2.length()); 표기를 원문 그대로 수록했다. 헤더나 인자를 덧붙이지 않았다.

출처: 책 195-196쪽 / 원문 PDF 192-193쪽

const string&를 받는 compose()와 비교해 원문 cat()은 다양한 저장 방식을 받을 수 있고, C 스타일 문자열에서 임시 string을 만들지 않으며, 부분 문자열을 쉽게 전달할 수 있다. 접미사 sv를 쓰는 문자열 뷰 리터럴은 길이를 컴파일 시간에 계산한다. 원문은 std::literals::string_view_literals를 지정한다.

지역 문자열을 가리키는 뷰 반환의 위험

코드 출처: 책 196쪽 / 원문 PDF 193쪽

C++ 원문

```
string_view bad()
{
    string s = "Once upon a time";
    return {&s[5],4};    // 위험: 지역 변수를 가리키는 포인터 반환
}
```

코드의 의미

사용 전에 파괴되는 지역 string의 문자를 가리키는 string_view를 반환하는 잘못된 원문 예다.

동작 과정

1. 지역 변수 s를 만든다.

2. 그 문자 영역의 주소와 길이로 뷰를 반환한다.

원문 실행 결과

[원문에 실행 결과 미제시]

원문 주의 사항

- string_view는 가리키는 문자를 소유하지 않는다. 원문은 이 반환을 위험하다고 명시한다.

출처: 책 196쪽 / 원문 PDF 193쪽

뷰를 반환할 때는 실제로 무엇을 가리키는지 수명을 확인해야 한다. string_view는 읽기 전용이므로 문자를 소문자로 바꾸는 함수의 쓰기 인터페이스로 사용할 수 없다. 원문은 이런 용도에 gsl::span 또는 gsl::string_span을 연결한다. 구간 밖 접근 결과를 예측해서는 안 되며, 검사가 필요하면 out_of_range를 던지는 at()이나 gsl::string_span을 사용하도록 설명한다.

9.4 정규 표현식

출처: 책 197-198쪽 / 원문 PDF 194-195쪽

정규 표현식은 텍스트 안의 패턴을 간단히 기술하고 찾는 도구다. <regex>는 regex 클래스와 검색·매칭·치환·반복자를 제공한다. 로우 문자열 리터럴은 백슬래시와 큰따옴표를 직접 표현할 수 있어 정규 표현식을 적을 때 편리하다. 일반 문자열에서는 백슬래시를 두 번 적어야 한다.

로우 문자열로 기술한 원문 우편번호 패턴

코드 출처: 책 197쪽 / 원문 PDF 194쪽

C++ 원문

```
regex pat {R"(\w{2}\s*\d{5}(-\d{4})*)"}; // 미국 우편번호 패턴: XXddddd-dddd와 그 변종
```

코드의 의미

백슬래시가 많은 정규 표현식을 로우 문자열 리터럴로 나타낸다.

동작 과정

1. 두 문자 부분, 공백 부분, 숫자 다섯 개 부분, 괄호의 부분 패턴으로 나누어 설명한다.

원문 실행 결과

[원문에 실행 결과 미제시]

원문 주의 사항

- 끝의 (-₩d{4})* 표기를 원문대로 보존했다. 원문이 제시하지 않은 다른 패턴으로 교체하지 않았다.

기능	원문에서의 역할
regex_match()	입력 전체를 정규 표현식과 매칭
regex_search()	입력 안에서 매칭되는 부분 검색
regex_replace()	매칭되는 부분을 찾아 교체
regex_iterator	매치와 부분 매치를 찾으며 순회
regex_token_iterator	매치되지 않는 부분 순회

9.4.1 검색

출처: 책 198-199쪽 / 원문 PDF 195-196쪽

regex_search(line,matches,pat)은 line에서 pat에 맞는 부분을 찾아 matches에 저장하고, 없으면 false를 반환한다. smatch는 string의 매치 결과들을 저장하는 컨테이너다. matches[0]은 전체 매치이고 이후 항목은 괄호로 지정한 부분 패턴에 대응한다. 선택적인 부분 패턴을 이용할 때는 항목의 존재와 matched 상태를 확인한다.

파일에서 전체 매치와 부분 매치 찾기

코드 출처: 책 198-199쪽 / 원문 PDF 195-196쪽

C++ 원문

```
void use()
{
    ifstream in("file.txt");    // 입력 파일
    if (!in)                   // 파일이 열렸는지 확인
        cerr << "파일 없음\n";

    regex pat {R"(\w{2}\s*\d{5}(-\d{4})*)"}; // 미국 우편번호 코드

    int lineno = 0;
    for (string line; getline(in,line); ) {
        ++lineno;
        smatch matches;        // 매치된 문자열 저장
        if (regex_search(line , matches, pat)) {
            cout << lineno << ": " << matches[0] << '\n'; // 전체 매치
            if (1<matches.size() && matches[1].matched) // 부분 패턴이 존재하고
                                                        // 그 부분 패턴이 매치됐으면
                cout << "\t: " << matches[1] << '\n';     // 부분 매치
        }
    }
}
```

코드의 의미

파일의 행마다 패턴을 검색하고 전체 매치 및 존재하는 부분 매치를 출력한다.

동작 과정

1. getline으로 행을 읽으며 행 번호를 증가시킨다.
2. regex_search가 성공하면 matches[0]을 출력한다.
3. 부분 패턴이 존재하며 매치된 경우 matches[1]을 출력한다.

원문 실행 결과

[원문에 실행 결과 미제시]

원문 주의 사항

- 원문은 TX77845와 DC 20500-0001을 찾는다고 설명하지만 입력 파일 전체와 완전한 출력 목록은 제시하지 않는다.

출처: 책 199쪽 / 원문 PDF 196쪽

여러 행에 걸친 패턴에도 개행 문자를 포함할 수 있지만 그런 입력을 한 행씩만 읽어서는 안 된다. regex는 패턴을 효율적으로 실행할 수 있는 상태 기계로 실행 시간에 컴파일한다고 설명한다.

9.4.2 정규 표현식 표기법

출처: 책 199-206쪽 / 원문 PDF 196-203쪽

출처: 책 199-200쪽 / 원문 PDF 196-197쪽

원문의 기본 표기법은 ECMAScript의 ECMA 표준을 따른다. 특수 문자는 보통 문자와 다른 역할을 갖는다. 괄호는 부분 패턴을 만들고, 대괄호는 문자 클래스를, 중괄호는 반복 횟수를 지정한다. 점은 문자 하나, 세로줄은 대안, ^와 $는 각각 행의 시작과 끝을 나타낸다. ^는 문자 클래스의 반전에도 쓰인다.

반복 표기	의미
{n}	정확히 n번
{n,}	n번 이상
{n,m}	n번 이상 m번 이하
*	0번 이상
+	1번 이상
?	0번 또는 1번

출처: 책 201-202쪽 / 원문 PDF 198-199쪽

접미 반복이 없으면 패턴은 기본적으로 한 번 등장한다. 반복 표기 뒤의 ?는 가장 짧은 매치를 고르는 지연 또는 비욕심쟁이 매칭을 뜻한다. 기본의 욕심쟁이 매칭은 가장 긴 매치를 찾는다. 원문 abababe에서 (ab)+는 전체를, (ab)+?는 첫 ab를 매치한다고 설명한다.

출처: 책 202-203쪽 / 원문 PDF 199-200쪽

문자 클래스에는 alnum·alpha·blank·cntrl·digit·graph·lower·print·punct·space·upper·xdigit 등 이름이 있다. 이름을 [: :]로 감싼 뒤 문자 클래스의 [ ] 안에서 사용한다. 숫자·공백·단어 문자의 축약어는 각각 ₩d·₩s·₩w이고, ₩D·₩S·₩W는 각각 그 여집합이다. 원문은 비표준인 ₩l·₩u·₩L·₩U도 구분하고 이식성을 위해 이름을 사용하라고 조언한다.

출처: 책 203-204쪽 / 원문 PDF 200-201쪽

식별자 예는 첫 문자가 밑줄 또는 알파벳이고, 이후가 밑줄·알파벳·숫자의 시퀀스라는 조건을 설명한다. alpha와 alnum에 밑줄이 포함되지 않는다는 점, [:alpha:]를 문자 클래스 바깥에 적으면 의도가 달라진다는 점을 잘못된 패턴과 비교한다. 입력 전체를 검사하는 함수에는 regex_match()를 사용한다.

출처: 책 205-206쪽 / 원문 PDF 202-203쪽

group은 괄호로 구분되어 sub_match에 저장될 수 있다. 그룹을 묶되 부분 매치를 저장할 필요가 없으면 (?: 형태를 사용한다. 원문은 그룹 개수·중첩·반복에 따른 저장과 ₩1로 첫 그룹을 다시 참조하는 태그 예를 비교한다. 태그 사이를 .*로 탐욕적으로 잡으면 첫 <부터 마지막 >까지 걸칠 수 있으므로, 의도한 짧은 부분을 찾는 데 .*?를 사용한다고 설명한다.

출처: 책 200, 204-205쪽 / 원문 PDF 197, 201-202쪽

[원문 확인 필요] 일부 식별자 검사 코드와 패턴에서 본문 설명과 기호의 대응을 다시 확인해야 한다. 이 교재에서는 판독에 의심이 있는 코드의 기호를 임의로 고쳐 재작성하지 않고, 해당 페이지에서 명시된 규칙을 중심으로 정리했다.

9.4.3 반복자

출처: 책 206-207쪽 / 원문 PDF 203-204쪽

regex_iterator는 시퀀스에서 패턴을 차례로 찾아 순회한다. string에 대해 사용하는 원문 sregex_iterator 예는 공백 뒤의 단어를 부분 매치로 출력한다.

정규 표현식 반복자로 단어 찾기

코드 출처: 책 206쪽 / 원문 PDF 203쪽

C++ 원문

```
void test()
{
    string input = "aa as; asd ++e^asdf asdfg";
    regex pat {R"(\s+(\w+))"};
    for (sregex_iterator p(input.begin(),input.end(),pat);
         p!=sregex_iterator{}; ++p)
        cout << (*p)[1] << '\n';
}
```

코드의 의미

공백 뒤의 단어 부분 패턴을 차례로 얻는 원문의 sregex_iterator 예다.

동작 과정

1. 문자열 구간과 패턴으로 반복자를 만든다.
2. 기본 생성한 끝 반복자와 비교하며 전진한다.
3. 각 매치의 첫 부분 패턴을 출력한다.

원문 실행 결과

원문 출력:

as
asd
asdfg

원문 주의 사항

- 첫 단어 aa 앞에는 공백이 없어 놓친다고 원문이 설명한다.

출처: 책 206-207쪽 / 원문 PDF 203-204쪽

원문의 첫 패턴 출력은 as·asd·asdfg다. 앞 공백이 없는 aa는 빠진다. 패턴을 R"((₩w+))"로 단순화한 다음 페이지의 출력은 aa·as·asd·e·asdf·asdfg다. 이 결과는 원문에 제시된 목록만 옮긴 것이다. regex_iterator는 양방향 반복자를 요구하므로 입력 반복자만 제공하는 istream을 직접 순회할 수 없다고 설명한다. 쓰기는 지원하지 않고 끝 표시는 기본 생성한 regex_iterator{}를 쓴다.

9.5 조언

출처: 책 207-208쪽 / 원문 PDF 204-205쪽

- 문자 시퀀스를 저장할 때 string을 사용하고 C 스타일 함수 대신 문자열 연산을 사용한다.
- 부분 문자열의 복사는 substr(), 교체는 replace()로 표현한다.
- 구간 검사가 필요할 때 at()을 사용한다.
- 여러 저장 형태의 문자 시퀀스를 읽는 함수는 string_view를 사용하고 소유·수명 문제를 기억한다.
- 문자열 리터럴의 s와 문자열 뷰 리터럴의 sv를 구별한다.
- 패턴은 로우 문자열로 명료하게 표현하고 전체 매칭과 부분 검색을 구별한다.
- 필요한 부분 패턴만 저장하고 욕심쟁이·지연 매칭의 차이를 확인한다.

중요 내용 요약

- string은 문자열을 소유하고 변경하며, string_view는 문자를 소유하지 않는 읽기 전용 뷰다.
- 부분 문자열의 위치와 길이, 원래 문자열과 뷰의 수명을 구분한다.
- regex_match는 전체 입력, regex_search는 입력 안의 부분을 대상으로 한다.
- 매치 결과의 0번은 전체 매치이며 괄호로 지정한 부분 패턴은 뒤 항목으로 표현된다.
- 접미사 ?의 선택 횟수 의미와 반복 뒤 ?의 지연 매칭 의미는 문맥이 다르다.

혼동하기 쉬운 내용

문자열 저장과 문자열 뷰

출처: 책 190-196쪽 / 원문 PDF 187-193쪽

기준	string	string_view
역할	문자 시퀀스 저장·조작	여러 저장 형태의 문자 시퀀스 읽기
소유	문자를 직접 관리	문자를 소유하지 않음
수정	인덱싱·replace 등으로 변경	읽기 전용
수명	자기 객체의 문자열	가리키는 문자 영역의 수명에 의존

장 마무리 복습

- substr의 복사와 replace의 교체를 원문 name 예로 구분한다.
- 임시 string을 줄이는 뷰의 장점과 소유하지 않는다는 제약을 함께 복습한다.
- 특수 문자·문자 클래스·반복·그룹을 서로 다른 구성 요소로 읽는다.
- 전체 매치·부분 매치·반복자 끝 표시를 원문 검색 코드와 연결한다.

원문 확인 메모

- cat의 문자열 생성 표기를 원문 그대로 보존했다. 일부 정규 표현식 코드 기호는 원문 확인 필요로 표시하고 임의로 수정하지 않았다.

제10장. 입력과 출력

출처: 책 209-232쪽 / 원문 PDF 206-229쪽

학습 내용 개요

- 스트림의 타입 변환·상태·자원 관리
- 기본·사용자 정의 타입의 입출력과 형식화
- 파일·문자열 스트림과 이식성 있는 파일 시스템

설명의 흐름

1. 값과 문자 흐름 → 기본 입출력 → 상태·실패 처리 → 사용자 정의 연산 → 형식화·다양한 저장소 → 경로·탐색·에러

10.1 소개

출처: 책 210-211쪽 / 원문 PDF 207-208쪽

입출력 스트림 라이브러리는 텍스트와 수치 값의 형식화·비형식화 및 버퍼링된 입출력을 지원한다. ostream은 타입 있는 값을 문자 또는 바이트의 스트림으로 바꾸고, istream은 반대 방향으로 문자 스트림을 타입 있는 객체로 변환한다. 이 연산은 타입 안전성과 타입 민감성을 갖고 사용자 정의 타입에도 확장할 수 있다.

스트림은 표준 입력·출력, 파일, string 버퍼에 적용된다. 소멸자가 자신이 소유한 버퍼와 파일 핸들 같은 자원을 해제하므로 RAII의 한 예다. 이진 입출력·여러 문자 타입·로케일·고급 버퍼링도 지원하지만 여기서는 상세히 다루지 않는다고 범위를 정한다.

10.2 출력

출처: 책 211-213쪽 / 원문 PDF 208-210쪽

<<는 ostream에 값을 내보내는 연산이다. cout은 표준 출력, cerr은 에러 보고 스트림이다. cout에 정수 10을 쓰면 기본적으로 정수의 바이트 표현 대신 문자 1과 0으로 변환하여 출력한다. 한 출력 표현식의 결과를 다음 <<에 연결할 수 있어 관련 항목을 한 문장에 묶을 수 있다.

출처: 책 212-213쪽 / 원문 PDF 209-210쪽

문자 상수를 출력하면 문자가 출력되지만 char 값을 int로 변환하여 출력하면 정수 값이 출력된다. 원문은 문자 a·정수에 저장한 b·문자 c를 출력하는 예에서, 사용한 ASCII 인코딩을 전제로 a98c가 나온다고 명시한다. 이 결과를 모든 인코딩에 관한 주장으로 넓히지 않는다.

10.3 입력

출처: 책 213-215쪽 / 원문 PDF 210-212쪽

>>는 istream에서 값을 가져오는 연산이며 cin은 표준 입력 스트림이다. 오른쪽 피연산자의 타입이 어떤 입력을 받아 어디에 저장하는지를 결정한다. 정수와 부동소수점 입력을 연결해서 읽을 수 있다. 정수 입력은 숫자가 아닌 문자를 만나면 끝나고, >>는 기본적으로 앞 공백을 지나친다.

출처: 책 214-215쪽 / 원문 PDF 211-212쪽

string에 대한 >>는 공백에서 읽기를 끝내므로 Eric Bloodaxe를 입력해도 첫 단어 Eric만 읽는다. 한 행 전체가 필요하면 getline()을 사용한다. 행을 마친 개행 문자는 버리고 다음 행을 읽을 수 있는 상태가 된다. 문자열은 입력에 따라 늘어나므로 최대 길이를 미리 계산할 필요가 없다고 설명한다.

한 행 전체를 읽는 getline

코드 출처: 책 215쪽 / 원문 PDF 212쪽

C++ 원문

```
void hello_line()
{
    cout << "이름을 입력하세요\n";
    string str;
    getline(cin,str);
    cout << "Hello, " << str << "!\n";
}
```

코드의 의미

공백을 포함한 한 행의 이름을 string에 읽는다.

동작 과정

1. getline은 행 전체를 읽고 종료시킨 개행 문자는 버린다.

2. 읽은 이름을 인사 문자열과 함께 출력한다.

원문 실행 결과

원문 입력 Eric Bloodaxe에 대한 출력:
Hello, Eric Bloodaxe!

출처: 책 215쪽 / 원문 PDF 212쪽

원문은 문자 단위 조작보다 형식화된 입출력이 대체로 오류 가능성이 낮고 효율적이며 코드도 짧다고 설명한다. istream은 메모리 관리와 구간 검사를 맡고, 메모리 안에서 형식화를 수행할 때는 문자열 스트림을 사용한다.

10.4 입출력 상태

출처: 책 215-217쪽 / 원문 PDF 212-214쪽

스트림은 연산의 성공 여부를 나타내는 상태를 보유한다. is>>i는 is의 참조를 반환하고, 이를 조건으로 검사하면 다음 연산을 수행할 수 있는지를 확인한다. 정수가 아닌 입력이나 입력 끝에서 반복을 멈추는 구조다.

입력 상태를 루프 조건으로 사용

코드 출처: 책 215-216쪽 / 원문 PDF 212-213쪽

C++ 원문

```
vector<int> read_ints(istream& is)
{
    vector<int> res;
    for (int i; is>>i; )
        res.push_back(i);
    return res;
}
```

코드의 의미

정수를 계속 읽어 벡터에 저장하고 읽기를 계속할 수 없으면 종료한다.

동작 과정

1. is>>i가 반환하는 스트림 참조의 상태를 검사한다.

2. 읽기가 성공한 값만 res에 추가한다.

원문 실행 결과

[원문에 실행 결과 미제시]

출처: 책 216쪽 / 원문 PDF 213쪽

입출력 상태에는 형식화 정보·에러 상태·버퍼링 종류 등 작업에 필요한 정보가 들어간다. 사용자는 에러를 알리거나 회복 가능한 실패를 무시하기 위해 상태를 설정할 수 있다. 원문의 종료 표식 버전은 eof()면 정상 종료하고, 정수 입력 fail()이면 clear()로 상태를 리셋하고 unget()으로 되돌린 뒤 문자열 표식인지 확인한다.

출처: 책 216-217쪽 / 원문 PDF 213-214쪽

[원문 확인 필요] 종료 표식 버전은 입력 스트림 매개변수 is를 받지만 뒷부분의 문자열 읽기와 실패 상태 설정에는 cin을 사용한다. 이 교재에서는 일반화된 다른 코드로 고치지 않고 원문 설명의 흐름과 해당 표기를 구분해 둔다. 종료 표식 호출 예는 read_ints(cin,"stop")이다.

10.5 사용자 정의 타입의 입출력

출처: 책 217-219쪽 / 원문 PDF 214-216쪽

사용자 정의 타입에도 의미 있는 텍스트 표현이 있다면 <<와 >>를 정의할 수 있다. ostream이나 istream 자체를 수정할 필요는 없다. Entry의 출력은 {"이름", 번호} 형식을 사용하고 스트림 참조를 반환한다. 입력은 형식 검사와 에러 처리가 필요해 더 복잡하다.

사용자 정의 타입의 입출력 연산

코드 출처: 책 217-219쪽 / 원문 PDF 214-216쪽

C++ 원문

```
struct Entry {
    string name;
    int number;
};

ostream& operator<<(ostream& os, const Entry& e)
{
    return os << "{\"" << e.name << "\", " << e.number << "}";
}

istream& operator>>(istream& is, Entry& e)
    // { "이름" ,번호 } 쌍 읽기. { " " , 와 } 로 형식화됨
{
    char c, c2;
    if (is>>c && c=='{' && is>>c2 && c2=='"') { // { "로 시작
        string name;                            // string의 기본 값은 빈 문자열: ""
        while (is.get(c) && c!='"')              // " 이전의 모든 문자는 이름으로 저장
            name+=c;

        if (is>>c && c==',') {
            int number = 0;
            if (is>>number>>c && c=='}') {        // 번호와 } 읽기
                e = {name, number};              // 항목에 대입
                return is;
            }
        }
    }
    is.setstate(ios_base::failbit);              // 스트림에 실패를 기록함
    return is;
}
```

코드의 의미

이름과 번호를 가진 Entry에 텍스트 표현을 부여하는 구조체·출력·입력 세 원문 영역이다.

동작 과정

1. 출력 함수는 스트림 참조를 첫 인자로 받고 동일한 참조를 반환한다.
2. 입력 함수는 괄호·따옴표·쉼표·번호·닫는 괄호를 확인한다.
3. 이름 부분은 get으로 공백까지 보존한다.
4. 완성된 형식에 맞으면 e에 대입하고, 아니면 failbit를 기록한다.

원문 실행 결과

원문은 이어지는 입력·출력 반복문에 다음 결과를 제시한다:

{"John Marwood Cleese", 123456}

{"Michael Edward Palin", 987654}

출처: 책 218-219쪽 / 원문 PDF 215-216쪽

조건에서 is>>c를 검사하면 읽기의 성공 여부도 확인한다. >>는 앞 공백을 지나치지만 get(c)는 그렇지 않으므로, 이름 바깥 공백은 건너뛰고 따옴표 안 공백은 이름에 보존한다. 잘못된 형식이면 failbit를 설정하여 호출 측의 스트림 조건에 실패를 전달한다. 원문은 John Marwood Cleese와 Michael Edward Palin을 공백 포함 이름의 예로 제시한다.

10.6 형식화

출처: 책 219-221쪽 / 원문 PDF 216-218쪽

조정자는 입력·출력 형식을 제어한다. 표준 조정자는 <ios>·<istream>·<ostream> 등에 있고 인자를 받는 조정자는 <iomanip>에 있다. 정수의 기본 10진 출력 외에 hex와 oct로 진법을 지정할 수 있다.

정수 출력 진법을 바꾸는 조정자

코드 출처: 책 219쪽 / 원문 PDF 216쪽

C++ 원문

```
cout << 1234 << ',' << hex << 1234 << ',' << oct << 1234 << '\n'; // 1234,4d2,2322 출력
```

코드의 의미

같은 정수를 기본 10진수·16진수·8진수 형식으로 출력하는 원문이다.

동작 과정

1. hex가 뒤 정수의 16진 출력, oct가 뒤 정수의 8진 출력을 지정한다.

원문 실행 결과

원문 명시: 1234,4d2,2322

부동소수점 형식	원문의 표현 방식	정밀도의 의미
defaultfloat	값을 잘 보존하도록 구현체가 스타일 선택	최대 자릿수
scientific	소수점 앞 한 자리와 지수	소수점 다음 최대 자릿수
fixed	정수부·소수점·소수 부분	소수점 다음 최대 자릿수

출처: 책 220-221쪽 / 원문 PDF 217-218쪽

원문은 hexfloat를 이용한 16진 부동소수점 표기도 제시한다. 부동소수점 값은 버림이 아닌 반올림으로 출력한다. precision()은 정수 값의 출력에 영향을 주지 않는다. precision과 부동소수점 형식 조정은 지속적이므로 이후 출력에도 영향을 주며, 다음 출력 한 번에만 적용된다고 가정해서는 안 된다.

10.7 파일 스트림

출처: 책 221-222쪽 / 원문 PDF 218-219쪽

<fstream>의 ifstream은 읽기, ofstream은 쓰기, fstream은 읽기와 쓰기용이다. 파일을 연 다음에는 스트림 상태를 확인하여 실제로 연결되었는지 확인해야 한다. 성공한 파일 스트림은 일반적인 istream·ostream처럼 사용할 수 있다.

파일 스트림을 연 뒤 상태 확인

코드 출처: 책 221쪽 / 원문 PDF 218쪽

C++ 원문

```
ofstream ofs {"target"};    // "o"는 "출력(output)"을 의미
if (!ofs)
    error("'target'을 쓰기용으로 열 수 없음");

ifstream ifs {"source"};    // "i"는 "입력(input)"을 의미
if (!ifs)
    error("'source'를 읽기용으로 열 수 없음 ");
```

코드의 의미

쓰기·읽기 파일 스트림을 만들고 연결이 성공했는지 확인하는 두 원문 영역이다.

동작 과정

1. 쓰기용 target과 읽기용 source를 연다.
2. 각 스트림의 상태가 실패면 error를 호출한다.

원문 실행 결과

[원문에 실행 결과 미제시]

원문 주의 사항

- error 함수의 정의와 실제 파일 내용은 이 예제에 제시하지 않는다.

출처: 책 221-222, 231쪽 / 원문 PDF 218-219, 228쪽

파일 스트림은 소유한 자원을 소멸자로 정리한다. 원문의 조언은 파일 스트림을 복사하지 않는 것이다. 세밀한 위치 제어나 열기 모드는 이 책의 범위를 벗어나며, 파일 이름과 파일 시스템 조작은 10.10으로 연결한다.

10.8 문자열 스트림

출처: 책 222-223쪽 / 원문 PDF 219-220쪽

<sstream>의 istringstream은 string에서 읽고 ostringstream은 string에 쓰며 stringstream은 둘 다 한다. ostringstream에 형식화된 출력을 수행하고 str()로 저장된 문자열을 얻는 예는 GUI 등에 표시할 문자열을 만드는 용도다. 반대로 입력 문자열을 istringstream에 넣어 형식화된 입력을 수행할 수 있다.

문자열 스트림으로 일반적인 타입 변환

코드 출처: 책 223쪽 / 원문 PDF 220쪽

C++ 원문

```
template<typename Target =string, typename Source =string>
Target to(Source arg)    // Source를 Target으로 변환
{
    stringstream interpreter;
    Target result;

    if (!(interpreter << arg)                  // arg를 스트림에 출력
        || !(interpreter >> result)           // 스트림에서 result를 읽기
        || !(interpreter >> std::ws).eof())    // 스트림에 남은 것이 있는가?
        throw runtime_error{"to<>() failed"};

    return result;
}
```

코드의 의미

문자열로 표현할 수 있는 Source에서 문자열로 표현할 수 있는 Target으로 스트림을 거쳐 변환한다.

동작 과정

1. arg를 스트림에 출력한다.
2. Target 타입 result를 읽는다.
3. 공백 처리 후 남는 입력이 없는지 검사한다.
4. 실패하면 runtime_error를 던지고 성공하면 result를 반환한다.

원문 실행 결과

[원문에 실행 결과 미제시]

출처: 책 223쪽 / 원문 PDF 220쪽

to()는 Source를 스트림에 출력하고 Target을 읽는 두 작업을 결합하며, 변환 후 공백 이외의 내용이 남으면 실패로 처리한다. 원문은 문자열로 표현 가능한 타입 사이의 일반적인 변환 사례로 설명한다. 함수 템플릿 인자가 추론 가능하거나 기본 값이 있으면 생략할 수 있으므로, 기본 Target=string과 Source 추론을 통해 to(1.2)처럼 쓸 수 있다.

10.9 C 스타일 입출력

출처: 책 223-224쪽 / 원문 PDF 220-221쪽

C++ 표준 라이브러리는 printf·scanf 계열도 지원한다. 원문은 대개 타입과 보안 면에서 안전하지 않고 입력을 안전하고 편리하게 구현하기 어려우며 사용자 정의 타입을 지원하지 않아 권장하지 않는다. C 스타일 입출력을 사용하지 않으면서 성능을 고려할 때 다음 호출을 제시한다.

C 스타일 입출력과의 동기화 설정

코드 출처: 책 224쪽 / 원문 PDF 221쪽

C++ 원문

```
ios_base::sync_with_stdio(false); // 큰 오버헤드를 피할 수 있음
```

코드의 의미

C 스타일 입출력을 사용하지 않으면서 입출력 성능을 고려할 때 원문이 제시하는 호출이다.

동작 과정

1. 원문은 C 입출력과의 호환성을 유지하는 데서 오는 부담을 줄이는 용도로 설명한다.

원문 실행 결과

[원문에 실행 결과 미제시]

원문 주의 사항

- C 스타일 입출력을 사용하지 않는다는 원문의 조건을 유지한다.

10.10 파일 시스템

출처: 책 224-230쪽 / 원문 PDF 221-227쪽

출처: 책 224-225쪽 / 원문 PDF 221-222쪽

<filesystem>은 시스템마다 다른 경로·파일 종류·권한·탐색 기능에 일관된 인터페이스를 제공한다. **path**는 운영체제의 기본 문자 집합과 관례를 다루는 경로 표현이다. 명령행의 이름이나 문자열에서 만들고 파일 열기에 사용할 수 있다. 실제 사용 전에는 유효성을 검사하지 않으며 무엇이 유효한지는 실행 시스템 관례에 달려 있다.

출처: 책 225쪽 / 원문 PDF 222쪽

파일 시스템은 다른 프로그램과 함께 사용된다. exists()로 파일 존재를 확인했어도 다음 is_regular_file() 호출 전에 파일이 없어질 수 있다. 이처럼 두 연산 사이의 외부 상태 변화 가능성을 원문이 경고한다.

디렉터리 순회와 파일 시스템 예외

코드 출처: 책 226쪽 / 원문 PDF 223쪽

C++ 원문

```
void print_directory(path p)
try
{
    if (is_directory(p)) {
        cout << p << ":\n";
        for (const directory_entry& x : directory_iterator{p})
            cout << " " << x.path() << '\n';
    }
}
catch (const filesystem_error& ex) {
    cerr << ex.what() << '\n';
}
```

코드의 의미

경로가 디렉터리일 때 항목의 경로를 출력하고 파일 시스템 예외를 보고한다.

동작 과정

1. is_directory로 검사한다.
2. directory_iterator로 항목을 순회한다.
3. filesystem_error를 잡아 what()을 cerr에 출력한다.

원문 실행 결과

[원문에 실행 결과 미제시]

출처: 책 226-227쪽 / 원문 PDF 223-224쪽

directory_entry는 한 항목이고 directory_iterator는 디렉터리 항목을 순회한다. 하위 디렉터리까지 필요하면 recursive_directory_iterator를 사용한다. 사전 순서 출력이 필요하면 원문은 path들을 vector에 담아 정렬하는 방법을 제시한다. 순회가 자동으로 정렬된다고 설명하지 않는다.

경로 기능	원문에서의 역할
/= 와 +=	파일 이름 구분자를 넣어 연결 / 구분자 없이 연결
native(), string(), generic_string()	네이티브 표현 또는 제네릭 표현의 문자열 얻기
filename(), stem(), extension()	파일 이름·꼬리·확장자 부분 다루기
begin(), end()	경로 구성 요소 시퀀스의 시작·끝
==, !=, 비교 연산·스트림 입출력	상등·사전 순서 비교 및 경로 입출력
u8path(s)	UTF-8로 인코딩된 s에서 경로 생성

출처: 책 227-228쪽 / 원문 PDF 224-225쪽

원문 표는 value_type이 POSIX에서 char, 윈도우에서 wchar_t라고 설명한다. string_type은 해당 문자 타입의 basic_string이며 const_iterator는 const 양방향 반복자, iterator는 그 별칭이다. [원문 확인 필요] extension()의 옮긴이 설명은 ".을 제외한 확장자"로 인쇄되어 있어 원문 표기를 그대로 알리고 임의로 수정하지 않는다.

파일 시스템 연산	원문의 기능 분류
exists, file_size	존재 여부·바이트 수 조회
copy, copy_file	파일 또는 디렉터리 복사 / 파일 내용 복사
create_directory, create_directories	상위 경로가 존재해야 하는 생성 / 중간 경로도 생성
current_path 계열	현재 작업 디렉터리 처리
remove	파일 또는 빈 디렉터리 삭제

출처: 책 229쪽 / 원문 PDF 226쪽

원문은 대부분의 연산에 실패하면 filesystem_error를 던지는 기본 버전과 error_code를 추가 인자로 받아 결과를 확인하는 버전이 있다고 설명한다. 실패가 자주 일어나 예외를 던지는 일이 예외적 상황과 맞지 않는 경우에는 에러 코드를 사용하라고 조언한다. 권한 등을 받는 세부 오버로드는 여기서 다루지 않는다.

출처: 책 229-230쪽 / 원문 PDF 226-227쪽

파일 종류는 is_block_file·is_character_file·is_directory·is_empty·is_fifo·is_other·is_regular_file·is_socket·is_symlink로 질의하고 status_known으로 상태를 아는지 확인한다. 알 수 없는 종류는 기타로 분류한다. 경로 이름·자연어·인코딩은 복잡하지만 이 추상화가 이식성과 단순함을 제공한다고 정리한다.

10.11 조언

출처: 책 230-232쪽 / 원문 PDF 227-229쪽

- 일반 출력은 cout, 에러 보고는 cerr을 사용하고 <<와 >>를 연결해 의도를 명확하게 한다.
- 형식이 잘못된 입력을 고려하고 회복 가능한 입력 실패는 스트림 상태로 처리한다.
- 문자 수준 입력은 꼭 필요할 때만 사용한다. 원문은 endl을 피하라고 조언한다.

- 사용자 정의 타입에 의미 있는 텍스트 표현이 있으면 입출력 연산자를 제공한다.
- 정밀도와 형식 설정은 이후 출력에도 지속된다.
- 파일 사용 전에 연결 상태를 확인하고 메모리상 형식화에는 문자열 스트림을 쓴다.
- 특정 운영체제 인터페이스보다 <filesystem>을 사용한다.

중요 내용 요약

- 입출력 연산의 반환 스트림을 검사하면 성공 여부와 반복 종료를 표현할 수 있다.
- >>와 getline·get의 공백 처리 차이를 구분한다.
- 사용자 정의 입력 연산자는 형식을 검사하고 실패 상태를 스트림에 기록한다.
- 형식 설정의 지속성과 precision 의미는 출력 형식에 따라 해석한다.
- 경로 존재 확인 후에도 외부 프로그램에 의해 상태가 바뀔 수 있다.

혼동하기 쉬운 내용

파일과 문자열 스트림의 대응

출처: 책 221-222쪽 / 원문 PDF 218-219쪽

방향	파일	문자열
읽기	ifstream	istringstream
쓰기	ofstream	ostringstream
읽기·쓰기	fstream	stringstream

파일 시스템 에러 보고

출처: 책 229쪽 / 원문 PDF 226쪽

방식	보고·확인
기본 버전	실패 시 filesystem_error 예외
error_code 인자 버전	인자로 전달한 에러 코드 확인

장 마무리 복습

- ostream과 istream의 변환 방향을 값과 문자 시퀀스에 연결한다.
- 공백을 건너뛰는 >>, 한 행의 getline, 공백을 보존하는 get의 목적을 구별한다.
- 상태 검사·failbit 설정·clear로 회복하는 흐름을 원문 사례로 복습한다.
- 스트림은 파일과 문자열에도 같은 형식화 연산을 적용한다.
- path 표현과 실제 파일 상태, 디렉터리 순서와 정렬은 구별한다.

원문 확인 메모

- 종료 표식 예제의 is/cin 혼용과 extension 설명은 원문 확인 필요로 표시했다. 책에서 다루지 않는 상세 열기 모드·인코딩·권한 오버로드는 보충하지 않았다.

제11장. 컨테이너

출처: 책 233-252쪽 / 원문 PDF 230-249쪽

학습 내용 개요

- 컨테이너는 객체를 저장하는 자원 핸들이며 시퀀스를 정의한다.
- vector의 크기·용량·요소 값 의미와 구간 검사를 이해한다.
- list·map·unordered_map을 저장 구조와 조회 조건으로 구분한다.

설명의 흐름

1. 연속 저장 vector와 성장 방식
2. 요소·다형성·구간 검사
3. 연결 구조와 연관 컨테이너
4. 공통 연산과 선택 기준

11.1 소개

출처: 책 234쪽 / 원문 PDF 231쪽

컨테이너는 객체를 저장하는 일이 주목적인 클래스다. 작업에 적합한 컨테이너와 기본 연산을 선택하는 일은 프로그램 구성의 중요한 단계다. 이 장은 이름과 전화번호를 담는 Entry의 전화번호부로 표준 컨테이너를 비교한다. 예제에서는 불필요한 복잡성을 피하려고 전화번호를 int로 표현한다.

11.2 vector

출처: 책 234-238쪽 / 원문 PDF 231-235쪽

출처: 책 234-235쪽 / 원문 PDF 231-232쪽

vector<T>는 T 타입 요소가 메모리에 연속적으로 저장되는 시퀀스다. 전형적인 구현은 첫 요소, 사용 중인 요소의 끝, 할당 공간의 끝을 가리키는 정보와 할당자로 구성된다. 기본 할당자는 new와 delete로 메모리를 관리한다.

출처: 책 235-236쪽 / 원문 PDF 232-233쪽

인덱스는 0에서 시작하며 size()는 실제 요소 수를 알려준다. 요소 목록을 주는 {} 초기화와 초기 크기를 지정하는 () 초기화를 구별한다. 크기만 지정하면 요소는 타입의 기본 값으로 초기화되고, 두 번째 인자로 별도의 초기 값을 지정할 수 있다. 구간 for는 vector가 제공하는 요소 구간 전체를 순회한다.

vector의 구간 for 순회

코드 출처: 책 235쪽 / 원문 PDF 232쪽

C++ 원문

```
void print_book(const vector<Entry>& book)
{
    for (const auto& x : book) // "auto"에 대해서는 1.4절 참고
        cout << x << '\n';
}
```

코드의 의미

각 Entry를 복사하지 않고 const 참조로 받아 순서대로 출력하는 원문 함수다. Entry의 << 연산이 정의되어 있다는 전제가 있다.

동작 과정

1. book의 구간에서 각 요소 x를 참조한다.
2. 각 요소를 cout으로 전달하고 줄바꿈 문자를 출력한다.

원문 실행 결과

[원문에 실행 결과 미제시]

출처: 책 236-237쪽 / 원문 PDF 233-234쪽

size와 capacity는 다르다. size는 실제 요소 수, capacity는 재할당 없이 저장할 수 있는 슬롯 수다. reserve()는 요소를 위한 공간을 확보하며, 새 공간을 할당하면 기존 요소를 그곳으로 이동시킨다. push_back()은 끝에 요소를 추가하고 size를 1 늘린다. 원문의 간략한 Vector 구현은 공간이 부족할 때 용량을 늘려 매번 재할당하는 일을 피한다.

용량을 확보한 뒤 요소를 생성하는 push_back

코드 출처: 책 237쪽 / 원문 PDF 234쪽

Vector 클래스 선언은 생략하고, 원문의 push_back 함수 정의 전체를 발췌했습니다.

C++ 원문

```
template<typename T>
void Vector<T>::push_back(const T& t)
{
    if (capacity()<size()+1)        // t를 저장할 공간이 있는지 확인
        reserve(size()==0?8:2*size()); // 용량을 2배로 증가
    new(space) T{t}; // *space 를 t로 초기화
    ++space;
}
```

코드의 의미

원문의 간략한 Vector 클래스에서 공간 부족 시 reserve()를 호출하고 사용하지 않던 첫 슬롯에 T를 생성한다. 클래스 선언은 앞선 원문에 있으며 여기서는 함수 정의를 발췌했다.

동작 과정

1. 현재 용량이 새 요소 하나를 담기에 충분한지 검사한다.
2. 필요하면 빈 경우 8, 그 외에는 size의 두 배를 요청한다.
3. space에 t로 요소를 생성한 후 space를 증가시킨다.

원문 실행 결과

[원문에 실행 결과 미제시]

출처: 책 237-238쪽 / 원문 PDF 234-235쪽

성능 향상을 기대한다는 이유만으로 reserve()를 호출할 필요는 없다. 원문은 vector의 증가 전략을 신뢰하고, 요소 포인터나 반복자의 유효성을 위해 재할당을 막아야 할 때 명시적으로 reserve()를 사용하도록 권한다. 대입과 초기화로 vector를 복사하면 각 요소도 복사되어 독립된 복사본이 생긴다. 요소가 많으면 비용이 클 수 있으므로 복사가 바람직하지 않을 때 참조·포인터 또는 이동 연산을 사용한다.

11.2.1 요소

출처: 책 238-239쪽 / 원문 PDF 235-236쪽

표준 컨테이너에는 내장 타입, 사용자 정의 타입, 포인터 등 거의 모든 타입을 요소로 사용할 수 있다. 요소를 삽입하면 값이 컨테이너 안으로 복사된다. 정수 7을 넣으면 요소 자체가 7의 값을 가지며, 다른 곳의 7을 가리키는 참조나 포인터를 저장하는 것이 아니다. 이 방식은 간결하고 빠르게 접근할 수 있는 저장 구조를 만든다.

가상 함수에 의존하는 클래스 계층의 **다형적인 동작**을 유지하려면 객체 자체 대신 포인터나 스마트 포인터를 요소로 저장한다. 원문은 vector<Shape>로 Circle이나 Smiley를 저장하려는 방식을 피하고, vector<unique_ptr<Shape>>를 사용하는 형태를 제시한다.

11.2.2 구간 검사

출처: 책 239-241쪽 / 원문 PDF 236-238쪽

출처: 책 239-240쪽 / 원문 PDF 236-237쪽

표준 vector의 operator[]는 구간 검사를 보장하지 않는다. book[book.size()]처럼 마지막 요소 다음을 참조하는 코드는 유효한 접근이 아니다. at()은 구간 밖 인덱스에 out_of_range를 던진다. 원문의 Vec는 vector의 생성자를 상속하고 operator[]를 at()에 연결한다.

at()에 연결하여 구간 검사하는 Vec

코드 출처: 책 239쪽 / 원문 PDF 236쪽

C++ 원문

```
template<typename T>
class Vec : public std::vector<T> {
public:
    using vector<T>::vector; // (Vec라는 이름으로) vector의 생성자를 사용

    T& operator[](int i) // 구간 검사
        { return vector<T>::at(i); }

    const T& operator[](int i) const // const 객체용 구간 검사; 4.2.1절
        { return vector<T>::at(i); }
};
```

코드의 의미

vector에서 다른 기능을 상속하고 const 여부에 맞춘 두 인덱싱 연산을 at() 호출로 재정의한다.

원문 실행 결과

[원문에 실행 결과 미제시]

원문 주의 사항

- at()은 인덱스가 구간을 벗어나면 out_of_range를 던진다.

출처: 책 240-241쪽 / 원문 PDF 237-238쪽

Vec의 구간 밖 접근은 예외를 발생시켜 호출자가 잡을 수 있게 한다. 잡지 않은 예외가 있으면 프로그램은 잘 정의된 방식으로 종료된다. 원문은 main()의 try 블록과 기본 예외 핸들러에서 cerr로 진단을 출력하는 예도 제시한다. 구간 검사의 비용은 하드웨어·최적화·접근 방식에 따라 다르므로, 표준은 모든 인덱싱에 검사를 강제하지 않는다. 일부 구현은 디버그 검사 기능을 제공한다.

출처: 책 241쪽 / 원문 PDF 238쪽

구간 for나 begin()/end() 범위를 이용한 순회는 추가적인 구간 검사 비용 없이 구간 밖 접근을 피한다. 코드에서 at()을 직접 사용할 수 있다면 Vec를 따로 만들 필요가 없다.

11.3 list

출처: 책 241-244쪽 / 원문 PDF 238-241쪽

출처: 책 241-242쪽 / 원문 PDF 238-239쪽

list는 이중 연결 리스트다. 다른 요소를 이동시키지 않고 시퀀스 중간에 삽입·삭제할 때 사용한다. vector처럼 인덱스로 접근하는 대신 처음부터 원하는 요소를 찾는다. begin()은 첫 요소를, end()는 마지막 요소 다음을 가리키는 반복자를 반환한다.

반복자로 전화번호부 선형 탐색

코드 출처: 책 243쪽 / 원문 PDF 240쪽

C++ 원문

```
int get_number(const string& s)
{
    for (auto p = phone_book.begin(); p!=phone_book.end(); ++p)
        if (p->name==s)
            return p->number;
    return 0; // 0 은 "수를 찾지 못함"을 의미
}
```

코드의 의미

list의 begin()부터 end()까지 이름을 비교해 일치하는 항목의 number를 반환한다. 0은 찾지 못했음을 표시한다.

동작 과정

1. p가 각 요소를 가리키게 이동한다.
2. name이 s와 같으면 number를 반환한다.
3. 끝에 이르면 0을 반환한다.

원문 실행 결과

[원문에 실행 결과 미제시]

출처: 책 243쪽 / 원문 PDF 240쪽

반복자 p에 대해 *p는 요소를 참조하고 ++p는 다음 요소로 진행한다. p->m은 (*p).m과 같다. insert(p,elem)은 elem을 복사한 요소를 p 앞에 넣으며, p가 end()여도 된다. erase(p)는 p의 요소를 제거하고 소멸시킨다. 원문의 전화번호부 예제는 vector로도 작성할 수 있고, 짧은 시퀀스에서는 삽입·삭제까지 vector 쪽이 더 효율적일 수 있으므로 직관만으로 list를 선택하지 않는다.

출처: 책 243-244쪽 / 원문 PDF 240-241쪽

forward_list는 순방향 순회만 제공하는 단일 연결 리스트다. 이전 요소 포인터가 없어 공간을 아끼며 빈 객체의 크기는 포인터 하나다. 요소 개수도 저장하지 않으므로 개수를 알려면 직접 세어야 한다. 원문은 이 특성이 필요할 때 선택하도록 한다.

11.4 map

출처: 책 244-245쪽 / 원문 PDF 241-242쪽

map은 키와 값의 쌍을 담고 조회에 최적화된 연관 컨테이너다. 원문은 일반적인 구현을 균형 이진 탐색 트리, 특히 레드-블랙 트리로 설명한다. 연관 배열 또는 딕셔너리라고도 한다. 키 타입의 값으로 인덱싱하면 대응하는 매핑된 타입의 값을 얻는다.

map의 키로 전화번호 조회

코드 출처: 책 245쪽 / 원문 PDF 242쪽

동일 페이지의 map 초기화는 생략하고 get_number 함수 정의 전체를 발췌했습니다.

C++ 원문

```
int get_number(const string& s)
{
    return phone_book[s];
}
```

코드의 의미

원문의 map<string,int> 전화번호부에서 이름 s로 번호를 조회한다. 같은 페이지의 map 초기화 코드는 생략했다.

원문 실행 결과

[원문에 실행 결과 미제시]

원문 주의 사항

- s가 없으면 기본 값 0인 항목이 삽입된다. 삽입을 원치 않으면 find()와 insert()를 사용한다.

출처: 책 245쪽 / 원문 PDF 242쪽

없는 키에 대한 []는 조회만 하는 연산이 아니다. 키를 못 찾으면 값 타입의 기본 값을 가진 항목이 삽입된다. 전화번호부에서는 int의 기본 값 0이 유효하지 않은 번호를 나타낸다. 이런 항목이 들어가는 것을 원치 않으면 [] 대신 find()와 insert()를 사용한다.

11.5 unordered_map

출처: 책 245-248쪽 / 원문 PDF 242-245쪽

출처: 책 245-246쪽 / 원문 PDF 242-243쪽

map의 조회 비용은 요소 수 n에 대해 O(log(n))이다. **unordered_map**은 해시 테이블을 이용하며 순서 비교 함수를 요구하지 않는다. map과 비슷하게 초기화하고 인덱싱할 수 있지만 요소 사이의 순서를 유지하지 않는다. 표준 라이브러리는 string·내장 타입·여러 라이브러리 타입의 기본 해시 함수를 제공한다.

출처: 책 246-248쪽 / 원문 PDF 243-245쪽

사용자 정의 타입에는 해시 함수 객체를 전달하거나 std::hash를 특수화할 수 있다. 원문의 Record 예제는 이름과 product_code의 표준 해시를 배타적 논리합(^)으로 결합한다. 좋은 해시 함수에는 데이터에 대한 지식과 실험이 필요할 수 있다. 좋은 함수를 쓸 때 큰 unordered_map은 map보다 훨씬 빠를 수 있지만, 잘못된 해시 함수에서는 최악의 경우가 map보다 훨씬 느릴 수 있다.

11.6 컨테이너 개요

출처: 책 248-250쪽 / 원문 PDF 245-247쪽

출처: 책 248쪽 / 원문 PDF 245쪽

표준 컨테이너	원문에서 정리한 역할
vector<T> / list<T> / forward_list<T>	가변 크기 벡터 / 이중 연결 리스트 / 단일 연결 리스트
deque<T>	양방향 큐
set<T> / multiset<T>	값만 가진 집합 / 값의 중복을 허용하는 집합
map<K,V> / multimap<K,V>	키와 값의 연관 배열 / 키의 중복을 허용하는 map
unordered_map / unordered_multimap	해시 조회를 사용하는 map / multimap
unordered_set / unordered_multiset	해시 조회를 사용하는 set / multiset

출처: 책 249-250쪽 / 원문 PDF 246-247쪽

컨테이너는 std에 정의되고 <vector>, <list>, <map> 등에서 제공된다. queue·stack·priority_queue는 컨테이너 어댑터이며 array와 bitset은 고정 크기 구조다. 여러 컨테이너는 일관된 표기와 의미를 갖도록 설계되어, 알고리즘을 개별 컨테이너 타입과 독립적으로 기술할 수 있다.

출처: 책 249-250쪽 / 원문 PDF 246-247쪽

연산	의미와 주의
value_type	요소 타입
begin()/end(), cbegin()/cend()	첫 요소 / 마지막 요소 다음; const 반복자도 제공
size()/empty()	요소 수 / 비어 있는지 검사
capacity()/reserve(k)	새 할당 없는 저장 용량 / 용량 확보
resize(k)	요소 수 변경; 추가 요소는 value_type{}
[k] / at(k)	구간 검사 없는 접근 / 구간 밖에서 예외
push_back(x) / emplace_back(a)	끝에 x 추가 / 인자로 새 요소를 직접 생성
insert(p,x) / erase(p)	p 앞에 삽입 / p 요소 제거
= / == / <	대입 / 요소의 상등성 / 사전 순 비교

출처: 책 250쪽 / 원문 PDF 247쪽

각 컨테이너에 맞는 연산을 선택해야 한다. vector는 연속 저장에 따른 저렴한 인덱싱·순회를 제공하지만 중간 삽입·삭제 때 요소 이동이 필요하다. list는 그 반대의 성격을 갖는다. 빈 경우가 많다면 한 워드만 필요한 forward_list를 고려할 수 있다. 배치 연산은 객체를 먼저 만들어 복사하는 대신, 전달한 생성자 인자로 컨테이너의 새 저장 공간에 요소를 직접 만든다.

11.7 조언

출처: 책 251-252쪽 / 원문 PDF 248-249쪽

- 표준 컨테이너를 익히고 기본 선택으로 vector를 고려한다. 성능은 직관이 아니라 측정으로 확인한다.
- 순회에는 구간 for나 시작/끝 반복자를 사용한다. 구간 검사가 필요하면 at()을 사용하며 []가 검사한다고 가정하지 않는다.
- vector의 크기 변경 후 반복자를 그대로 사용하지 않는다. 포인터·반복자 유효성을 위한 재할당 방지와 단순 성능 추측을 구별한다.
- 컨테이너 요소는 값이다. 다형적인 동작이 필요하면 포인터를 저장한다. 컨테이너 전달은 참조로, 반환은 값으로 한다.
- 많은 데이터의 빠른 조회에는 순서 없는 컨테이너, 순서 있는 순회에는 map·set 같은 순서 있는 연관 컨테이너를 고려한다.
- 크기로 생성할 때 ()와 요소 목록으로 생성할 때 {}를 구별한다. 해시 함수의 적절성은 실험으로 확인한다.

중요 내용 요약

- 기본 컨테이너는 vector로 고려하되 성능은 측정한다.
- size는 요소 수, capacity는 저장 가능한 슬롯 수다.
- []의 구간 검사와 없는 키 삽입 동작은 컨테이너에 따라 다르다.
- 컨테이너의 값 저장과 다형적인 객체 포인터 저장을 구분한다.

혼동하기 쉬운 내용

map과 unordered_map

출처: 책 245-248쪽 / 원문 PDF 242-245쪽

비교 기준	map	unordered_map
조회 구조	균형 이진 탐색 트리	해시 테이블
필요한 함수	순서 비교 함수; 기본적으로 <	해시 함수
요소 순서	순서 있는 시퀀스	순서 유지 안 함
성능 조건	조회 O(log(n))	좋은 해시는 큰 데이터에서 빠름; 나쁜 해시는 최악의 경우 훨씬 느림

장 마무리 복습

- vector의 연속 저장·복사·성장·reserve와 반복자 유효성의 관계를 복습한다.
- 구간 밖 접근을 at()·구간 for·시작/끝 반복자로 다루는 방식을 정리한다.
- list의 삽입·삭제와 vector의 짧은 시퀀스 효율을 함께 고려한다.
- map의 []는 없는 키를 삽입하며 unordered_map은 해시 품질과 순서 유무를 고려해야 한다.

제12장. 알고리즘

출처: 책 253-274쪽 / 원문 PDF 250-271쪽

학습 내용 개요

- 반복자로 표현한 구간에 공통 알고리즘을 적용한다.
- 컨테이너·알고리즘 분리, 검색 실패, 출력 공간 조건을 확인한다.
- 술어·스트림 반복자·컨셉·실행 정책으로 알고리즘 사용을 확장한다.

설명의 흐름

1. 반개구간과 출력 삽입 반복자
2. 반복자 사용과 구현 타입
3. 입출력 및 술어
4. 알고리즘 표·원문 시점의 Ranges 컨셉
5. 컨테이너 래퍼와 병렬 정책

12.1 소개

출처: 책 254-255쪽 / 원문 PDF 251-252쪽

표준 라이브러리 알고리즘은 컨테이너에 저장한 요소의 정렬·출력·검색·부분집합 추출 같은 공통 작업을 제공한다. 입력 시퀀스는 첫 요소를 포함하고 끝 위치는 포함하지 않는 **반개구간 [begin:end)**이며, 첫 요소와 마지막 요소 다음을 가리키는 반복자 쌍으로 나타낸다.

sort는 비교 연산으로 입력 구간을 정렬하고 unique_copy는 이웃한 중복을 복사하지 않는다. 기존 출력 위치를 지정하면 그 뒤 요소를 덮어쓰므로 충분한 출력 공간이 있어야 한다. back_inserter는 컨테이너 뒤에 삽입하는 반복자를 만들어 필요에 따라 컨테이너를 확장한다. 이 방식은 미리 크기를 정하거나 C 스타일 realloc()으로 직접 관리할 필요를 줄인다.

정렬 후 중복 없이 새 list에 삽입

코드 출처: 책 255쪽 / 원문 PDF 252쪽

C++ 원문

```
list<Entry> f(vector<Entry>& vec)
{
    list<Entry> res;
    sort(vec.begin(),vec.end());
    unique_copy(vec.begin(),vec.end(),back_inserter(res)); // res 뒤에 붙이기
    return res;
}
```

코드의 의미

vec를 정렬하고 이웃한 중복을 제외한 요소를 res 뒤에 넣는다. Entry에는 순서 비교와 상등성 비교가 정의되어 있어야 한다.

동작 과정

1. sort가 vec의 요소를 정렬한다.
2. unique_copy가 연속 중복을 제외한다.
3. back_inserter가 res를 확장하며 요소를 추가하고 res를 값으로 반환한다.

원문 실행 결과

[원문에 실행 결과 미제시]

12.2 반복자의 사용

출처: 책 255-259쪽 / 원문 PDF 252-256쪽

출처: 책 255-256쪽 / 원문 PDF 252-253쪽

begin()과 end()는 컨테이너에서 반복자를 얻는 대표적인 함수다. find는 찾은 요소의 반복자를 반환하고, 찾지 못하면 입력 구간의 end()를 반환한다. 그러므로 반환값을 끝 반복자와 비교하여 검색 성공을 판정한다.

출처: 책 256-258쪽 / 원문 PDF 253-255쪽

원문의 find_all은 일치하는 값 자체 대신 그 위치를 가리키는 반복자들의 vector를 반환한다. 이 반복자로 원래 요소를 바꿀 수도 있으므로, 변경할 문자열이나 컨테이너는 const가 아닌 참조로 받는다. 처음에는 string 전용으로 작성하고 이후 컨테이너 C와 값 V의 템플릿으로 일반화한다.

반복자 타입 별칭을 사용한 제네릭 find_all

코드 출처: 책 258쪽 / 원문 PDF 255쪽

C++ 원문

```
template<typename T>
using Iterator = typename T::iterator; // T의 반복자

template<typename C, typename V>
vector<Iterator<C>> find_all(C& c, V v) // c에서 v가 나타나는 곳을 모두 찾기
{
    vector<Iterator<C>> res;
    for (auto p = c.begin(); p!=c.end(); ++p)
        if (*p==v)
            res.push_back(p);
    return res;
}
```

코드의 의미

일치하는 요소를 가리키는 컨테이너 C의 반복자들을 반환한다. 원문의 string·list<double>·vector<string>에 공통으로 쓰는 알고리즘이다.

원문 실행 결과

[원문에 실행 결과 미제시]

출처: 책 258-259쪽 / 원문 PDF 255-256쪽

typename C::iterator의 typename은 iterator가 값이 아닌 타입임을 컴파일러에 알려준다. 타입 별칭 Iterator<T>는 이 세부사항을 숨긴다. 알고리즘은 반복자로 데이터에 적용되므로 저장 컨테이너를 알 필요가 없고, 컨테이너 역시 어떤 알고리즘이 쓰이는지 알 필요 없이 반복자를 제공한다. 이 분리가 일반적이고 유연한 소프트웨어의 바탕이다.

12.3 반복자 타입

출처: 책 259-260쪽 / 원문 PDF 256-257쪽

반복자는 특정 타입의 객체이며 저장 구조에 맞는 정보를 가진다. vector에서는 단순 포인터가 합리적인 구현일 수 있고, vector 포인터와 인덱스를 함께 저장하면 구간 검사를 할 수 있다. list에서는 다음 링크를 따라가야 하므로 단순 요소 포인터가 아니라 링크를 가리키는 정보가 필요하다.

구현은 달라도 ++로 다음 요소를 가리키고 *로 요소를 참조하는 공통 의미와 명명 규칙을 따른다. 이러한 요구를 표현한 것이 Iterator 컨셉이다. 사용자에게 내부 구현 전체가 필요한 것은 아니며 각 컨테이너의 iterator·const_iterator라는 관례적인 이름을 이용하면 된다.

12.4 스트림 반복자

출처: 책 261-263쪽 / 원문 PDF 258-260쪽

출처: 책 261쪽 / 원문 PDF 258쪽

요소 시퀀스는 컨테이너에만 있는 것이 아니다. 입력 스트림도 값의 시퀀스를 만들고 출력 스트림도 시퀀스를 받는다. ostream_iterator는 스트림과 출력 타입을 지정해 만들며, 역참조 위치에 대입하면 스트림으로 출력한다. ++는 배열 포인터와 같은 사용 방식을 흉내 낸다. istream_iterator는 입력 스트림을 읽기 전용 컨테이너처럼 다루게 한다.

파일 단어를 정렬하고 중복 없이 출력

코드 출처: 책 262쪽 / 원문 PDF 259쪽

C++ 원문

```
int main()
{
    string from, to;
    cin >> from >> to; // 원본과 출력 파일 이름 가져오기

    ifstream is {from}; // 파일 "from"의 입력 스트림
    istream_iterator<string> ii {is}; // 스트림에 사용할 입력 반복자
    istream_iterator<string> eos {}; // 입력(의 끝을 나타내는) 보초 값

    ofstream os {to}; // 파일 "to"의 출력 스트림
    ostream_iterator<string> oo {os,"\n"}; // 스트림의 출력 반복자

    vector<string> b {ii,eos}; // b는 입력으로부터 초기화된 벡터
    sort(b.begin(),b.end()); // 버퍼를 정렬

    unique_copy(b.begin(),b.end(),oo); // 버퍼를 출력에 복사, 중복된 값은 버림

    return !is.eof() || !os; // 에러 상태 반환(1.2.1절, 10.4절)
}
```

코드의 의미

입력·출력 파일 이름을 읽어 단어를 vector에 저장하고 정렬한 뒤 중복을 제외해 출력하는 원문 프로그램이다.

동작 과정

1. 입력 반복자 ii와 끝 보초 eos로 vector를 초기화한다.
2. 정렬한 버퍼에 unique_copy를 적용한다.
3. oo를 통해 단어마다 줄바꿈을 붙여 출력하고 스트림 상태로 반환값을 정한다.

원문 실행 결과

[원문에 실행 결과 미제시]

출처: 책 262-263쪽 / 원문 PDF 259-260쪽

기본 생성한 istream_iterator는 입력 끝의 보초 값으로 쓰인다. ostream_iterator의 두 번째 인자는 출력 값 사이의 구분자다. 예제는 파일의 단어를 vector로 읽고 정렬한 뒤 unique_copy로 중복을 제거해 출력한다. 원문은 set을 사용해 입력 단계에서 중복을 없애고 순서를 유지하면 sort·unique_copy 대신 copy만 쓸 수 있다는 대안도 제시한다. 코드 길이를 줄이는 선택은 취향과 가독성을 함께 고려한다.

12.5 술어

출처: 책 263-264쪽 / 원문 PDF 260-261쪽

술어는 알고리즘이 각 요소에 적용할 조건을 전달하는 수단이다. find가 지정 값과 같은 요소를 찾는 데 비해 find_if는 전달된 술어가 참이 되는 첫 요소를 찾는다. 원문의 map 예제는 값 부분이 42보다 큰 쌍을 찾는다. 비교 기준을 저장한 함수 객체와 람다 표현식 모두 사용할 수 있다.

람다 술어를 전달한 find_if

코드 출처: 책 264쪽 / 원문 PDF 261쪽

원문 map 검색 설명 중 람다 호출 블록 전체를 발췌했습니다. m의 정의와 Greater_than 대안은 생략했습니다.

C++ 원문

```
auto p = find_if(m.begin(), m.end(), [](const pair<string,int>& r) {
    return r.second>42;
});
```

코드의 의미

원문의 map<string,int>에서 값 부분이 42보다 큰 첫 항목을 찾는 람다 버전이다. m의 정의 및 함수 객체 버전은 같은 설명에서 발췌하지 않았다.

원문 실행 결과

[원문에 실행 결과 미제시]

원문 주의 사항

- 술어는 적용 대상 요소를 변경하지 않는다.

출처: 책 264쪽 / 원문 PDF 261쪽

술어는 적용 대상 요소를 변경하면 안 된다. 원문의 Greater_than은 비교값을 val에 저장하고 pair의 second를 이 값과 비교한다. 람다 버전도 같은 비교를 직접 표현한다.

12.6 알고리즘 요약

출처: 책 264-266쪽 / 원문 PDF 261-263쪽

출처: 책 264-266쪽 / 원문 PDF 261-263쪽

이 책에서 표준 알고리즘은 요소 시퀀스에 적용되는 함수 템플릿 연산이다. std에 정의되고 <algorithm>에서 제공된다. 컨테이너뿐 아니라 string과 내장 배열에도 적용할 수 있다.

출처: 책 265쪽 / 원문 PDF 262쪽

알고리즘	구간에 대한 역할
for_each	모든 요소에 함수 적용
find / find_if	지정 값 / 술어를 만족하는 첫 위치
count / count_if	지정 값 / 술어에 맞는 요소 개수
replace / replace_if	지정 값 / 술어에 맞는 요소를 새 값으로 대체
copy / copy_if / move	전체 복사 / 조건에 맞는 요소 복사 / 이동
unique_copy	이웃한 중복을 제외하고 복사
sort	기본 < 또는 주어진 비교 연산으로 정렬
equal_range	정렬된 구간에서 같은 값의 부분 구간을 찾음; 기본적으로 이진 탐색
merge	정렬된 두 구간을 기본 또는 지정 비교 연산으로 병합

출처: 책 266쪽 / 원문 PDF 263쪽

일부 알고리즘은 요소 값을 변경하지만 입력 시퀀스 자체에 요소를 추가하거나 제거하지는 않는다. 반복자만으로는 소유 컨테이너를 알 수 없기 때문이다. 크기를 바꾸려면 back_inserter처럼 컨테이너를 아는 수단이나 push_back·erase 같은 컨테이너 연산이 필요하다. 원문은 직접 루프를 만들기 전에 잘 설계된 표준 알고리즘으로 표현할 수 있는지 살피도록 권한다.

12.7 컨셉(C++20)

출처: 책 266-272쪽 / 원문 PDF 263-269쪽

출처: 책 266쪽 / 원문 PDF 263쪽

이 절은 원문 집필 시점의 Ranges Technical Specification과 C++20에 대한 전망을 설명한다. 원문은 <experimental/ranges>의 컨셉을 소개하며 비슷한 기능이 C++20의 std에 들어가기를 기대한다고 쓴다. 이 노트는 그 시점의 명칭과 조건을 그대로 다룬다.

출처: 책 266-267쪽 / 원문 PDF 263-264쪽

Range는 begin/end 시퀀스의 일반화다. {begin,end}, {begin,n} 또는 {begin,pred}로 구간을 정의할 수 있다. n은 요소 수, pred는 반복자가 끝에 도달했는지 판정하는 술어다. 마지막 방식은 동적으로 정의되거나 무한한 시퀀스도 표현한다. 이 구간을 받는 알고리즘은 sort(v)와 같은 호출을 가능하게 한다.

출처: 책 267-268쪽 / 원문 PDF 264-265쪽

핵심 컨셉	원문에서 요구하는 성질
Same / DerivedFrom / ConvertibleTo	같은 타입 / 파생 관계 / 변환 가능
CommonReference / Common	공통 참조 타입 / 공통 타입을 가짐
Integral / SignedIntegral / UnsignedIntegral	정수 / 부호 있는 정수 / 부호 없는 정수
Assignable / SwappableWith / Swappable	대입 가능 / 두 타입 값의 교환 가능 / 같은 타입 사이의 교환 가능

출처: 책 268쪽 / 원문 PDF 265쪽

Common은 여러 타입을 공통 타입 c로 변환한 뒤에도 비교가 합리적이어야 하는 알고리즘을 기술하는 데 쓰인다. 원문은 string과 char*, int와 double의 비교를 들고 string과 int는 비교하지 않는다고 구별한다. 라이브러리가 적당한 정의를 제공하지 않는 경우가 아니라면 common_type_t를 직접 특수화할 필요가 없다.

출처: 책 268-269쪽 / 원문 PDF 265-266쪽

비교·객체 컨셉	핵심 관계
Boolean	불리언 값으로 사용 가능
WeaklyEqualityComparableWith / WeaklyEqualityComparable	서로 다른 타입 / 같은 타입 값의 ==·!= 비교
EqualityComparableWith / EqualityComparable	상등성 비교; 같은 타입 버전은 T,T 관계
StrictTotallyOrderedWith / StrictTotallyOrdered	<·<=·>·>=를 이용한 완전 순서; 같은 타입 버전은 T,T 관계
Destructible / Constructible / DefaultConstructible	소멸 및 주소 연산 / 주어진 인자로 생성 / 기본 생성
MoveConstructible / CopyConstructible	이동 생성 / 복사와 이동 생성
Movable / Copyable	이동 생성·대입·교환 / 복사 생성 및 복사 대입을 포함
Semiregular / Regular	Copyable와 기본 생성 / 여기에 상등성 비교를 포함

출처: 책 269쪽 / 원문 PDF 266쪽

원문은 WeaklyEqualityComparableWith와 WeaklyEqualityComparable를 혼용하면 오버로드 기회가 줄 수 있다고 경고한다. Regular 타입은 int처럼 동작해 사용이 단순해지는 이상적인 성격으로 설명하며, ==가 없는 기본적인 클래스는 Semiregular로 구분한다.

출처: 책 270쪽 / 원문 PDF 267쪽

호출 가능 컨셉	원문에서 설명한 의미
Invocable<F,Args>	Args 인자 목록으로 F 호출
InvocableRegular<F,Args>	호출 가능하고 상등성을 보존
Predicate<F,Args>	호출하여 bool 반환
Relation<F,T,U>	Predicate<F,T,U>
StrictWeakOrder<F,T,U>	Relation이 엄격하고 약한 순서를 제공

출처: 책 270쪽 / 원문 PDF 267쪽

상등성 보존은 x==y일 때 f(x)==f(y)가 되는 성질이다. 비교 연산에는 일반적으로 엄격하고 약한 순서를 가정한다. 원문은 Relation과 StrictWeakOrder의 차이가 의미론적인 것이며, 이 차이를 이름으로 의도에 드러낸다고 설명한다.

출처: 책 270-271쪽 / 원문 PDF 267-268쪽

반복자·구간 컨셉	원문에서의 조건과 관계
Iterator / Sentinel / SizedSentinel	증가·역참조 / 구간 끝의 보초 / 차이 연산이 가능한 보초
InputIterator / OutputIterator	입력의 읽기 / 출력의 쓰기
ForwardIterator / BidirectionalIterator	여러 번 순방향 순회 / 여기에 -- 지원
RandomAccessIterator	양방향 반복자에 원문 표의 산술·인덱싱 연산을 추가
Permutable / Mergeable / Sortable	이동·교환 / 정렬 구간 병합 / 지정 관계로 정렬 가능
Range / SizedRange	시작 반복자와 보초 / 크기를 상수 시간에 알 수 있음
View / BoundedRange	상수 시간 복사·이동·대입 / 반복자와 보초의 타입이 같음
InputRange / OutputRange / ForwardRange / BidirectionalRange / RandomAccessRange	각 이름에 대응하는 반복자 컨셉을 만족하는 구간

출처: 책 271-272쪽 / 원문 PDF 268-269쪽

보초는 주어진 반복자에서 시작해 종료 술어를 만족할 때까지 순회하게 하는 개념이다. 원문은 C 스타일 문자열에서 *p==0을 끝 조건으로 삼는 예를 든다. 반복자의 종류는 알고리즘을 더 적절하게 선택하는 데 활용한다.

12.8 컨테이너 알고리즘

출처: 책 272쪽 / 원문 PDF 269쪽

원문은 Ranges를 기다리지 않고도 컨테이너를 받아 내부에서 begin()/end() 버전을 호출하는 간단한 sort 래퍼를 작성할 수 있다고 설명한다. 기본 비교 버전과 술어 버전을 별도 Estd 네임스페이스에 두면 std 사용 시 충돌을 피하고 나중에 Range로 교체하기도 쉽다.

12.9 병렬 알고리즘

출처: 책 273-274쪽 / 원문 PDF 270-271쪽

각 데이터 항목의 계산이 서로 독립적이면 같은 작업을 병렬로 수행할 수 있다. **병렬 실행**은 여러 스레드에서 작업하고 **벡터화된 실행(SIMD)**은 한 스레드에서 벡터화로 작업한다. <execution>은 seq·par·par_unseq 실행 정책을 제공한다.

정렬의 실행 정책

코드 출처: 책 273쪽 / 원문 PDF 270쪽

C++ 원문

```
sort(v.begin(),v.end()); // 순차
sort(seq,v.begin(),v.end()); // 순차(기본과 동일)
sort(par,v.begin(),v.end()); // 병렬
sort(par_unseq,v.begin(),v.end()); // 병렬 혹은 벡터화
```

코드의 의미

같은 정렬 작업에 순차·병렬·병렬 또는 벡터화 실행 정책을 요청하는 원문 비교다.

원문 실행 결과

[원문에 실행 결과 미제시]

원문 주의 사항

- 병렬 정책은 가능한 경우에 적용하는 힌트다. 효과는 실험으로 확인하며 데이터 경합과 데드락을 피해야 한다.

정책은 실행에 대한 힌트이며 실제 병렬성은 컴파일러와 런타임 스케줄러가 정한다. 효과는 알고리즘·요소 수·하드웨어·사용률에 따라 달라지므로 반드시 실험으로 확인한다. 원문은 12.6절 표의 알고리즘 가운데 equal_range를 병렬 정책 대상에서 제외해 설명한다. 병렬 실행을 요청할 때 데이터 경합과 데드락을 피해야 한다.

12.10 조언

출처: 책 274쪽 / 원문 PDF 271쪽

- 알고리즘의 입력은 반복자로 정의한 반개구간이며, 검색 실패는 일반적으로 입력 끝으로 표시한다.
- 알고리즘은 전달된 시퀀스 자체에 요소를 추가·제거하지 않는다.
- 직접 루프를 만들기 전에 표준 제네릭 알고리즘을 고려한다. 함수 객체와 술어로 동작을 확장하되 술어가 인자를 바꾸지 않게 한다.
- 반복자 쌍 방식이 번거로우면 컨테이너/구간 알고리즘을 사용한다.

중요 내용 요약

- 입력 끝은 마지막 요소 다음이며 검색 실패 표시에 사용한다.
- 출력 알고리즘에 충분한 공간을 제공하거나 back_inserter로 확장한다.
- 알고리즘의 요소 값 변경과 컨테이너 크기 변경은 다르다.
- 원문의 Ranges 명칭과 C++20 전망은 집필 시점의 설명이다.
- 병렬 정책은 실제 병렬 실행이나 성능 향상을 보장하지 않는다.

혼동하기 쉬운 내용

병렬 실행과 벡터화된 실행

출처: 책 273쪽 / 원문 PDF 270쪽

기준	병렬 실행	벡터화된 실행
실행 수단	여러 스레드; 여러 코어일 수도 있음	한 스레드에서 벡터화(SIMD)
정책 요청	par	par_unseq가 병렬 또는 벡터화를 허용
보장 조건	가능한 경우; 정책은 힌트	가능한 경우; 정책은 힌트

장 마무리 복습

- 반복자가 컨테이너 저장 구조와 알고리즘을 연결하는 공통 인터페이스임을 정리한다.
- find와 find_if, 일반 출력 반복자와 back_inserter의 조건을 구분한다.
- 입력·출력·순방향·양방향·임의 접근 반복자의 요구와 Range 계열을 연결한다.
- 술어의 비변경 조건과 병렬 계산의 독립성·경합 방지 조건을 복습한다.

원문 확인 메모

- [원문 확인 필요] 책 270쪽 RandomAccessIterator 표의 일부 산술 기호는 제공 이미지에서 물음표 모양으로 보인다. 해당 기호를 임의로 복원하지 않고 연산 범주만 요약했다.

제13장. 유틸리티

출처: 책 275-287, 290-314쪽 / 원문 PDF 272-309쪽

학습 내용 개요

- RAII·스마트 포인터·이동·포워딩으로 자원과 소유권을 표현한다.
- 비소유 구간·유사 컨테이너·선택지 타입을 구별한다.
- 할당자·시간·함수 어댑션·타입 함수의 역할과 원문 조건을 익힌다.

설명의 흐름

1. 자원 수명과 소유권
2. 이동·포워딩과 구간 참조
3. 특수 컨테이너와 선택지
4. 할당·시간·호출 연결
5. 컴파일 시간의 타입 정보와 선택

13.1 소개

출처: 책 276쪽 / 원문 PDF 273쪽

이 장은 컨테이너·입출력 같은 큰 분류에 딱 들어맞지 않지만 널리 쓰이는 표준 라이브러리 유틸리티를 다룬다. 설계와 프로그램을 표현하는 일반적인 용어로 쓰이는 타입과, 다른 라이브러리 기능을 구현하는 재료가 포함된다. 유틸리티는 지나치게 복잡하거나 다른 기능에 의존하지 않아야 한다.

13.2 자원 관리

출처: 책 276-277쪽 / 원문 PDF 273-274쪽

자원은 획득한 후 언젠가 해제해야 하는 것으로 메모리·소켓·스레드 핸들·파일 핸들이 예다. 누수는 장시간 프로그램의 성능 저하나 충돌뿐 아니라 짧은 프로그램의 자원 부족도 일으킬 수 있다. 표준 라이브러리는 자원을 책임지는 객체의 생성·소멸과 자원의 생애주기를 연결해 누수를 막는다.

RAII는 생성자에서 자원을 획득하고 소멸자에서 해제하는 관례적 관리 방식이다. 원문의 scoped_lock은 생성 시 mutex를 얻고, return·함수 끝·예외 등 어떤 방식으로 스코프를 벗어나도 소멸자로 해제한다. vector·map·string·iostream의 저장 공간이나 파일 핸들도 같은 원리에 따라 관리된다.

13.2.1 unique_ptr과 shared_ptr

출처: 책 277-281쪽 / 원문 PDF 274-278쪽

출처: 책 277-278쪽 / 원문 PDF 274-275쪽

<memory>의 **unique_ptr**은 유일한 소유권을, **shared_ptr**은 공유된 소유권을 표현한다. 자유 저장소의 객체를 스마트 포인터에 맡기면 예외나 이른 반환에서 해제를 빠뜨리는 일을 막을 수 있다. 다만 지역 객체나 컨테이너 같은 더 높은 수준의 자원 핸들로 표현할 수 있다면 그것을 먼저 사용한다.

내장 포인터와 unique_ptr의 해제 비교

코드 출처: 책 278쪽 / 원문 PDF 275쪽

C++ 원문

```
void f(int i, int j) // X* vs. unique_ptr<X>
{
    X* p = new X; // 새로운 X 할당
    unique_ptr<X> sp {new X}; // 새로운 X를 할당하고, 그 포인터를 unique_ptr에 넘겨줌
    // ...

    if (i<99) throw Z{}; // 여기서 예외를 던지거나
    if (j<77) return; // "일찍" 반환될 수 있음
    // ... p와 sp 사용 ..
    delete p; // *p 소멸
}
```

코드의 의미

원문의 조기 종료·예외 사례다. p의 delete까지 도달하지 않는 경로와 달리 sp는 스코프를 떠날 때 자원을 해제한다. 원문에 포함된 생략 주석을 그대로 유지했다.

원문 실행 결과

[원문에 실행 결과 미제시]

원문 주의 사항

- 원문 자체가 X·Z의 정의와 실제 사용 부분을 생략한 예다.

출처: 책 278-279쪽 / 원문 PDF 275-276쪽

unique_ptr은 개별 객체나 배열의 핸들이며 소유권 전달에 이동을 사용한다. shared_ptr은 복사하여 소유권을 나누고 마지막 소유자가 소멸할 때 자원을 해제한다. 함수에 shared_ptr를 넘기면 함수가 종료된 뒤에도 복사본이 남을 수 있으므로, 자원 소멸 시점을 단순히 호출 함수의 끝으로 예상하면 안 된다. 공유에는 추가 비용과 생애주기 예측의 어려움이 있어 꼭 공유해야 할 때만 사용한다.

출처: 책 280-281쪽 / 원문 PDF 277-278쪽

make_unique와 make_shared는 객체 생성과 포인터 인계를 함께 처리해 잘못된 포인터 전달이나 인계 누락을 줄인다. 원문은 make_shared가 소유자 수를 세기 위한 별도 할당을 줄일 수 있다는 이점도 설명한다. 포인터가 필요한 대표적인 경우는 다형적인 객체를 참조하거나 객체를 공유하는 경우다. 객체 컬렉션 반환은 컨테이너로 처리할 수 있다.

출처: 책 280쪽 / 원문 PDF 277쪽

스마트 포인터는 여전히 포인터다. 특히 shared_ptr 자체는 소유자들의 읽기·쓰기 규칙을 명시하지 않으므로, 수명 관리만으로 자원 경합 같은 문제까지 해결되지는 않는다.

13.2.2 move()와 forward()

출처: 책 281-283쪽 / 원문 PDF 278-280쪽

출처: 책 281-282쪽 / 원문 PDF 278-279쪽

이동과 복사의 선택은 대부분 암묵적이지만, 복사할 수 없는 unique_ptr을 다른 소유자로 넘길 때처럼 명시적인 이동이 필요할 수 있다. **std::move() 자체가 자원을 이동시키는 것은 아니다.** 전달한 객체를 rvalue 참조로 변환해 이동 생성자나 이동 대입 연산이 선택되게 유도한다.

이동을 이용한 swap

코드 출처: 책 282쪽 / 원문 PDF 279쪽

C++ 원문

```
template <typename T>
void swap(T& a, T& b)
{
    T tmp {move(a)}; // T의 생성자에는 rvalue로 전달되므로 이동을 수행
    a = move(b); // T의 대입 연산에는 rvalue로 전달되므로 이동을 수행
    b = move(tmp); // T의 대입 연산에는 rvalue로 전달되므로 이동을 수행
}
```

코드의 의미

복사를 계속 수행하지 않고 임시 객체와 두 대입을 이동으로 유도하는 원문 사례다. move 자체의 역할과 실제 이동 연산 선택을 구분한다.

원문 실행 결과

[원문에 실행 결과 미제시]

출처: 책 282-283쪽 / 원문 PDF 279-280쪽

이동 후에도 원본 객체는 남는다. 원문은 move한 문자열을 나중에 다시 인덱싱하는 위험을 들어, 확실하고 큰 성능 개선이 있을 때만 명시적으로 move를 사용하도록 한다. 이동 후 상태는 일반적으로 정해지지 않지만 표준 라이브러리 타입은 소멸·대입 가능한 상태로 남는다고 설명한다. 이 원문은 vector·string 같은 컨테이너를 이동 후 비어 있는 상태로 설명한다.

출처: 책 283쪽 / 원문 PDF 280쪽

완벽한 포워딩은 인자의 성격을 바꾸지 않고 다른 함수로 넘기는 것이다. std::forward()는 lvalue와 rvalue의 차이를 처리한다는 점에서 move와 구별된다. 포워딩할 때는 std::forward를 사용하고 같은 객체를 두 번 포워딩하지 않는다. 포워딩 뒤에 그 객체를 다시 사용하지 않도록 주의한다.

13.3 구간 검사: gsl::span

출처: 책 284-286쪽 / 원문 PDF 281-283쪽

출처: 책 284-285쪽 / 원문 PDF 281-282쪽

포인터와 요소 수를 따로 전달하는 인터페이스는 두 값이 실제 배열과 일치한다는 관례에 의존해 구간 에러를 일으키기 쉽다. **gsl::span**은 요소 시퀀스의 포인터와 길이를 함께 표현하는 구간 참조다. vector나 내장 배열 등의 연속 요소를 가리키며, 그 요소를 소유하지 않는다. string_view나 반복자 쌍과 유사하지만 문자 읽기 전용에 한정되지 않는다.

출처: 책 285-286쪽 / 원문 PDF 282-283쪽

배열에서 span을 직접 만들면 컴파일러가 요소 수를 세므로 간단하고 안전하다. 별도로 포인터·길이로 span을 만들 때는 그 선택이 명시적으로 드러난다. 함수끼리 span을 넘기면 포인터와 개수를 각각 넘기는 것보다 단순하다. 인덱싱의 구간 검사는 에러 시 gsl::fail_fast를 던진다고 원문이 설명하며, 성능이 중요한 코드에서는 검사를 막을 수 있다고 한다.

출처: 책 284-286쪽 / 원문 PDF 281-283쪽

이 절의 span은 원문 시점에 표준으로 제안된 GSL 타입이다. 원문은 향후 std::span의 구간 에러 처리에 계약을 사용하기를 기대한다고 쓴다. 이 전망을 현재의 표준 사실로 바꾸어 보충하지 않는다. 문자 구간에는 gsl::string_span을 제시한다.

13.4 특수한 컨테이너

출처: 책 287쪽 / 원문 PDF 284쪽

유사 컨테이너	원문에서 정리한 역할
T[N] / array<T,N>	연속 저장된 T 요소 N개의 고정 크기 시퀀스; array는 내장 배열의 여러 문제를 줄임
bitset<N> / vector<bool>	N비트의 고정 시퀀스 / vector 특수화로 저장한 비트 시퀀스
pair<T,U> / tuple<T...>	두 요소 / 임의의 여러 타입을 가진 요소 시퀀스
basic_string<C> / valarray<T>	문자열 연산이 있는 문자 시퀀스 / 수치 연산을 제공하는 수치 배열

여러 컨테이너는 서로 다르거나 겹치는 요구를 만족한다. pair와 tuple은 이형이며, 다른 컨테이너는 같은 타입의 요소를 담는 동형이다. array·vector·tuple의 요소는 연속 할당되고 forward_list·map은 연결 구조다.

13.4.1 array

출처: 책 287, 290쪽 / 원문 PDF 284-285쪽

출처: 책 287, 290쪽 / 원문 PDF 284-285쪽

정리 범위 제한: 제공 자료에는 책 288~289쪽이 없다. array 설명의 앞부분을 확인할 수 없으므로 287쪽 표, 290쪽의 이어지는 설명, 313쪽 조언에서 확인되는 내용만 정리한다.

출처: 책 290쪽 / 원문 PDF 285쪽

array는 크기를 알고 있어 표준 알고리즘과 함께 쓸 수 있고 =로 복사할 수 있다. 원문이 강조하는 장점은 내장 배열에서 생길 수 있는 예기치 않은 포인터 변환을 막는다는 것이다. Circle 배열을 Shape*로 받아 인덱싱하면 sizeof(Shape)<sizeof(Circle)이라는 원문의 전제하에 잘못된 오프셋이 생기지만 array는 그 포인터 변환을 허용하지 않는다. 290쪽의 이어지는 문장은 스택이 제한된 자원이며 스택 오버플로우가 생길 수 있다는 주의도 포함한다.

13.4.2 bitset

출처: 책 290-292쪽 / 원문 PDF 285-287쪽

출처: 책 290-291쪽 / 원문 PDF 285-286쪽

bitset<N>은 컴파일 시간에 정한 N비트의 구간에 연산을 제공한다. 참/거짓·켜짐/꺼짐 등의 플래그 집합을 나타내며, 내장 정수에 담기 어려운 비트 집합을 다루는 데 편리하다. 비트에 번호 대신 이름을 붙여야 한다면 원문은 set이나 열거형을 권한다.

출처: 책 291-292쪽 / 원문 PDF 286-287쪽

정수나 문자열로 초기화할 수 있고 비트별 연산과 시프트를 제공한다. 시프트는 0을 밀어 넣는다. to_ullong()과 to_string()은 생성자의 반대 방향으로 변환한다. 출력은 높은 자릿수의 비트가 왼쪽에 오며 bitset 자체의 출력 연산자도 사용할 수 있다.

정수를 bitset 문자열로 출력

코드 출처: 책 291쪽 / 원문 PDF 286쪽

C++ 원문

```
void binary(int i)
{
    bitset<8*sizeof(int)> b = i; // 8비트 바이트를 가정(14.7절 참고)
    cout << b.to_string() << '\n'; // i를 비트로 출력
}
```

코드의 의미

정수의 비트를 가장 높은 자리부터 문자열로 출력한다. 바이트가 8비트라는 원문의 전제를 유지한다.

원문 실행 결과

원문은 인자 123에 대해 00000000000000000000000001111011을 제시한다.

13.4.3 pair와 tuple

출처: 책 292-295쪽 / 원문 PDF 287-290쪽

출처: 책 292-293쪽 / 원문 PDF 287-288쪽

값들의 모음 자체가 필요하면 간단한 struct 또는 pair·tuple을 사용할 수 있다. **pair**는 first와 second라는 일관된 멤버 이름을 가진 두 값의 묶음이다. equal_range는 정렬된 입력에서 같은 값에 해당하는 부분 구간을 두 반복자의 pair로 반환한다. 요소가 연산을 지원하면 pair의 대입·비교도 가능하다.

출처: 책 293-294쪽 / 원문 PDF 288-289쪽

타입 추론이나 make_pair로 중복된 타입 명시를 줄일 수 있다. 둘보다 많거나 적은 요소에는 **tuple**을 사용한다. 원문은 생성자 인자에서의 타입 추론이 C++17부터 제공되며 이전 코드에서는 make_tuple을 사용할 수 있다고 설명한다.

tuple 생성과 타입 추론

코드 출처: 책 294쪽 / 원문 PDF 289쪽

C++ 원문

```
tuple<string,int,double> t1 {"Shark",123,3.14}; // 타입을 명시적으로 지정
auto t2 = make_tuple(string{"Herring"},10,1.23); // 타입을
    // tuple<string,int,double>로 추론
tuple t3 {"Cod"s,20,9.99}; // 타입을 tuple<string,int,double>로 추론
```

코드의 의미

명시한 타입, make_tuple, 생성자 인자에서 타입을 추론하는 세 방식을 비교하는 원문 선언이다.

원문 실행 결과

[원문에 실행 결과 미제시]

출처: 책 294-295쪽 / 원문 PDF 289-290쪽

get<0>()처럼 0부터 시작하는 상수 인덱스로 tuple 요소에 접근한다. 그 타입이 tuple 안에서 유일할 때는 get<string>()처럼 타입으로도 지정할 수 있고 쓰기에도 get을 사용한다. pair도 get으로 접근 가능하고 pair·tuple 모두 구조화된 바인딩을 사용할 수 있다. 제네릭할 필요가 없다면 이름 있는 멤버를 둔 struct가 더 읽기 쉽고 유지보수하기 좋을 수 있다.

13.5 선택지

출처: 책 295쪽 / 원문 PDF 290쪽

표준 라이브러리는 지정한 선택지 중 하나를 담는 variant, 특정 타입의 값 또는 값 없음을 표현하는 optional, 타입 집합에 제한이 없는 any를 제공한다. 각각 <variant>·<optional>·<any>에 정의되며, 관련 기능을 표현하지만 인터페이스가 통일되어 있지는 않다.

13.5.1 variant

출처: 책 295-298쪽 / 원문 PDF 290-293쪽

출처: 책 295-296쪽 / 원문 PDF 290-291쪽

variant<A,B,C>는 union을 직접 사용하는 것보다 안전하고 편리한 선택지다. 대입·초기화한 값의 타입을 기억하며 사용자는 어떤 선택지가 저장되었는지 질의하고 값을 추출한다. 원문은 문자열 또는 에러 코드를 반환하는 경우와, 컴파일러 노드의 Expression·Statement·Declaration·Type을 구분하는 경우를 든다.

출처: 책 297-298쪽 / 원문 PDF 292-293쪽

저장 타입별로 여러 번 검사하는 대신 visit과 오버로드된 함수 집합으로 행동을 선택할 수 있다. 원문의 overloaded는 람다 등으로부터 오버로드 집합을 만드는 보조 타입이며 표준 기능 자체는 아니라고 명시한다. 추론 가이드도 이 구현에 사용된다. visit 이후 오버로드 규칙으로 가장 적절한 람다가 호출된다. 잘못된 타입으로 접근하면 bad_variant_access를 던진다. 가상 호출보다 빠를 가능성은 실측해야 한다.

13.5.2 optional

출처: 책 298-299쪽 / 원문 PDF 293-294쪽

optional<A>는 A 또는 아무것도 없음을 나타낸다. 객체를 반환하거나 반환하지 않는 함수에 유용하며 빈 {}가 값 없음에 해당한다. 조건 검사로 값 유무를 살피고 *로 저장 값에 접근하므로 포인터와 비슷한 사용 형태를 가진다.

값이 있는 optional만 합산

코드 출처: 책 299쪽 / 원문 PDF 294쪽

C++ 원문

```
int cat(optional<int> a, optional<int> b)
{
    int res = 0;
    if (a) res+=*a;
    if (b) res+=*b;
    return res;
}

int x = cat(17,19);
int y = cat(17,{});
int z = cat({},{});
```

코드의 의미

조건으로 존재를 검사한 뒤 *a·*b를 사용한다. {}는 빈 optional을 나타낸다. 원문에 각 호출의 출력은 별도로 제시되지 않는다.

원문 실행 결과

[원문에 실행 결과 미제시]

원문 주의 사항

- 값이 없는 optional을 바로 역참조하지 않는다.

출처: 책 299쪽 / 원문 PDF 294쪽

원문은 값이 없는 optional에 접근하면 동작을 예측할 수 없고 예외도 던지지 않으므로, 이 접근 방식에서 타입 안전성을 보장하지 않는다고 경고한다. 따라서 예제처럼 먼저 존재 여부를 검사하는 조건을 유지한다.

13.5.3 any

출처: 책 299-300쪽 / 원문 PDF 294-295쪽

any는 임의의 타입을 저장하며 저장된 객체가 있으면 그 타입을 기억한다. 제약을 제거한 variant처럼 설명할 수 있다. any_cast로 타입을 지정하여 값을 추출한다. 원문은 저장된 타입과 다른 타입으로 접근하면 bad_any_access를 던진다고 표기하고, 예외에 의존하지 않는 접근 방법도 있다고 언급한다.

출처: 책 300쪽 / 원문 PDF 295쪽

[원문 확인 필요] 여기서 예외 이름 bad_any_access는 제공된 책 300쪽의 표기다. 이 노트에서는 의심되는 명칭을 다른 이름으로 조용히 바꾸지 않았다.

13.6 할당자

출처: 책 300-302쪽 / 원문 PDF 295-297쪽

출처: 책 300쪽 / 원문 PDF 295쪽

컨테이너의 기본 할당은 new·delete를 사용하지만, 특별한 성능·보안·스레드별 할당·메모리 구조 요구가 있으면 **할당자**를 지정할 수 있다. 원문은 고급 할당자 전반 대신 이벤트 큐에서 풀 할당자로 단편화를 줄인 실제 사례를 제시한다.

출처: 책 301-302쪽 / 원문 PDF 296-297쪽

원래 시스템은 vector를 가진 이벤트를 shared_ptr로 전달하여 마지막 사용자가 자동 소멸시키는 단순한 구조였지만 단편화로 많은 메모리를 소비했다. 풀 할당자는 같은 크기의 여러 객체를 한꺼번에 할당해 관리한다. C++17의 pmr 하위 네임스페이스에는 다형성 메모리 자원과 synchronized_pool_resource가 있어 컨테이너를 전면 교체하지 않고 할당자를 바꿀 수 있다.

출처: 책 301-302쪽 / 원문 PDF 296-297쪽

원문 사례에서 프로듀서 16개와 컨슈머 4개가 이벤트 100,000개를 전달한 뒤 사용 메모리가 6GB에서 3MB 미만으로 줄었다. 이는 단편화 낭비를 줄인 결과이며 실제로 필요했던 메모리양이 바뀐 것은 아니다. 이 수치는 해당 사례의 관측값으로만 이해한다.

13.7 시간

출처: 책 302-303쪽 / 원문 PDF 297-298쪽

<chrono>의 clock::now()는 시각인 time_point를 반환하고 두 시각의 차이는 시간 구간인 duration이 된다. 시계별로 단위가 다르므로 duration_cast로 적절한 단위에 맞춰 표현한다. 성능을 판단할 때는 추측 대신 실제 시간을 측정한다.

chrono로 작업 시간 측정

코드 출처: 책 302쪽 / 원문 PDF 297쪽

C++ 원문

```
using namespace std::chrono; // 하위 네임스페이스 std::chrono; 3.4절 참고

auto t0 = high_resolution_clock::now();
do_work();
auto t1 = high_resolution_clock::now();
cout << duration_cast<milliseconds>(t1-t0).count() << "msec\n";
```

코드의 의미

작업 전후의 time_point 차이를 milliseconds로 변환한 뒤 count()를 출력한다. 실제 소요 시간은 원문에서 특정하지 않는다.

원문 실행 결과

[원문에 실행 결과 미제시]

출처: 책 303쪽 / 원문 PDF 298쪽

시간 단위 접미사는 std::chrono_literals에 정의되어 10ms·33us 같은 표현을 가능하게 한다. 원문은 년·월·달력·시간대·윤초를 다루는 확장 기능을 C++20 표준에 추가 중이라고 소개한다. 이는 원문 시점의 상태이며 현재 상황으로 보충하지 않는다.

13.8 함수 어댑션

출처: 책 303-304쪽 / 원문 PDF 298-299쪽

함수를 다른 함수의 인자로 넘길 때 호출 형태나 타입이 정확히 맞지 않으면 람다, mem_fn, function을 이용하는 세 방법을 고려한다.

13.8.1 람다를 어댑터로 사용

출처: 책 304쪽 / 원문 PDF 299쪽

표준 알고리즘은 f(x) 형태로 인자를 호출하지만 멤버 함수는 x->f() 형태를 사용한다. 람다는 이 호출 방식의 차이를 연결할 수 있다.

람다로 멤버 함수 호출 연결

코드 출처: 책 304쪽 / 원문 PDF 299쪽

C++ 원문

```
void draw_all(vector<Shape*>& v)
{
    for_each(v.begin(),v.end(),[](Shape* p) { p->draw(); });
}
```

코드의 의미

for_each의 함수 호출 형식을 각 Shape 포인터의 draw() 멤버 호출로 연결한다.

원문 실행 결과

[원문에 실행 결과 미제시]

13.8.2 mem_fn()

출처: 책 304-305쪽 / 원문 PDF 299-300쪽

mem_fn(mf)는 멤버 함수 mf를 비멤버 함수처럼 호출할 수 있는 함수 객체로 바꾼다. 원문은 mem_fn(&Shape::draw)를 for_each에 넘겨 람다와 같은 연결을 수행한다. 람다 도입 이전에 객체지향 호출과 함수 스타일 호출을 연결하던 대표 수단이다.

13.8.3 function()

출처: 책 305쪽 / 원문 PDF 300쪽

function 객체는 ()로 호출할 수 있는 대상을 저장한다. 함수·람다를 넣고 그 호출 타입을 명시하거나 초기 값에서 추론할 수 있다. 콜백이나 연산을 인자로 전달하는 데 유용하지만 직접 함수 호출보다 실행 비용이 있고, function은 객체이므로 자체에 오버로딩을 적용할 수 없다. 함수 객체의 오버로딩에는 앞선 overloaded를 고려한다.

13.9 타입 함수

출처: 책 306쪽 / 원문 PDF 301쪽

타입 함수는 타입 자체를 인자로 받거나 타입을 반환하며 컴파일 시간에 평가된다. numeric_limits는 수치 타입 정보를, sizeof는 객체의 바이트 수를 알려준다. 이런 기능은 컴파일 시간 계산을 통한 타입 검사와 성능 향상의 재료다. 관련 기법을 메타프로그래밍, 템플릿과 관련되면 템플릿 메타프로그래밍이라고 한다. 원문은 아직 컨셉이 범용 표준이 아닌 시점이어서 타입 함수가 널리 쓰인다고 설명한다.

13.9.1 iterator_traits

출처: 책 306-310쪽 / 원문 PDF 301-305쪽

출처: 책 306-309쪽 / 원문 PDF 301-304쪽

표준 sort는 임의 접근 반복자를 요구하지만 forward_list는 순방향 반복자만 제공한다. 원문은 임의 접근 구간에는 바로 sort를, 순방향 구간에는 vector로 복사·정렬·원래 구간으로 재복사를 선택하는 sort_helper 두 버전을 제시한다. 반복자 종류를 나타내는 태그로 적절한 오버로드를 컴파일 시간에 선택하는 기법이 **태그 디스패치**다.

포인터용 iterator_traits

코드 출처: 책 309쪽 / 원문 PDF 304쪽

원문의 iterator_traits<T*> 특수화 블록 전체를 발췌했습니다. 앞뒤 sort_helper와 타입 별칭 코드는 생략했습니다.

C++ 원문

```
template<class T>
struct iterator_traits<T*> {
    using difference_type = ptrdiff_t;
    using value_type = T;
    using pointer = T*;
    using reference = T&;
    using iterator_category = random_access_iterator_tag;
};
```

코드의 의미

멤버 타입이 없는 포인터에 반복자 관련 타입 정보를 제공하는 원문 특수화다. 포인터의 종류를 임의 접근으로 분류하는 정보가 태그 디스패치에 쓰인다.

원문 실행 결과

[원문에 실행 결과 미제시]

출처: 책 308-310쪽 / 원문 PDF 303-305쪽

컨테이너의 iterator·value_type 멤버는 타입 별칭으로 표현할 수 있다. 포인터처럼 이런 멤버 타입이 없는 경우에는 <iterator>의 iterator_traits 특수화를 통해 값 타입·참조·포인터·반복자 종류를 얻는다. 따라서 int*도 random_access_iterator_tag를 가진 반복자로 다룰 수 있다. 원문은 같은 구분이 컨셉으로 더 직접 표현될 수 있고 표준화가 진행 중이라고 설명한다.

13.9.2 타입 술어

출처: 책 310-311쪽 / 원문 PDF 305-306쪽

<type_traits>는 is_arithmetic·is_class·is_pod·is_literal_type·has_virtual_destructor·is_base_of 같은 **타입 술어**를 제공한다. 템플릿은 이 값으로 타입의 성질을 검사할 수 있다. 원문은 산술 타입만 받는 complex의 static_assert를 예로 들고, std::is_arithmetic<T>::value를 반환하는 Is_arithmetic() 래퍼로 표기를 간단히 한다.

타입 술어의 표기를 단순화한 래퍼

코드 출처: 책 311쪽 / 원문 PDF 306쪽

C++ 원문

```
template<typename T>
constexpr bool Is_arithmetic()
{
    return std::is_arithmetic<T>::value;
}
```

코드의 의미

표준 타입 특성의 value를 함수 호출 표기로 감싼 원문 래퍼다. 같은 절의 complex에서 static_assert 조건으로 활용한다.

원문 실행 결과

[원문에 실행 결과 미제시]

13.9.3 enable_if

출처: 책 311-312쪽 / 원문 PDF 306-307쪽

타입 술어는 static_assert·컴파일 시간 if·enable_if의 조건으로 사용된다. **enable_if**는 조건에 따라 정의를 선택하는 데 널리 쓰인다. 원문의 스마트 포인터 설명은 T가 클래스일 때에만 화살표 연산을 제공하려는 의도를 든다. 함수 바깥에서는 컴파일 시간 if를 쓸 수 없다는 문맥에서 enable_if를 소개한다.

출처: 책 312쪽 / 원문 PDF 307쪽

enable_if는 표기가 낯설고 많은 경우 컨셉으로 대체될 수 있지만, 원문 시점의 템플릿 메타프로그래밍과 여러 표준 라이브러리 구성 요소의 기반이다. 이 기법은 **SFINAE(Substitution Failure Is Not An Error)** 언어 기능에 의존한다.

13.10 조언

출처: 책 312-314쪽 / 원문 PDF 307-309쪽

- 자원에는 RAII 자원 핸들을 사용한다. 포인터가 필요하면 unique_ptr를 우선하고, 소유권을 공유해야 할 때만 shared_ptr를 쓴다. 생성에는 make_unique·make_shared를 사용한다.
- move를 남용하지 않으며 forward는 포워딩에만 사용한다. 이동·포워딩한 객체를 나중에 읽지 않는다.

- 포인터와 개수를 별도로 전달하기보다 span을 사용한다. 고정 크기에는 array, N비트에는 bitset을 고려한다.
- pair·tuple보다 이름 있는 struct가 읽기 쉬운 경우를 구별한다. union을 직접 쓰기보다 variant를 고려한다.
- 단편화에는 할당자를 고려하고, 효율성은 실제 시간 측정으로 판단한다. duration은 단위를 명시하고 필요하면 duration_cast로 바꾼다.
- 호출을 연결할 때 람다·mem_fn을, 호출 대상을 저장할 때 function을 쓴다. 타입 특성·enable_if·컨셉과 타입 별칭의 역할을 구분한다.

중요 내용 요약

- RAII의 획득·해제 쌍은 예외와 이른 반환에도 연결된다.
- shared_ptr는 동시 접근 규칙까지 보장하는 수단이 아니다.
- move는 rvalue 변환이며 이동·포워딩 뒤 원본 재사용을 주의한다.
- span은 요소를 소유하지 않는 구간 참조다.
- variant·optional·any는 선택지 범위와 접근 방식이 다르다.
- 타입 특성과 태그는 컴파일 시간에 적절한 구현을 선택하는 재료다.

혼동하기 쉬운 내용

스마트 포인터의 소유권

출처: 책 277-281쪽 / 원문 PDF 274-278쪽

기준	unique_ptr	shared_ptr
소유권	유일한 소유자	여러 소유자가 공유
전달	이동; 복사할 수 없음	복사하여 공유
자원 해제	소유 핸들의 수명에 연결	마지막 소유자가 소멸할 때
선택	포인터가 필요하고 소유자가 하나일 때 우선	공유 소유권이 실제로 필요할 때

세 가지 선택지 타입

출처: 책 295-300쪽 / 원문 PDF 290-295쪽

타입	표현하는 선택지	원문의 접근 방식
variant	미리 지정한 여러 타입 중 하나	타입 질의·get·visit
optional	한 타입의 값 또는 없음	존재 검사 후 *
any	미리 제한하지 않은 타입	any_cast로 지정 타입 추출

장 마무리 복습

- 지역 객체·컨테이너·unique_ptr·shared_ptr 가운데 필요한 자원 관리 수준과 소유권을 구분한다.
- 구간의 길이와 포인터를 함께 다루는 span, 고정 크기 array, 비트 집합 bitset의 목적을 복습한다.
- pair·tuple의 위치·타입 접근과 이름 있는 struct의 가독성 차이를 정리한다.
- 할당자의 단편화 개선 사례와 chrono의 실제 성능 측정을 연결한다.

- 람다·mem_fn·function의 호출 연결과 iterator_traits·타입 술어·enable_if의 컴파일 시간 역할을 구분한다.

원문 확인 메모

- 책 288~289쪽이 제공되지 않아 13.4.1 array의 앞부분은 확인할 수 없다. 확인된 앞뒤 페이지·표·조언만 정리했다.
- [원문 확인 필요] 책 300쪽의 any 예외 이름 bad_any_access를 원문대로 보존했다. 의심되는 표기를 외부 지식으로 수정하지 않았다.

제14장. 수치

출처: 책 315-325쪽 / 원문 PDF 310-320쪽

학습 내용 개요

- 표준 수학 함수와 에러 표현, 시퀀스 수치 알고리즘을 다룬다.
- 복소수·난수·수치 배열·수치 타입 속성을 연결한다.

설명의 흐름

1. 수학 함수와 errno
2. 수치 누적 및 병렬 알고리즘
3. 복소수와 엔진·분포
4. valarray와 numeric_limits

14.1 소개

출처: 책 316쪽 / 원문 PDF 311쪽

수치 계산은 데이터베이스·네트워킹·장비 제어·그래픽스·시뮬레이션·금융 분석 등 큰 시스템의 일부로 쓰인다. C++는 계산 과정에 복잡한 자료 구조가 함께 필요할 때 장점을 가진다. 이 장은 그런 작업을 지원하는 표준 라이브러리 수치 기능을 정리한다.

14.2 수학 함수

출처: 책 316-317쪽 / 원문 PDF 311-312쪽

<cmath>는 float·double·long double에 대한 표준 수학 함수를 제공한다. 대응하는 복소수 버전은 <complex>에서 찾을 수 있고 반환 타입은 인자 타입과 같다.

출처: 책 316-317쪽 / 원문 PDF 311-312쪽

함수	원문에서 설명한 역할·조건
abs / ceil / floor	절댓값 / x 이상인 가장 작은 정수 / x 이하인 가장 큰 정수
sqrt	제곱근; x는 음수가 아니어야 함
cos / sin / tan	코사인 / 사인 / 탄젠트
acos / asin / atan	역코사인 / 역사인 / 역탄젠트; acos는 음이 아닌 결과, asin은 0에서 가장 가까운 결과
sinh / cosh / tanh	하이퍼볼릭 사인 / 코사인 / 탄젠트
exp / log / log10	밑 e의 지수 / 자연 로그 / 밑 10의 로그; log의 x는 양수

출처: 책 317쪽 / 원문 PDF 312쪽

원문은 <cerrno>의 errno로 에러를 나타내며 EDOM은 정의역 에러, ERANGE는 구간 에러라고 설명한다. 예제는 먼저 errno를 0으로 지운 뒤 음수의 sqrt와 너무 큰 pow 결과를 검사한다. <cstdlib>에도 몇 가지 수학 함수가 있고, <cmath>에는 beta·riemann_zeta·sph_bessel 같은 특수 수학 함수도 있다.

14.3 수치 알고리즘

출처: 책 318-319쪽 / 원문 PDF 313-314쪽

<numeric>은 시퀀스의 합·내적·부분합·이웃한 차이 등을 일반화한다. 각 알고리즘에는 흔히 쓰는 기본 연산 버전과, 적용 연산을 인자로 받는 일반화 버전이 있다. 모든 표준 라이브러리 시퀀스에 적용할 수 있다.

출처: 책 318쪽 / 원문 PDF 313쪽

알고리즘	원문에서 정리한 역할
accumulate(b,e,i) / accumulate(b,e,i,f)	초기 값 i를 바탕으로 합산 / + 대신 f 사용
inner_product	두 시퀀스의 대응 요소 곱을 합산하고 초기 값에 더함; +·*를 별도 연산으로 바꾸는 버전 제공
partial_sum	각 위치까지의 부분합; + 대신 f를 지정할 수 있음
adjacent_difference	이웃한 요소의 차이; 비어 있지 않으면 첫 출력은 첫 입력 값; - 대신 f 지정 가능
iota	연속적인 값으로 구간을 채우는 기능; 원문 표는 각 요소에 ++v를 대입한다고 표기
gcd / lcm	정수의 최대공약수 / 최소공배수

accumulate의 초기 값과 합산

코드 출처: 책 318쪽 / 원문 PDF 313쪽

C++ 원문

```
list<double> lst {1, 2, 3, 4, 5, 9999.99999};
auto s = accumulate(lst.begin(),lst.end(),0.0); // 합 계산: 10014.9999
```

코드의 의미

초기 값 0.0을 바탕으로 list의 구간을 합산한다. 원문 주석의 자릿수는 변경하지 않았다.

원문 실행 결과

원문 코드 주석은 합을 10014.9999로 표기한다.

원문 주의 사항

- [원문 확인 필요] 주석의 자릿수 표기를 그대로 보존했으며 별도 계산 결과로 교체하지 않았다.

출처: 책 318쪽 / 원문 PDF 313쪽

[원문 확인 필요] 318쪽 iota 표는 ++v와 v+1, v+2, ...로 설명한다. 이 노트는 의심되는 시작 값 표기를 임의로 고치거나 다른 실행 결과를 만들지 않았다.

14.3.1 병렬 알고리즘

출처: 책 319-320쪽 / 원문 PDF 314-315쪽

<numeric>의 병렬 수치 알고리즘은 기존 함수와 비슷하지만 일부 의미가 다르다. **reduce**는 accumulate와 달리 요소 연산의 순서가 정해지지 않는다. 초기 값과 실행 정책을 주는 형태, 기본 값을 사용하는 형태가 제공된다.

출처: 책 319쪽 / 원문 PDF 314쪽

알고리즘	차이와 역할
reduce	순서가 정해지지 않는 누적
exclusive_scan / inclusive_scan	i번째 합에서 i번째 입력 제외 / 포함
transform_reduce	각 요소에 f를 적용한 뒤 reduce
transform_exclusive_scan / transform_inclusive_scan	각 요소에 f를 적용한 뒤 각각의 scan 실행

reduce와 병렬 실행 정책

코드 출처: 책 320쪽 / 원문 PDF 315쪽

C++ 원문

```
vector<double> v {1, 2, 3, 4, 5, 9999.99999};
auto s = reduce(v.begin(),v.end()); // double을 누적자로 사용해 합을 계산

vector<double> large;
// ... 많은 값들로 large를 채운다 ...
auto s2 = reduce(par_unseq, large.begin(),large.end()); // 가능한 병렬화 정도로 합을 계산
```

코드의 의미

기본 누적과 par_unseq를 요청한 누적의 원문 예제다. large의 실제 데이터는 원문부터 생략되어 있다.

원문 실행 결과

[원문에 실행 결과 미제시]

원문 주의 사항

- reduce는 accumulate와 달리 요소에 연산을 적용하는 순서가 정해져 있지 않다.

출처: 책 319-320쪽 / 원문 PDF 314-315쪽

일반 병렬 알고리즘과 마찬가지로 실행 정책을 지정할 수 있다. 기본 연산 대신 별도의 연산을 받거나 기본 정책·기본 값을 쓰는 형태도 있다고 원문은 설명하며, 표에서는 편의상 일부를 생략했다.

14.4 복소수

출처: 책 320-321쪽 / 원문 PDF 315-316쪽

<complex>의 **complex<Scalar>**는 실수부와 허수부 타입을 매개변수로 받는 복소수 템플릿이다. 원문은 float·double·long double에 대한 복소수 사용을 보여 준다. 일반적인 산술 연산과 sqrt·pow 같은 수학 함수를 사용할 수 있고 생성자의 실수부·허수부에는 기본 인자가 제공된다.

14.5 난수

출처: 책 321-323쪽 / 원문 PDF 316-318쪽

출처: 책 321쪽 / 원문 PDF 316쪽

<random>의 난수 생성기는 **엔진**과 **분포**를 결합한다. 엔진은 난수 또는 의사 난수 시퀀스를 만들고, 분포는 그 값을 수학적 분포의 특정 구간으로 매핑한다. 원문은 같은 확률로 정수를 만드는 uniform_int_distribution, 종형 곡선의 normal_distribution, exponential_distribution을 예로 든다.

출처: 책 321-322쪽 / 원문 PDF 316-317쪽

엔진과 분포를 직접 조합한 주사위 예제에 이어 원문은 두 구성 요소를 Rand_int 클래스에 묶어 간단한 사용 인터페이스를 제시한다. 이 클래스는 기본 난수 엔진, 정수 분포, 호출 연산자, 시드 설정 연산을 가진다.

엔진과 분포를 묶은 Rand_int

코드 출처: 책 322쪽 / 원문 PDF 317쪽

C++ 원문

```
class Rand_int {
public:
    Rand_int(int low, int high) :dist{low,high} { }
    int operator()() { return dist(re); } // int 생성
    void seed(int s) { re.seed(s); } // 난수 엔진에 새로운 시드 선택
private:
    default_random_engine re;
    uniform_int_distribution<> dist;
};
```

코드의 의미

생성자에서 정수 분포의 구간을 정하고, 호출할 때 dist가 엔진 re를 사용해 int를 생성한다. seed는 엔진의 시드를 바꾼다.

원문 실행 결과

[원문에 실행 결과 미제시]

출처: 책 322-323쪽 / 원문 PDF 317-318쪽

원문의 히스토그램 예는 0부터 9까지의 값을 200번 생성하고 각 값의 빈도만큼 별표를 출력한다. 결과는 통계적 변동이 있는 분포로 제시되므로 이 노트에서 새로운 난수값이나 고정 출력표를 만들지 않는다. 조언에서는 충분한 무작위성을 확인하고 C의 rand()를 사용하지 않도록 권한다.

14.6 벡터 산술

출처: 책 323-324쪽 / 원문 PDF 318-319쪽

일반 컨테이너 **vector**는 값을 저장하고 반복자·알고리즘 구조와 결합하는 데 초점을 두며 수학적인 벡터 연산을 직접 제공하지 않는다. <valarray>의 **valarray**는 일반성이 다소 낮은 대신 수치 계산 최적화에 더 적합하도록 제공되는 배열 템플릿이다.

valarray의 배열 수준 산술

코드 출처: 책 324쪽 / 원문 PDF 319쪽

C++ 원문

```
void f(valarray<double>& a1, valarray<double>& a2)
{
    valarray<double> a = a1*3.14+a2/a1; // 수치 배열 연산자 *, +, /, =
    a2 += a1*3.14;
    a = abs(a);
    double d = a2[7];
    // ...
}
```

코드의 의미

배열 수준 연산과 수학 함수 적용, 인덱스 접근을 함께 보여 주는 원문 함수다. 원문의 생략 주석을 유지했다.

원문 실행 결과

[원문에 실행 결과 미제시]

출처: 책 324쪽 / 원문 PDF 319쪽

valarray는 배열 수준의 산술 연산과 일반적인 수학 함수, 요소 인덱싱을 제공한다. 다차원 계산을 구현하기 위한 건너뛰기 접근(stride access)도 지원한다. 원문은 연산·요소 타입의 유연성보다 실행 효율이 중요할 때의 선택으로 설명한다.

14.7 수치 한계

출처: 책 324쪽 / 원문 PDF 319쪽

<limits>의 **numeric_limits**는 내장 수치 타입의 속성을 기술한다. 원문은 char의 부호 유무와 int가 필요한 큰 값을 표현할 수 있는지 static_assert로 검사한다. max()가 constexpr 함수이므로 컴파일 시간 조건으로 사용할 수 있다.

수치 타입의 성질을 컴파일 시간에 검사

코드 출처: 책 324쪽 / 원문 PDF 319쪽

C++ 원문

```
static_assert(numeric_limits<char>::is_signed,"부호 없는 문자!");
static_assert(100000<numeric_limits<int>::max(),"작은 int!");
```

코드의 의미

char의 부호 유무와 int가 100000보다 큰 최대 값을 제공하는지 원문의 static_assert 조건으로 확인한다. max()는 constexpr 함수다.

원문 실행 결과

[원문에 실행 결과 미제시]

14.8 조언

출처: 책 325쪽 / 원문 PDF 320쪽

- 수치 문제를 수학적으로 확신할 수 없다면 전문적인 조언과 실험을 활용한다. 기본 언어 기능만으로 직접 만들기보다 라이브러리를 사용한다.
- 시퀀스를 계산하는 루프 전에 accumulate·inner_product·partial_sum·adjacent_difference를 고려한다.
- 복소수에는 std::complex를 사용한다. 난수 생성은 엔진과 분포를 결합하고 무작위성에 주의를 기울인다.
- 수치 효율에 우선순위가 있으면 valarray를 고려한다. numeric_limits로 수치 타입이 용도에 맞는지 검사한다.

중요 내용 요약

- 수학 함수의 정의역·구간 에러 조건을 유지한다.
- 수치 알고리즘은 기본 연산을 일반화하고 다른 연산도 인자로 받는다.
- reduce의 연산 순서는 accumulate와 구별한다.
- 난수 생성기는 엔진과 분포의 결합이다.
- numeric_limits와 constexpr로 타입의 용도 적합성을 검사할 수 있다.

혼동하기 쉬운 내용

vector와 valarray

출처: 책 323-325쪽 / 원문 PDF 318-320쪽

기준	vector	valarray
목적	일반적인 값 저장·컨테이너 구조	수치 계산 최적화
일반성	유연한 일반 컨테이너	일반성은 상대적으로 낮음
수치 배열 연산	수학 벡터 연산을 직접 제공하지 않음	배열 산술·수학 함수·stride access

accumulate와 reduce

출처: 책 318-320쪽 / 원문 PDF 313-315쪽

기준	accumulate	reduce
구분	순차 수치 알고리즘	병렬 수치 알고리즘
적용 순서	순차 처리의 누적	요소 연산 순서가 정해지지 않음
정책	원문의 순차 예	실행 정책 지정 가능

장 마무리 복습

- 합·내적·부분합·인접 차이의 목적과 연산 매개변수를 구분한다.
- 엔진은 값의 시퀀스를 만들고 분포는 필요한 범위와 분포로 매핑한다.
- 일반 저장 vector와 수치 계산 valarray의 선택 기준을 복습한다.
- 수치 타입의 표현 가능성은 numeric_limits로 확인한다.

원문 확인 메모

- [원문 확인 필요] 318쪽 iota 표의 ++v 및 시작 값, accumulate 예제의 합 주석 자릿수는 원문 표기를 유지했다.

제15장. 동시성

출처: 책 327-345쪽 / 원문 PDF 321-339쪽

학습 내용 개요

- 태스크·스레드와 인자·결과 전달을 다룬다.
- 공유 데이터의 경합·락·데드락·이벤트 대기 조건을 이해한다.
- future·promise·packaged_task·async로 태스크 통신을 표현한다.

설명의 흐름

1. 시스템 수준 지원과 태스크 분리
2. thread 시작·인자·결과
3. RAII 락과 읽기/쓰기 동기화
4. 조건 변수 대기
5. 값·예외의 태스크 통신

15.1 소개

출처: 책 327-328쪽 / 원문 PDF 321-322쪽

동시성은 여러 태스크를 동시에 실행하는 것으로, 여러 프로세서로 처리량을 늘리거나 대기 중에도 다른 일을 진행하여 반응성을 높이는 데 사용한다. 원문의 표준 라이브러리는 정교한 고수준 모델보다는 시스템 수준 동시성을 이식성과 타입 안전성을 고려해 제공한다.

C++는 한 주소 공간의 여러 스레드를 지원하며 메모리 모델과 원자적 연산을 제공한다. **데이터 경합**은 변경 가능한 데이터에 대한 무질서한 동시 접근이다. 원자적 연산은 락 없는 프로그래밍의 기반이지만 원문은 이를

전문가에게 맡기도록 조언한다. 동시성 자체가 해답은 아니며, 순차 해결이 더 간단하고 빠를 수 있다. 직접 스레드를 관리하는 대신 병렬 알고리즘을 이용하는 방법도 있다.

15.2 태스크와 thread

출처: 책 328-330쪽 / 원문 PDF 322-324쪽

출처: 책 328-329쪽 / 원문 PDF 322-323쪽

태스크는 다른 계산과 동시에 실행할 수 있는 작업이며, **thread**는 이를 표현하는 시스템 수준 개념이다. <thread>의 std::thread에 함수 또는 함수 객체를 넘겨 실행을 시작한다. join()은 해당 스레드의 종료를 기다린다.

함수와 함수 객체를 스레드에서 실행

코드 출처: 책 329쪽 / 원문 PDF 323쪽

C++ 원문

```
void f(); // 함수

struct F { // 함수 객체
    void operator()(); // F의 호출 연산자(6.3.2절)
};

void user()
{
    thread t1 {f}; // 별도 스레드에서 f() 실행
    thread t2 {F()}; // 별도 스레드에서 F()() 실행

    t1.join(); // t1을 기다림
    t2.join(); // t2를 기다림
}
```

코드의 의미

함수 f와 함수 객체 F를 각각 thread로 시작하고 두 join으로 종료를 기다리는 원문 예다.

원문 실행 결과

[원문에 실행 결과 미제시]

출처: 책 329-330쪽 / 원문 PDF 323-324쪽

한 프로그램의 스레드는 주소 공간을 공유하므로 객체를 통한 통신이 가능하지만 락 등으로 접근을 제어해야 한다. 원문은 두 태스크가 동기화 없이 cout을 쓰면 실행 순서가 정해지지 않아 출력이 뒤섞일 수 있음을 보여 준다. 명확한 통신 외에는 태스크를 분리하고, 인자를 받아 결과를 반환하는 함수처럼 다루는 것이 가장 단순한 접근이다.

15.3 인자 전달

출처: 책 330-331쪽 / 원문 PDF 324-325쪽

태스크에는 값·포인터·참조로 데이터를 전달할 수 있다. thread의 가변 템플릿 생성자는 실행할 함수와 그 인자들을 받는다. <functional>의 ref()를 쓰면 객체를 값으로 전달하는 대신 참조로 취급한다. 함수 객체는 생성 시 받은 참조를 멤버에 저장할 수도 있다.

참조 인자를 전달하는 두 방식

코드 출처: 책 330-331쪽 / 원문 PDF 324-325쪽

C++ 원문

```
void f(vector<double>& v); // v로 무언가를 수행하는 함수

struct F { // 함수 객체: v로 무언가를 수행
    vector<double>& v;
    F(vector<double>& vv) :v{vv} { }
    void operator()(); // 적용 연산자; 6.3.2절
};

int main()
{
    vector<double> some_vec {1,2,3,4,5,6,7,8,9};
    vector<double> vec2 {10,11,12,13,14};

    thread t1 {f,ref(some_vec)}; // f(some_vec)을 별도의 스레드에서 실행
    thread t2 {F{vec2}}; // F(vec2)()를 별도의 스레드에서 실행

    t1.join();
    t2.join();
}
```

코드의 의미

ref(some_vec)는 참조 전달을 명시하고 F{vec2}는 함수 객체 멤버에 참조를 저장한다.

원문 실행 결과

[원문에 실행 결과 미제시]

원문 주의 사항

- F가 vec2를 사용하는 동안 다른 태스크에서 vec2에 접근하지 않아야 한다.

원문의 F{vec2}가 vec2를 참조하는 동안 다른 태스크가 vec2에 접근해서는 안 된다. vec2를 값으로 넘기면 이 공유 위험을 피할 수 있다. 컴파일러는 함수가 주어진 인자로 호출 가능한지 검사하고 스레드가 실행할 함수 객체를 구성한다.

15.4 결과 반환

출처: 책 331-332쪽 / 원문 PDF 325-326쪽

비const 참조로 입력을 받아 그것을 변경하는 방식으로 결과를 돌려줄 수도 있다. 원문은 입력을 const 참조로 받고 결과 저장 위치를 별도 인자로 넘기는 편이 더 명확하다고 설명한다. 함수 포인터 인자·함수 객체 멤버·참조 캡처 람다의 세 방식을 나란히 보여 준다.

별도 출력 위치로 결과를 돌려받기

코드 출처: 책 332쪽 / 원문 PDF 326쪽

C++ 원문

```
void f(const vector<double>& v, double* res); // v에서 입력을 받아 *res에 결과를 저장

class F {
public:
    F(const vector<double>& vv, double* p) :v{vv}, res{p} { }
    void operator()(); // *res에 결과 저장
private:
    const vector<double>& v; // 입력
    double* res; // 출력을 저장할 곳
};

double g(const vector<double>&); // 반환 값 사용

void user(vector<double>& vec1, vector<double> vec2, vector<double> vec3)
{
    double res1;
    double res2;
    double res3;

    thread t1 {f,cref(vec1),&res1}; // f(vec1,&res1)을 별도 스레드로 실행
    thread t2 {F{vec2,&res2}}; // F{vec2,&res2}()를 별도 스레드로 실행
    thread t3 { [&](){ res3 = g(vec3); }}; // 지역 변수를 참조로 전달

    t1.join();
    t2.join();
    t3.join();

    cout << res1 << ' ' << res2 << ' ' << res3 << '\n';
}
```

코드의 의미

입력의 const 참조와 결과 저장 위치를 분리한다. 함수·함수 객체·람다의 세 태스크가 완료된 후 결과들을 출력한다. f·F의 호출 연산자·g의 계산 내용은 원문부터 선언만 제시한다.

원문 실행 결과

[원문에 실행 결과 미제시]

모든 join() 뒤에 결과를 사용하므로 각 태스크의 완료를 기다린다. 원문은 이런 참조 기반 결과 전달이 흔하고 동작하지만 덜 우아하다고 평가하고, 이후 future·promise로 더 높은 수준의 결과 전달을 설명한다.

15.5 데이터 공유

출처: 책 333-335쪽 / 원문 PDF 327-329쪽

출처: 책 333쪽 / 원문 PDF 327쪽

공유 데이터를 동시에 변경할 때는 한 번에 한 태스크만 해당 데이터에 접근하도록 동기화한다. 변경 불가능한 데이터를 여러 태스크가 읽는 경우는 구별한다. <mutex>의 **mutex**는 상호 배제 객체이며 lock()으로 획득하고 unlock()으로 해제한다. scoped_lock은 이 획득·해제를 RAII로 묶는다.

scoped_lock으로 공유 데이터 보호

코드 출처: 책 333쪽 / 원문 PDF 327쪽

C++ 원문

```
mutex m; // 제어용 mutex
int sh; // 공유 데이터

void f()
{
    scoped_lock lck {m}; // mutex 획득
    sh += 7; // 공유 데이터 조작
} // 암묵적인 mutex 해제
```

코드의 의미

RAII 락이 mutex 획득과 해제를 스코프에 연결해 공유 값 sh의 변경을 보호한다.

원문 실행 결과

[원문에 실행 결과 미제시]

출처: 책 333-334쪽 / 원문 PDF 327-328쪽

다른 스레드가 mutex를 가진 동안 획득 시도는 대기하며, 해제되면 기다리던 스레드가 진행한다. mutex와 보호 데이터의 관계는 관례적이므로 같은 Record 안에 두거나 이름·주석으로 관계를 명확히 해야 한다. 명시적인 lock/unlock보다 자원 핸들이 간단하고 안전하다.

출처: 책 334쪽 / 원문 PDF 328쪽

서로 다른 순서로 여러 mutex를 획득하면 상대가 가진 mutex를 기다리느라 모두 멈추는 **교착 상태(데드락)**가 생길 수 있다. 원문의 scoped_lock은 여러 mutex를 한 번에 받고 모두 획득한 뒤 진행하며, 일부만 획득한 채로 블록되지 않는다. 소멸 시에는 모두 해제한다.

출처: 책 334-335쪽 / 원문 PDF 328-329쪽

공유 데이터는 호출·반환보다 낮은 수준의 통신이며 락 비용도 있다. 데이터 복사보다 더 빠르다고 측정 없이 가정하지 않는다. **shared_mutex**는 읽기 스레드의 shared_lock을 함께 허용하고, 쓰기 스레드의 unique_lock에는 배타적 접근을 제공하여 읽기/쓰기 락을 구현한다.

15.6 이벤트 대기

출처: 책 335-338쪽 / 원문 PDF 329-332쪽

출처: 책 335-336쪽 / 원문 PDF 329-330쪽

시간 경과는 가장 간단한 이벤트다. this_thread는 현재 스레드를 가리키며 sleep_for로 기다릴 수 있다. chrono의 시각 차이와 duration_cast로 경과 시간을 표시할 수 있고 별도 스레드를 만들 필요는 없다.

출처: 책 336-337쪽 / 원문 PDF 330-331쪽

<condition_variable>의 **condition_variable**은 다른 태스크의 행동으로 조건이나 이벤트가 생기기를 기다리는 수단이다. 원문의 메시지 큐는 queue·조건 변수·mutex를 함께 사용한다. 소비자는 unique_lock을 획득한 뒤 큐가 비어 있지 않다는 술어로 기다린다. wait는 대기 중 락을 해제하고 깨어날 때 다시 획득한다.

조건 변수로 메시지 큐를 기다리는 소비자

코드 출처: 책 337쪽 / 원문 PDF 331쪽

소비자 함수 전체를 발췌했습니다. 336쪽의 전역 선언과 338쪽의 producer 함수는 생략했습니다. 원문의 메시지 처리 생략 주석을 유지했습니다.

C++ 원문

```
void consumer()
{
    while(true) {
        unique_lock lck {mutex}; // mutex 획득
        mcond.wait(lck,[] { return !mqueue.empty(); }); // lck을 해제하고 대기;
            // 깨어날 때 lck을 다시 획득
            // mqueue가 비어있는 동안 깨어나지 않음
        auto m = mqueue.front(); // 메시지 가져오기
        mqueue.pop();
        lck.unlock(); // lck 해제
        // ... m을 처리 ...
    }
}
```

코드의 의미

큐가 비어 있지 않을 때까지 기다리고 메시지를 꺼낸 후 락을 해제하고 처리한다. mqueue·mcond·보호 mutex의 선언과 생산자는 원문의 앞뒤 페이지에 있다.

동작 과정

1. 락을 획득하고 wait에서 조건을 검사한다.
2. 대기 중 락을 놓고 깨어나면 다시 획득한다.
3. 큐에서 메시지를 제거한 뒤 unlock하여 처리 중 락을 오래 잡지 않는다.

원문 실행 결과

[원문에 실행 결과 미제시]

원문 주의 사항

- [원문 확인 필요] 336쪽의 mutex 변수 선언은 mmutex이고 이 소비자 코드는 mutex로 표기되어 있다. 이름을 임의로 통일하지 않았다.

출처: 책 337-338쪽 / 원문 PDF 331-332쪽

깨어났다는 이유만으로 필요한 조건이 성립하는 것은 아니다. 다른 태스크가 먼저 큐에 접근했을 수 있으므로 !mqueue.empty()를 다시 확인한다. 메시지를 가져와 제거한 뒤에는 락을 해제하고 메시지를 처리해 임계 영역 시간을 줄인다. 생산자는 락으로 큐 삽입을 보호하고 notify_one()으로 통지한다.

출처: 책 337쪽 / 원문 PDF 331쪽

unique_lock은 lock()/unlock()으로 낮은 수준의 제어가 가능하며 원문은 하나의 mutex만 다룬다고 설명한다. [원문 확인 필요] 337쪽은 scoped_lock과 비교하면서 unique_lock은 복사가 가능하다고 서술한다. 이 문장은 의심되는 원문 표기로 남겨 두며, 이 노트에서 다른 규칙으로 조용히 교체하지 않는다.

15.7 태스크 간 통신

출처: 책 338쪽 / 원문 PDF 332쪽

<future>는 직접 thread·락을 다루는 것보다 높은 수준의 통신 수단을 제공한다. future·promise는 태스크 사이의 결과 전달, packaged_task는 태스크 실행과 결과의 연결, async는 함수 호출과 비슷한 태스크 시작에 사용한다.

15.7.1 future와 promise

출처: 책 338-340쪽 / 원문 PDF 332-334쪽

출처: 책 338-339쪽 / 원문 PDF 332-333쪽

promise에 값을 넣으면 연결된 **future**에서 그 값을 볼 수 있다. future<X>::get()은 필요한 경우 값 계산이 끝날 때까지 기다리고 X를 얻는다. 값 대신 예외가 전달되거나 값을 계산할 수 없으면 get()에서 예외를 던질 수 있다.

출처: 책 339-340쪽 / 원문 PDF 333-334쪽

promise의 set_value()와 set_exception()은 future의 get()에 대응한다. 태스크는 결과 또는 예외 중 하나를 전달한다. current_exception()은 현재 잡힌 예외를 참조하며, 전달된 예외는 get()의 호출 경로 어딘가에서 처리해야 한다. 락을 명시적으로 조작하지 않고도 시스템이 전달을 수행한다.

promise로 값 또는 예외 전달

코드 출처: 책 339-340쪽 / 원문 PDF 333-334쪽

C++ 원문

```
void f(promise<X>& px) // 태스크: 결과를 px에 넣기
{
    // ...
    try {
        X res;
        // ... res에 값을 계산 ...
        px.set_value(res);
    }
    catch (...) { // res를 계산할 수 없는 경우
        px.set_exception(current_exception()); // future의 스레드로 예외 전달
    }
}
```

코드의 의미

정상 계산이면 set_value, 예외이면 현재 예외를 set_exception으로 연결된 future에 전달한다. 계산·연결 설정은 원문부터 생략되어 있으며 get() 호출자는 전달된 예외를 처리해야 한다.

원문 실행 결과

[원문에 실행 결과 미제시]

15.7.2 packaged_task

출처: 책 340-342쪽 / 원문 PDF 334-336쪽

출처: 책 340-341쪽 / 원문 PDF 334-335쪽

packaged_task는 태스크의 반환값이나 예외를 promise에 넣는 래퍼를 제공하고, get_future()로 대응 future를 돌려준다. 템플릿 인자는 태스크의 호출 타입이며 생성자 인자는 수행할 함수다. 원문의 accum 예제는 vector 구간의 두 절반을 별도 태스크로 합산한다.

packaged_task로 두 구간의 계산과 결과 연결

코드 출처: 책 341쪽 / 원문 PDF 335쪽

C++ 원문

```
double accum(double* beg, double* end, double init)
    // 초깃값 init부터 시작해 [beg:end)를 모두 더함
{
    return accumulate(beg,end,init);
}

double comp2(vector<double>& v)
{
    using Task_type = double(double*,double*,double); // 태스크의 타입

    packaged_task<Task_type> pt0 {accum}; // 태스크(즉, accum)를 패키징
    packaged_task<Task_type> pt1 {accum};

    future<double> f0 {pt0.get_future()}; // pt0의 future를 저장
    future<double> f1 {pt1.get_future()}; // pt1의 future를 저장

    double* first = &v[0];
    thread t1 {move(pt0),first,first+v.size()/2,0}; // pt0를 위한 스레드 시작
    thread t2 {move(pt1),first+v.size()/2,first+v.size(),0}; // pt1을 위한 스레드 시작

    // ...

    return f0.get()+f1.get(); // 결과 가져오기
}
```

코드의 의미

accum 함수와 comp2의 전체 원문 블록을 보존했다. packaged_task에서 future를 얻고 두 절반의 계산을 시작한 다음 두 결과를 가져와 합친다.

원문 실행 결과

[원문에 실행 결과 미제시]

원문 주의 사항

- 원문 자체에 // ... 생략 구간이 있다. join 등 이 구간에서 확인되지 않는 코드를 추가하지 않았다.

출처: 책 341-342쪽 / 원문 PDF 335-336쪽

packaged_task는 promise를 소유하는 자원 핸들이므로 복사할 수 없고 thread로 전달할 때 move가 필요하다. 명시적인 락 없이 태스크와 결과에 집중할 수 있으며 별도 스레드의 두 작업은 잠재적으로 병렬 실행된다.

15.7.3 async()

출처: 책 342-343쪽 / 원문 PDF 336-337쪽

출처: 책 342쪽 / 원문 PDF 336쪽

async()는 잠재적으로 비동기 실행할 태스크를 함수 호출과 유사하게 시작한다. 호출 시작·결과 가져오기·실제 실행을 분리하여 thread나 락보다 태스크 자체에 집중하게 한다. 원문은 충분히 큰 vector를 네 구간으로 나누고 future의 결과를 모으는 예를 든다.

async로 네 구간의 결과를 모으기

코드 출처: 책 342쪽 / 원문 PDF 336쪽

C++ 원문

```
double comp4(vector<double>& v)
    // v가 충분히 크면 많은 태스크를 시작
{
    if (v.size()<10000) // 동시성을 활용할 만한가?
        return accum(v.begin(),v.end(),0.0);

    auto v0 = &v[0];
    auto sz = v.size();

    auto f0 = async(accum,v0,v0+sz/4,0.0); // 첫 번째 1/4
    auto f1 = async(accum,v0+sz/4,v0+sz/2,0.0); // 두 번째 1/4
    auto f2 = async(accum,v0+sz/2,v0+sz*3/4,0.0); // 세 번째 1/4
    auto f3 = async(accum,v0+sz*3/4,v0+sz,0.0); // 네 번째 1/4

    return f0.get()+f1.get()+f2.get()+f3.get(); // 결과를 모아서 병합
}
```

코드의 의미

작으면 순차 누적을 호출하고 그렇지 않으면 네 태스크를 시작하여 결과를 합치는 원문 함수다. 앞 절의 accum을 사용한다.

원문 실행 결과

[원문에 실행 결과 미제시]

원문 주의 사항

- 10000이라는 기준은 원문이 원시적이고 잘못된 성능 가정일 수 있다고 경고하는 예다.
- 원문은 async의 스레드 수를 알 수 없으며 락으로 데이터를 공유하는 태스크에는 사용할 수 없다는 제한으로 설명한다.

출처: 책 343쪽 / 원문 PDF 337쪽

원문은 락을 이용해 데이터를 공유하는 태스크에 async를 사용할 수 없다는 한계로 서술하며, 스레드 수는 호출 시점의 시스템 자원을 고려해 결정되므로 알 수 없다고 설명한다. v.size()<10000 같은 기준은 스레드 시작 비용과 계산 비용에 대한 원시적인 가정으로, 성능 판단을 잘못할 수 있음을 경고한다.

출처: 책 343쪽 / 원문 PDF 337쪽

accumulate 같은 표준 연산을 직접 병렬화하기보다 reduce(par_unseq, ...) 같은 병렬 알고리즘이 더 잘 동작할 수 있다고 원문은 권한다. async는 성능을 위한 병렬 계산만을 위한 것이 아니며 메인 작업 중 사용자 입력을 받는 태스크 같은 반응성 용도도 있다.

15.8 조언

출처: 책 343-345쪽 / 원문 PDF 337-339쪽

- 처리량·반응성에 동시성이 필요한지 살피고 가능한 높은 추상화 수준에서 작업한다. 순차 해법과 병렬 알고리즘을 먼저 고려한다.
- 공유 데이터와 데이터 경합을 가능한 피한다. thread의 종료는 join으로 기다린다.
- mutex는 보호하는 데이터와 가까이 두고 RAII 핸들로 관리한다. 여러 락은 scoped_lock으로 함께 획득한다.
- 이벤트 통신에는 condition_variable과 unique_lock을 쓰고 조건 없이 기다리지 않는다. 임계 영역 시간을 최소화한다.

- 태스크를 동시에 실행할 수 있는 함수로 생각한다. promise로 결과를 제공하고 future로 얻으며 packaged_task로 값·예외 전달을 묶는다.
- 간단한 태스크 시작에는 async를 사용하고 thread·mutex를 직접 다루는 일을 줄인다.

중요 내용 요약

- 동시성은 처리량·반응성 목적에 맞춰 선택하며 순차 해법보다 항상 빠른 것은 아니다.
- thread의 join은 실행 종료를 기다린다.
- 참조 전달 시 공유 데이터의 접근 규칙이 필요하다.
- 락은 RAII로 관리하고 여러 락 획득에는 데드락을 주의한다.
- 조건 변수 대기에는 조건 술어를 유지한다.
- promise는 값·예외를 넣고 future는 get으로 받아들이며 packaged_task가 이 연결을 묶는다.

혼동하기 쉬운 내용

태스크 통신 수단의 역할

출처: 책 338-343쪽 / 원문 PDF 332-337쪽

수단	원문에서의 역할
promise	값이나 예외를 넣는 쪽
future	값이 준비될 때까지 기다리고 결과·예외를 받는 쪽
packaged_task	태스크와 promise를 묶고 대응 future를 제공
async	함수 호출처럼 태스크를 시작하고 결과 획득을 분리

원문에서 구별하는 락 핸들

출처: 책 333-337쪽 / 원문 PDF 327-331쪽

핸들	용도와 원문 조건
scoped_lock	스코프 기반 mutex 관리; 여러 mutex를 함께 획득
unique_lock	조건 변수 wait와 명시적인 lock/unlock; 한 mutex 관리
shared_lock	shared_mutex에서 여러 읽기 접근을 공유

장 마무리 복습

- 분리된 태스크를 함수처럼 다루는 방식과 명시적인 공유 데이터 접근의 차이를 복습한다.
- scoped_lock·unique_lock·shared_lock의 역할과 보호 데이터의 관계를 정리한다.
- wait의 락 해제·재획득·조건 확인 순서를 설명한 원문 흐름을 재확인한다.
- 값 또는 예외가 promise에서 future로 전달되는 관계를 복습한다.
- packaged_task의 이동과 async의 실행 자원·성능 가정에 관한 원문 조건을 정리한다.

원문 확인 메모

- [원문 확인 필요] 337쪽은 unique_lock이 복사 가능하다고 서술한다. 원문 표기를 보존하고 다른 규칙을 보충하지 않았다.

- [원문 확인 필요] 메시지 큐 예제는 336쪽 선언에서 mmutex, 337~338쪽 사용에서 mutex로 표기한다. 코드를 조용히 수정하지 않았다.

제16장. 역사와 호환성

출처: 책 347-370쪽 / 원문 PDF 1-12, 340-351쪽

학습 내용 개요

- 실제 시스템 문제를 해결하면서 언어와 라이브러리가 함께 발전한 과정.
- C++11·14·17의 언어 기능과 라이브러리 추가 사항, 제거·지원 중단의 구별.
- C/C++의 역사적 관계와 스타일·void*·링크 호환성.

설명의 흐름

1. 역사와 표준화 → 표준의 동작 규정과 효과적인 스타일의 구별 → 버전별 기능 목록 → C와 C++의 관계 → 호환성 문제

16.1 역사

출처: 책 347-348쪽 / 원문 PDF 340-341쪽

C++는 시뮬러의 프로그램 조직화 기능과 C의 시스템 프로그래밍 효율성·유연성을 함께 제공하도록 설계됐다. 클래스와 파생 클래스, 가상 함수는 시뮬러에서 영향을 받았고, 템플릿과 예외는 이후에 다른 아이디어를 바탕으로 추가됐다.

원문은 언어의 진화를 실제 활용 사례와 연결한다. 사용자와 동료의 의견을 듣고 당면한 문제를 해결하는 과정에서 언어와 라이브러리가 함께 발전했다. 아래 연혁과 기능 상태는 원문이 서술하던 시점을 그대로 따른다.

16.1.1 연혁

출처: 책 348-350쪽 / 원문 PDF 341-343쪽

시기	원문에서 설명한 변화
1979	C with Classes 작업 시작. 클래스·파생 클래스, public/private, 생성자·소멸자, 타입 검사되는 함수 선언을 도입했다.
1984~1985	이름을 C++로 바꾸고 가상 함수, 함수·연산자 오버로딩, 참조를 포함했다. 1985년 10월 14일 첫 상용 릴리스가 나왔다.
1989~1991	ARM과 두 번째 The C++ Programming Language가 등장했다. 템플릿 기반 제네릭 프로그래밍, 예외 기반 에러 처리, 일반적인 자원 관리 관례인 RAII가 소개됐다.
1997~1998	네임스페이스·dynamic_cast·템플릿 개선과 STL을 포함한 ISO C++가 소개되고, 1998년 첫 ISO 표준이 제정됐다.
2002~2003	C++0x 개정 작업을 시작했다. 2003년 표준 버그 수정과 Technical Report의 새 라이브러리 구성 요소는 이후 C++11로 이어졌다.
2011~2013	C++11은 초기화, 이동, auto, 구간 for, 가변 템플릿, 람다, 타입 별칭, 동시성 메모리 모델 등을 포함했다. 원문은 완전한 구현체가 2013년에 등장했다고 서술한다.
2014~2015	C++14의 가변 템플릿·자릿수 구분자·제네릭 람다·라이브러리 개선, Core Guidelines 프로젝트 시작, 컨셉트 TS 승인을 기록한다.
2017	C++17과 첫 구현체, 모듈 TS 및 Ranges TS 승인을 기록한다.
2020	원문에서는 ISO C++20 표준을 예정 사항으로 적었다. 현재 상태로 바꾸지 않는다.

C++11은 개발 중 C++0x로 불렸다. 원문은 완료 시점에 대한 낙관과 긴 지연을 설명하면서, C++14·C++17은 정해진 시점에 완료됐다고 대조한다.

16.1.2 초창기

출처: 책 350-354쪽 / 원문 PDF 343-347쪽

처음 해결하려던 문제는 멀티프로세서와 로컬 네트워크에 유닉스 커널 서비스를 배포하는 일이었다. 시스템 부분과 상호작용을 정확히 명시하면서 하드웨어를 직접 다루고 높은 성능을 얻어야 했다. 시뮬러는 조직화에 적합했으나 성능이 문제였고, C는 하드웨어와 성능에 적합했으나 모듈화가 어려웠다. 이 두 요구를 결합한 것이 C with Classes였다.

초기에는 이후의 연산자 오버로딩·참조·가상 함수·템플릿·예외 등이 없었다. C++라는 이름은 릭 매시티가 1983년 여름 제안했고, C의 증가 연산자 ++를 사용해 C의 확장을 표현했다. 설계·문서화·구현은 함께 진행됐으며, 초창기의 진화는 공식 위원회보다 사용자 문제에 대한 토론에 기초했다.

초기 설계의 중심에는 함수 인자 타입 검사, 클래스의 public/private에 의한 인터페이스와 구현 분리, 생성자와 소멸자가 있었다. 원문은 생성자·소멸자 도입을 특히 중요하게 평가한다. 객체의 실행 환경을 만들고 되돌리는 기능이 자원 관리 전략과 간결한 표현 기법의 출발점이 됐기 때문이다.

가상 함수는 실행 시간의 선택을 표현하는 다형성을 제공했지만, 시스템 코드에 충분히 빠른지에 대한 논쟁이 있었다. 원문은 근본적으로 복잡한 시스템을 만들 때 언어와 도구의 도움을 활용해야 한다고 설명한다. C++를 범용 언어, 더 나은 C, 데이터 추상화를 제공하며 객체지향 프로그래밍을 지원하는 언어로 서술한다. 데이터 추상화에는 정보 은닉뿐 아니라 클래스 계층에 속하지 않는 클래스와 제네릭 프로그래밍도 포함한다.

템플릿을 설계할 때 유연성·효율성·이른 타입 검사를 동시에 얻기 어려웠다. C 스타일 시스템 코드와 경쟁하기 위해 유연성과 효율성을 선택했고, 타입 확인 개선은 이어졌다. 예외 설계는 여러 호출 수준으로의 전파, 핸들러에 임의 정보 전달, 로컬 객체의 소멸자에 기반한 자원 관리와의 통합에 초점을 뒀다. 이 자원 관리 기법을 Resource Acquisition Is Initialization, RAII라 부른다.

추상 클래스의 다중 상속을 포함한 상속 일반화도 이루어졌지만, 원문은 템플릿과 예외보다 중요도가 낮다고 평가한다. 복소수·벡터·스택·입출력 스트림을 설계하면서 연산자 오버로딩도 설계했고, 문자열·리스트와 템플릿 라이브러리 경험이 언어에 영향을 주었다. 1998년 표준 라이브러리의 주요 혁신은 제네릭 컨테이너와 알고리즘의 프레임워크인 STL이었다.

초기의 태스크 라이브러리와 달리 표준 동시성 기능이 널리 사용되기까지는 2011년까지 기다려야 했다고 서술한다. 코루틴의 C++20 포함은 원문에서 예정으로 표현된다. 여러 언어의 영향보다 당면한 응용 분야의 요구를 결정적인 기준으로 삼은 문제 지향적 개발이 핵심이다.

16.1.3 ISO C++ 표준

출처: 책 354-357쪽 / 원문 PDF 347-350쪽

사용이 늘어나면서 구현자와 사용자 사이의 일관된 약속이 필요해졌다. 1987년 무렵 표준화의 필요성이 분명해졌고, 공유·검토된 레퍼런스 매뉴얼이 ANSI 표준의 기초가 됐다. ANSI X3J16 위원회는 1989년 12월 소집됐고, 1991년에는 ISO 작업에 포함됐다. ISO C++ 위원회는 WG21로 불린다.

첫 ISO C++ 표준은 1998년 22:0으로 승인됐다. 2003년 버그 수정판은 원문에서 기본적으로 C++98과 같은 언어라고 설명한다. 두 번째 표준 C++11은 2011년 8월 21:0으로 승인됐다. 까다로워진 작업 절차는 명세의 엄격성을 높이는 동시에 혁신을 제한하기도 했다.

표준 문서가 커진 것은 새 기능뿐 아니라 더 자세한 명세 때문이었다. 기존 수십억 줄의 코드를 망가뜨리지 않도록 장기간 안정성을 유지하는 것도 필수 요구였다. C++11의 전반적인 목표는 시스템 프로그래밍과 라이브러리 제작에 더 적합한 언어를 만들고, 가르치고 배우기 쉽게 하는 것이었다. 타입 안전성과 이식성을 갖춘 동시성, 적합한 메모리 모델과 스레드 라이브러리는 이 목표의 일부다.

C++11 이후에는 13년 간격이 너무 길다는 공감 아래 정해진 시점에 표준을 내는 기차 모델과 3년 주기를 채택했다. 기능을 병렬로 개발하기 위해 기술 명세 TS를 사용했지만, 추가 단계와 지연이 생기기도 했다. 원문은 C++17이 계획된 대규모 변화에 이르지 못했고 컨셉·모듈·레인지가 준비되지 않았다고 평가한다. 이 평가는 원문 시점의 평가로 보존한다.

16.1.4 표준과 스타일

출처: 책 357-358쪽 / 원문 PDF 350-351쪽

표준은 기능의 동작을 규정하지만 효과적인 사용법을 모두 알려주지는 않는다. 기능 자체의 기술적 이해와, 기능·라이브러리·도구를 함께 써서 관리하기 쉽고 에러가 적으며 빠른 소프트웨어를 만드는 일은 구별된다. 일관된 프로그래밍 스타일을 개발·지원하고 기존 코드도 그 스타일로 발전시키는 일이 필요하다.

2015년에 시작한 C++ Core Guidelines는 원문에서 타입 안전성과 자원 안전성에 초점을 둔 노력으로 소개된다. 정적 분석 도구와 이를 지원하는 라이브러리의 도움으로 효과적인 스타일을 지원하려는 것이다. 이 교재는 원문이 언급한 목적만 정리하며 외부 지침의 세부 내용을 덧붙이지 않는다.

16.1.5 C++의 사용

출처: 책 358-359쪽 / 원문 PDF 1, 351쪽

원문은 C++를 산업계의 요구 속에서 발전한 언어로 설명한다. 장치 드라이버·네트워킹·임베디드 시스템을 비롯해 운영체제, 웹 기반 시설, 금융, 의료, 자동차, 항공우주, 과학, 그래픽스·게임 등 여러 응용 분야를 든다. 공통점은 하드웨어를 효과적으로 사용하면서 복잡도를 낮추는 능력이 필요하다는 것이다. 사용 분야와 사용자 수에 관한 서술은 원문의 역사적 관점에 속한다.

16.2 C++ 기능의 진화

출처: 책 359쪽 / 원문 PDF 1쪽

이 절은 C++11·C++14·C++17에 추가된 언어 기능과 표준 라이브러리를 구별해 나열한다. 기능은 따로 고립돼 쓰이지 않으며, 새 기능 상당수는 기존 기능이 제공하는 틀 위에서 의미를 갖는다. 아래 목록은 원문의 기능 이름을 학습 항목별로 묶은 것이며, 이 장에서 설명하지 않은 세부 규칙은 새로 보충하지 않는다.

16.2.1 C++11 언어 기능

출처: 책 359-361쪽 / 원문 PDF 1-3쪽

관련 항목	원문 목록의 주요 기능
초기화·상수·타입	{} 범용 초기화, auto 타입 추론, 타입 축소 방지, constexpr, enum class, static_assert, initializer_list 매핑
순회·값·함수	구간 for, nullptr, rvalue 참조와 이동, 람다, 가변 템플릿 인자, 타입·템플릿 별칭
문자·표현·선언	유니코드 문자, long long, alignas/alignof, decltype, 로우 문자열, 일반화된 POD·union, 로컬 클래스의 템플릿 인자 사용, 접미사 반환 타입 구문
예외·속성	[[carries_dependency]], [[noreturn]], noexcept 지정자와 noexcept 연산자
매크로·이름 공간	C99 관련 확장 정수·문자열·매크로 기능, __func__, inline 네임스페이스
생성·변환·템플릿 제어	생성자 위임, 클래스 내 멤버 초기화, default/delete, 명시적 변환 연산자, 사용자 정의 리터럴, extern template, 함수 템플릿 기본 인자, 생성자 상속
언어 규칙·동시성	중첩 템플릿 인자의 >> 표기, override/final, SFINAE 규칙, 메모리 모델, thread_local

16.2.2 C++14 언어 기능

출처: 책 361쪽 / 원문 PDF 3쪽

- 함수 반환 타입 추론과 개선된 constexpr 함수(예: for 루프 허용).
- 가변 템플릿, 바이너리 리터럴과 자릿수 구분자.
- 제네릭 람다, 더 일반적인 람다 캡처, [[deprecated]] 속성과 작은 확장들.

16.2.3 C++17 언어 기능

출처: 책 361-362쪽 / 원문 PDF 3-4쪽

- 안전성이 보장된 복사 생략, 과정렬된 타입의 동적 할당, 더 엄격한 평가 순서.
- UTF-8 리터럴(u8), 16진 부동소수점 리터럴, 접힘 표현식.
- auto 템플릿 파라미터, 클래스 템플릿 인자 타입 추론, 컴파일 시간 if.
- 초깃값을 포함한 선택 구문, constexpr 람다, inline 변수, 구조화된 바인딩.

- [[fallthrough]], [[nodiscard]], [[maybe_unused]], std::byte, 기저 타입 값으로 enum 초기화와 작은 확장들.

16.2.4 C++11 표준 라이브러리 구성 요소

출처: 책 362-363쪽 / 원문 PDF 4-5쪽

C++11 라이브러리의 확장은 새 구성 요소의 추가와 기존 구성 요소의 개선을 모두 포함한다. 예를 들어 정규 표현식은 새 구성 요소이고, 컨테이너의 이동 생성자 지원은 기존 구성 요소의 확장이다.

묶음	원문 목록
컨테이너·자원	initializer_list 생성자, 컨테이너 이동, forward_list, unordered_map/unordered_multimap/unordered_set/unordered_multiset, unique_ptr/shared_ptr/weak_ptr, array, emplace(), 스코프 할당자
동시성·예외	thread, 뮤텍스·락·조건 변수, future·promise·async(), 예외 복사와 다시 던짐, system_error, atomic
범용 라이브러리	tuple, regex, 난수 분포·엔진, 정수 타입 이름, constexpr/noexcept 사용, function·bind(), 문자열 수치 변환
특성·시간·알고리즘	타입 특성, duration/time_point, ratio, quick_exit, move()/copy_if()/is_sorted(), 가비지 컬렉션 ABI

[원문 확인 필요] 원문 363쪽의 높은 수준 동시성 목록에는 packaged_thread라고 인쇄돼 있다. 이 장에서는 원문 표기를 보존하며, 다른 식별자로 조용히 고치지 않는다. 해당 기능의 구체적 코드는 15장 원문과 구별해 확인한다.

16.2.5 C++14 표준 라이브러리 구성 요소

출처: 책 363-364쪽 / 원문 PDF 5-6쪽

- shared_mutex와 사용자 정의 리터럴.
- 타입을 이용한 튜플 접근, 연관 컨테이너의 이형 조회, 작은 기능 추가.

16.2.6 C++17 표준 라이브러리 구성 요소

출처: 책 364쪽 / 원문 PDF 6쪽

- 파일 시스템, 병렬 알고리즘, 특수 수학 함수.
- string_view, any, variant, optional, invoke().
- 기초 문자열 변환 to_chars/from_chars, 다형적 할당과 작은 확장.

16.2.7 제거되거나 지원 중단된 기능

출처: 책 364-365쪽 / 원문 PDF 6-7쪽

광범위한 기존 코드 때문에 기능 제거는 오랜 경고 뒤에 신중히 이뤄진다. 원문은 C++17에서 예외 명세와 그 지원 기능인 unexcepted_handler·set_unexpected()·get_unexpected()·unexpected()가 제거됐다고 설명하고 noexcept 사용을 권한다. [원문 확인 필요] unexcepted_handler는 원문 365쪽의 표기를 그대로 옮겼다.

원문에서 제거 사례로 든 예외 명세

코드 출처: 책 364쪽 / 원문 PDF 6쪽

C++ 원문

```
void f() throw(X,Y); // C++98 스타일, 이제부턴 에러
```

코드의 의미

원문이 C++17에서 제거된 과거의 예외 명세 형태로 제시한 선언이다.

원문 실행 결과

[원문에 실행 결과 미제시]

원문 주의 사항

- 원문은 같은 맥락에서 noexcept 사용을 권한다. 이 선언을 유효한 최신 예제로 고치지 않고 보존했다.
- 문자 세 개의 예약어(trigraphs), 저장 영역 명시자 register, bool에 대한 ++가 제거되거나 허용되지 않는다.
- auto_ptr 대신 unique_ptr을 사용한다. C++98의 export 기능은 제거됐고, 원문은 모듈 키워드의 export를 따로 언급한다.
- 소멸자가 있는 클래스에서 복사 연산을 자동 생성하는 기능은 지원 중단으로 설명된다.
- 문자열을 char*에 대입하는 기능은 제거됐으며 const char* 또는 auto를 사용하도록 한다.
- 기존 일부 함수 객체와 관련 함수는 람다와 function으로 대체하도록 권한다.

지원 중단은 해당 기능을 피하라는 강한 신호이지만 즉시 제거와 동일하지 않다. 컴파일러는 경고를 낼 수 있고, 호환성을 위해 지원 중단 기능이 오래 유지될 수도 있다.

16.3 C/C++ 호환성

출처: 책 365쪽 / 원문 PDF 7쪽

원문은 사소한 예외를 제외하면 C++가 C11의 대부분을 포함하며, 차이의 상당 부분이 C++의 엄격한 타입 검사에서 생긴다고 설명한다. 이 설명은 두 언어의 모든 프로그램과 기능이 완전히 호환된다는 뜻으로 확대하지 않는다. 차이는 컴파일러 진단으로 확인할 수 있다고 서술한다.

16.3.1 C와 C++는 형제

출처: 책 366-367쪽 / 원문 PDF 8-9쪽

ISO C와 ISO C++는 Classic C에서 갈라져 서로 다른 속도와 방향으로 발전한 형제 언어로 설명된다. 둘 다 전통적인 C 스타일을 제공하지만 방식이 약간 다르고, 둘 모두 Classic C와도 100% 호환되지는 않는다. 가계도는 기능을 직접 계승한 관계와 다른 언어에서 차용한 관계를 구별한다.

호환성 차이의 원인이 언제나 C++의 변경인 것은 아니다. 분리 후 C에 추가된 기능에서 생긴 차이도 있다. 원문은 T*와 void* 대입, 전역 const의 링크, inline의 세부 의미를 예로 든다. 한 언어의 라이브러리를 다른 언어에서 쓰거나 공통 도구를 만드는 경우 이 차이를 고려해야 한다.

16.3.2 호환성 문제

출처: 책 367쪽 / 원문 PDF 9쪽

- 설계와 프로그래밍 스타일이 C++의 기능을 활용하지 못할 수 있다.
- void*를 캐스트 없이 T*로 바꾸는 코드는 문제가 될 수 있다.
- class·private 등 C++ 키워드를 C 코드의 식별자로 쓸 수 있다.
- C와 C++로 각각 컴파일한 코드 사이의 링크 호환성을 고려해야 한다.

다른 문제가 없다면 C 코드를 C로 컴파일한 뒤 extern "C"를 이용해 C++에 링크할 수 있다. 따라서 문법 변환, 설계 개선, 링크 문제는 각각 구별해서 다뤄야 한다.

16.3.2.1 스타일 문제

출처: 책 367-369쪽 / 원문 PDF 9-11쪽

C 프로그램을 한 줄씩 C++로 바꾸는 것만으로는 기본 구조와 에러의 근본 원인이 사라지지 않는다. 원문은 불완전한 에러 처리, 자원 누수, 버퍼 오버플로우가 번역 뒤에도 남을 수 있다고 설명한다. 혜택을 얻으려면 설계와 구현 스타일도 바꿔야 한다.

- 표준 라이브러리를 새로운 스타일의 학습 자료로 사용하고 C 라이브러리와의 차이를 확인한다.
- 매크로 대신 원문에 제시된 const·constexpr·enum/enum class·inline·template·namespace를 목적에 맞게 사용한다.
- 변수를 필요 지점에서 선언하고 초기화한다. 원문은 구문 위치, for 초기화 절, 조건 절에서의 선언을 언급한다.
- malloc 대신 new, realloc 대신 vector를 권하지만 malloc/free를 무조건 new/delete로 맞바꾸는 방식은 피하도록 한다.
- 매우 낮은 수준의 구현 외에는 void*·공용체·캐스트를 줄인다. 명시 변환이 불가피하면 의도가 드러나는 명명된 캐스트를 사용한다.
- 배열·C 스타일 문자열 대신 string·array·vector를 활용한다. 특화 코드나 단순 배열 순회 외에는 포인터 산술을 사용하지 않도록 한다.
- 장황한 C 스타일 코드가 더 짧은 표준 라이브러리 사용 코드보다 반드시 효율적이라고 가정하지 않는다.

16.3.2.2 void*

출처: 책 369-370쪽 / 원문 PDF 11-12쪽

C에서 void*를 다른 포인터 타입의 대입 우측이나 초깃값으로 사용할 수 있는 경우에도 C++에서는 항상 허용되지 않는다. 원문은 malloc의 결과를 int*로 바로 초기화하는 코드를 이 차이의 예로 제시한다.

malloc 결과의 암묵적 포인터 변환

코드 출처: 책 369쪽 / 원문 PDF 11쪽

C++ 원문

```
void f(int n)
{
    int* p = malloc(n*sizeof(int));  /* C++에서는 불허. C++의 "new"로 할당하자. */
    // ...
}
```

코드의 의미

C에서는 가능한 void*의 대입이 C++에서는 허용되지 않는 사례다. 코드의 // ...는 원문에 있는 생략 표시다.

동작 과정

1. malloc 호출 결과로 int* p를 초기화하려 한다. 원문은 그 암묵적 변환을 C++에서 허용하지 않는다고 설명한다.

원문 실행 결과

[원문에 실행 결과 미제시]

원문 주의 사항

- 원문은 C++만 쓸 때 malloc 사용을 피하라고 권한다. 누락된 본문이나 헤더를 보충하지 않았다.

이 제한은 단순한 불편만을 의미하지 않는다. void*를 임의의 포인터 타입으로 암묵적으로 바꿀 때 잘못된 메모리 접근이 생길 수 있음을 다음 예가 보여 준다. 결과를 수치나 출력으로 새로 예측하지 않는다.

void* 변환이 초래할 수 있는 문제

코드 출처: 책 369쪽 / 원문 PDF 11쪽

C++ 원문

```
char ch;
void* pv = &ch;
int* pi = pv;     // C++에서는 허용하지 않음
*pi = 666;       // ch와 근처의 바이트를 덮어씀
```

코드의 의미

서로 다른 포인터 타입을 void*를 거쳐 암묵적으로 바꿀 때 생기는 위험을 보여 주는 원문 코드다.

동작 과정

1. ch의 주소를 pv에 저장한다.
2. pv를 int* pi로 암묵적으로 바꾸는 줄은 원문에서 C++에 허용되지 않는다고 표시한다.
3. 마지막 대입의 영향은 원문의 주석 범위로만 설명한다.

원문 실행 결과

[원문에 실행 결과 미제시]

원문 주의 사항

- 실행 가능한 올바른 C++ 예제가 아니라, 금지된 변환의 이유를 설명하는 코드다. 잘못된 줄을 수정하지 않았다.

원문은 두 언어에서 모두 사용할 malloc 코드라면 결과를 적절한 타입으로 캐스트해야 한다고 설명한다. C++만 사용할 때는 malloc을 사용하지 않도록 권한다.

16.3.2.3 링크

출처: 책 370쪽 / 원문 PDF 12쪽

C와 C++는 서로 다른 링크 방식으로 구현될 수 있다. 원문은 타입 검사와 함수 오버로딩을 그 이유로 든다. C 함수와 C++ 코드 사이에서 호출할 수 있게 하려면 C 링크를 명시한다.

C 링크를 지정하는 선언

코드 출처: 책 370쪽 / 원문 PDF 12쪽

C++ 원문

```
extern "C" double sqrt(double);
```

코드의 의미

sqrt(double)에 C 링크를 지정해 C와 C++ 코드 조각 사이에서 호출할 수 있도록 하는 원문 선언이다.

원문 실행 결과

[원문에 실행 결과 미제시]

원문 주의 사항

- 한 스코프에서 같은 이름의 C 링크 함수는 하나뿐이다.
- 링크 명세는 C++의 함수 호출 및 인자 타입 검사를 바꾸지 않는다.

이 선언을 통해 C 또는 C++ 코드 조각에서 sqrt(double)을 호출할 수 있으며, 정의는 C나 C++ 함수로 컴파일될 수 있다. C는 오버로딩을 지원하지 않으므로 한 스코프에서 특정 이름으로 C에 링크되는 함수는 하나뿐이다. 링크 명세는 타입 검사를 바꾸지 않으며, C++ 함수 호출·인자 검사 규칙이 그대로 적용된다.

16.4 참고 자료

출처: 책 370쪽 / 원문 PDF 12쪽

제공 파일에서 확인되는 참고 자료는 Boost 라이브러리 항목과 1990년 C 표준 항목까지다. 여기서 인용된 자료는 추가 지식의 출처로 사용하지 않았다. 이어지는 인쇄 371쪽 이후의 목록은 [원문에서 확인할 수 없음].

16.5 조언

출처: 원문 PDF 17쪽

[원문에서 확인할 수 없음] 목차에는 16.5 조언이 인쇄 376쪽부터 시작한다고 나오지만 해당 본문은 제공되지 않았다. 목차의 위치만 확인했으며 조언의 내용을 만들어 넣지 않았다.

중요 내용 요약

- C++의 핵심 설계는 C의 시스템 효율성과 시뮬러의 조직화·추상화 요구를 함께 다루는 데서 출발했다.
- 생성자·소멸자와 예외는 로컬 객체에 기반한 자원 관리 전략으로 연결된다.
- 표준은 동작을 규정하고, 프로그래밍 스타일은 기능·라이브러리·도구를 효과적으로 결합한다.
- 지원 중단은 제거 의사의 표현이지만 즉시 제거와 같지 않다.
- C 코드를 줄 단위로 옮기는 일과 C++의 설계·자원 관리 혜택을 얻는 일은 구별된다.
- extern "C"는 링크 명세이며 인자 타입 검사 규칙을 없애지 않는다.

혼동하기 쉬운 내용

표준과 스타일

출처: 책 357-358쪽 / 원문 PDF 350-351쪽

기준	표준	스타일
주요 역할	무엇이 어떻게 동작하는지 규정	기능·라이브러리·도구를 함께 효과적으로 사용
원문의 관심	기능의 정확한 명세	관리하기 쉽고 에러가 적으며 빠른 소프트웨어

지원 중단과 제거

출처: 책 364-365쪽 / 원문 PDF 6-7쪽

구분	원문의 설명
지원 중단	피해야 한다는 강한 신호이며 경고를 낼 수 있다. 호환성 때문에 유지될 수도 있다.
제거	표준에서 기능을 없앤다. 원문은 대규모 기존 코드 때문에 오랜 경고와 신중함이 필요하다고 설명한다.

C/C++ 호환성의 서로 다른 층위

출처: 책 367-370쪽 / 원문 PDF 9-12쪽

층위	원문에서 든 쟁점
설계와 스타일	문법만 옮겨도 자원 누수·불완전한 에러 처리의 근본 원인은 남을 수 있다.
타입 검사	void*의 암묵적 변환처럼 C++에서 더 엄격한 제한이 있다.
링크	각 언어로 컴파일한 코드를 연결할 때 extern "C"로 C 링크를 명시한다.

장 마무리 복습

- 문제 지향적인 설계와 언어·라이브러리의 공동 발전을 연결해 복습한다.
- 초기 C with Classes, C++98, C++11, C++14, C++17의 흐름은 원문의 연혁으로 확인한다.
- 이 장의 C++20과 코루틴에 대한 예정 표현은 원문 시점의 설명으로 읽는다.
- C에서 C++로 옮길 때 스타일, void* 변환, 키워드 충돌, 링크를 구별한다.
- 실행 결과가 없는 코드에는 출력이나 예상값을 만들어 붙이지 않는다.

원문 확인 메모

- 인쇄 371쪽 이후의 참고 자료 나머지와 16.5 조언 본문은 제공되지 않았다. 16.5는 목차에서 존재와 시작 번호만 확인했다.
- 제공 PDF의 1~12쪽에 놓인 인쇄 359~370쪽을 347~358쪽 뒤로 돌려 원문 장 순서로 편집했다.
- 연혁과 기능 지원·예정 상태를 현재 사실로 갱신하지 않았다.
- 363쪽 packaged_thread 표기는 원문 확인이 필요한 표기로 보존했다.

핵심 문법 및 코드 모음

핵심 문법

제1장. 기초 - Hello, World! 출력

출처: 책 22쪽 / 원문 PDF 21쪽

C++ 원문

```
#include <iostream>

int main()
{
    std::cout << "Hello, World!\n";
}
```

입출력 선언을 포함하고 main()에서 문자열 리터럴을 표준 출력에 쓴다. 상세 동작·주의 사항과 실행 결과 제시 여부는 해당 장의 같은 제목 코드 해설에서 확인한다.

제2장. 사용자 정의 타입 - Node*와 int의 저장 공간 공유

출처: 책 56쪽 / 원문 PDF 55쪽

C++ 원문

```
union Value {
    Node* p;
    int i;
};
```

동시에 쓰지 않는 두 종류의 값을 같은 메모리에 저장한다. 상세 동작·주의 사항과 실행 결과 제시 여부는 해당 장의 같은 제목 코드 해설에서 확인한다.

제3장. 모듈화 - 컴파일 시간 속성 검사

출처: 책 80쪽 / 원문 PDF 79쪽

C++ 원문

```
static_assert(4<=sizeof(int), "정수가 너무 작음"); // 정수 크기 확인
```

현재 시스템에서 int 크기가 최소 4바이트라는 기대를 컴파일 시간에 확인한다. 상세 동작·주의 사항과 실행 결과 제시 여부는 해당 장의 같은 제목 코드 해설에서 확인한다.

제4장. 클래스 - 구체 구현을 모르는 사용자 함수

출처: 책 103쪽 / 원문 PDF 101쪽

C++ 원문

```
void use(Container& c)
{
    const int sz = c.size();
    for (int i=0; i!=sz; ++i)
        cout << c[i] << '\n';
}
```

Container 인터페이스만으로 모든 요소를 순회해 출력한다. 상세 동작·주의 사항과 실행 결과 제시 여부는 해당 장의 같은 제목 코드 해설에서 확인한다.

제5장. 필수적인 연산 - 허수 접미사를 정의하는 리터럴 연산자

출처: 책 137쪽 / 원문 PDF 135쪽

C++ 원문

```
constexpr complex<double> operator""i(long double arg) // 허수
{
    return {0,arg};
}

complex<double> z = 2.7182818+6.283185i;
```

부동소수점 리터럴에 i 접미사를 붙여 complex<double> 값을 만든다. 상세 동작·주의 사항과 실행 결과 제시 여부는 해당 장의 같은 제목 코드 해설에서 확인한다.

제6장. 템플릿 - 일부 인자를 지정한 템플릿 별칭

출처: 책 160쪽 / 원문 PDF 157쪽

C++ 원문

```
template<typename Value>
using String_map = Map<string,Value>;

String_map<int> m;    // m의 타입은 Map<string,int>
```

Map의 키 타입을 string으로 고정한 새 템플릿 이름을 제공한다. 상세 동작·주의 사항과 실행 결과 제시 여부는 해당 장의 같은 제목 코드 해설에서 확인한다.

제7장. 컨셉과 제네릭 프로그래밍 - 표현식의 유효성 확인

출처: 책 169쪽 / 원문 PDF 166쪽

C++ 원문

```
requires(Iter p, int i) { p[i]; p+i; }
```

인덱싱과 더하기 표현식이 유효한 코드인지를 판단하는 requires 표현식이다. advance() 예제에서 requires 표현식만 발췌했다. 함수 정의는 생략했다. 상세 동작·주의 사항과 실행 결과 제시 여부는 해당 장의 같은 제목 코드 해설에서 확인한다.

제8장. 라이브러리의 개요 - 기능을 제공하는 표준 헤더

출처: 책 186쪽 / 원문 PDF 183쪽

C++ 원문

```
#include<string>
#include<list>
```

표준 string과 list를 제공하는 원문 헤더다. 상세 동작·주의 사항과 실행 결과 제시 여부는 해당 장의 같은 제목 코드 해설에서 확인한다.

제9장. 문자열과 정규 표현식 - 로우 문자열로 기술한 원문 우편번호 패턴

출처: 책 197쪽 / 원문 PDF 194쪽

C++ 원문

```
regex pat {R"(\w{2}\s*\d{5}(-\d{4})*)"}; // 미국 우편번호 패턴: XXddddd-dddd와 그 변종
```

백슬래시가 많은 정규 표현식을 로우 문자열 리터럴로 나타낸다. 상세 동작·주의 사항과 실행 결과 제시 여부는 해당 장의 같은 제목 코드 해설에서 확인한다.

제10장. 입력과 출력 - C 스타일 입출력과의 동기화 설정

출처: 책 224쪽 / 원문 PDF 221쪽

C++ 원문

```
ios_base::sync_with_stdio(false); // 큰 오버헤드를 피할 수 있음
```

C 스타일 입출력을 사용하지 않으면서 입출력 성능을 고려할 때 원문이 제시하는 호출이다. 상세 동작·주의 사항과 실행 결과 제시 여부는 해당 장의 같은 제목 코드 해설에서 확인한다.

제11장. 컨테이너 - map의 키로 전화번호 조회

출처: 책 245쪽 / 원문 PDF 242쪽

C++ 원문

```
int get_number(const string& s)
{
    return phone_book[s];
}
```

원문의 map<string,int> 전화번호부에서 이름 s로 번호를 조회한다. 같은 페이지의 map 초기화 코드는 생략했다. 동일 페이지의 map 초기화는 생략하고 get_number 함수 정의 전체를 발췌했습니다. 상세 동작·주의 사항과 실행 결과 제시 여부는 해당 장의 같은 제목 코드 해설에서 확인한다.

제12장. 알고리즘 - 람다 술어를 전달한 find_if

출처: 책 264쪽 / 원문 PDF 261쪽

C++ 원문

```
auto p = find_if(m.begin(), m.end(), [](const pair<string,int>& r) {
    return r.second>42;
});
```

원문의 map<string,int>에서 값 부분이 42보다 큰 첫 항목을 찾는 람다 버전이다. m의 정의 및 함수 객체 버전은 같은 설명에서 발췌하지 않았다. 원문 map 검색 설명 중 람다 호출 블록 전체를 발췌했습니다. m의 정의와 Greater_than 대안은 생략했습니다. 상세 동작·주의 사항과 실행 결과 제시 여부는 해당 장의 같은 제목 코드 해설에서 확인한다.

제13장. 유틸리티 - 타입 술어의 표기를 단순화한 래퍼

출처: 책 311쪽 / 원문 PDF 306쪽

C++ 원문

```
template<typename T>
constexpr bool Is_arithmetic()
{
    return std::is_arithmetic<T>::value;
}
```

표준 타입 특성의 value를 함수 호출 표기로 감싼 원문 래퍼다. 같은 절의 complex에서 static_assert 조건으로 활용한다. 상세 동작·주의 사항과 실행 결과 제시 여부는 해당 장의 같은 제목 코드 해설에서 확인한다.

제14장. 수치 - accumulate의 초기 값과 합산

출처: 책 318쪽 / 원문 PDF 313쪽

C++ 원문

```
list<double> lst {1, 2, 3, 4, 5, 9999.99999};
auto s = accumulate(lst.begin(),lst.end(),0.0); // 합 계산: 10014.9999
```

초기 값 0.0을 바탕으로 list의 구간을 합산한다. 원문 주석의 자릿수는 변경하지 않았다. 상세 동작·주의 사항과 실행 결과 제시 여부는 해당 장의 같은 제목 코드 해설에서 확인한다.

제15장. 동시성 - scoped_lock으로 공유 데이터 보호

출처: 책 333쪽 / 원문 PDF 327쪽

C++ 원문

```
mutex m; // 제어용 mutex
int sh; // 공유 데이터

void f()
{
    scoped_lock lck {m}; // mutex 획득
    sh += 7; // 공유 데이터 조작
} // 암묵적인 mutex 해제
```

RAII 락이 mutex 획득과 해제를 스코프에 연결해 공유 값 sh의 변경을 보호한다. 상세 동작·주의 사항과 실행 결과 제시 여부는 해당 장의 같은 제목 코드 해설에서 확인한다.

제16장. 역사와 호환성 - C 링크를 지정하는 선언

출처: 책 370쪽 / 원문 PDF 12쪽

C++ 원문

```
extern "C" double sqrt(double);
```

sqrt(double)에 C 링크를 지정해 C와 C++ 코드 조각 사이에서 호출할 수 있도록 하는 원문 선언이다. 상세 동작·주의 사항과 실행 결과 제시 여부는 해당 장의 같은 제목 코드 해설에서 확인한다.

원문 코드 찾아보기

제1장. 기초

코드	출처	학습 교재 쪽
Hello, World! 출력	출처: 책 22쪽 / 원문 PDF 21쪽	8
인자 타입에 따른 함수 오버로딩	출처: 책 25쪽 / 원문 PDF 24쪽	9
초기화와 축소 변환	출처: 책 30쪽 / 원문 PDF 29쪽	10
constexpr 함수와 상수 표현식	출처: 책 33쪽 / 원문 PDF 32쪽	11
지역 변수와 루프를 사용하는 constexpr	출처: 책 34쪽 / 원문 PDF 33쪽	11
구간 for와 요소 참조	출처: 책 36쪽 / 원문 PDF 35쪽	12
널 검사 후 C 스타일 문자열 탐색	출처: 책 38쪽 / 원문 PDF 37쪽	13
if 초기화 절과 조건	출처: 책 42쪽 / 원문 PDF 41쪽	14

제2장. 사용자 정의 타입

코드	출처	학습 교재 쪽
Vector의 데이터 표현	출처: 책 50쪽 / 원문 PDF 49쪽	16
구조체를 초기화하는 별도 함수	출처: 책 51쪽 / 원문 PDF 50쪽	17
인터페이스와 표현을 분리한 Vector	출처: 책 53쪽 / 원문 PDF 52쪽	18
Node*와 int의 저장 공간 공유	출처: 책 56쪽 / 원문 PDF 55쪽	19
variant로 표현하는 대안 값	출처: 책 57쪽 / 원문 PDF 56쪽	19
서로 독립된 enum class	출처: 책 57쪽 / 원문 PDF 56쪽	20
열거형의 전치 증가 연산 정의	출처: 책 59쪽 / 원문 PDF 58쪽	20

제3장. 모듈화

코드	출처	학습 교재 쪽
분할 컴파일을 위한 헤더	출처: 책 64쪽 / 원문 PDF 63쪽	22
헤더를 사용하는 독립 사용자 파일	출처: 책 64쪽 / 원문 PDF 63쪽	23
모듈 선언과 노출할 클래스	출처: 책 66-67쪽 / 원문 PDF 65-66쪽	24
범위 검사와 예외 핸들러	출처: 책 72쪽 / 원문 PDF 71쪽	25
생성자에서 유효한 크기 확인	출처: 책 74쪽 / 원문 PDF 73쪽	26
컴파일 시간 속성 검사	출처: 책 80쪽 / 원문 PDF 79쪽	27
여러 멤버를 가진 반환 객체	출처: 책 86쪽 / 원문 PDF 85쪽	29
구조화된 바인딩으로 멤버 사용	출처: 책 87쪽 / 원문 PDF 86쪽	29

제4장. 클래스

코드	출처	학습 교재 쪽
구체 산술 타입 complex	출처: 책 94-95쪽 / 원문 PDF 92-93쪽	32
생성자·소멸자로 배열을 관리하는 Vector	출처: 책 98쪽 / 원문 PDF 96쪽	33
초깃값 목록 생성자	출처: 책 101쪽 / 원문 PDF 99쪽	34
순수 가상 인터페이스 Container	출처: 책 102-103쪽 / 원문 PDF 100-101쪽	35
구체 구현을 모르는 사용자 함수	출처: 책 103쪽 / 원문 PDF 101쪽	35
Vector를 이용한 추상 인터페이스 구현	출처: 책 104쪽 / 원문 PDF 102쪽	36
포인터 dynamic_cast와 대안 처리	출처: 책 114쪽 / 원문 PDF 112쪽	37
unique_ptr로 소유권을 표현한 Smiley	출처: 책 116쪽 / 원문 PDF 114쪽	38

제5장. 필수적인 연산

코드	출처	학습 교재 쪽
함께 설계할 생애 주기 연산	출처: 책 120쪽 / 원문 PDF 118쪽	40
기반 클래스 복사의 사용 금지	출처: 책 122쪽 / 원문 PDF 120쪽	41
explicit 생성자로 암묵 변환 제한	출처: 책 123쪽 / 원문 PDF 121쪽	42
자기 저장 공간을 갖는 Vector 복사 생성자	출처: 책 126쪽 / 원문 PDF 124쪽	43
기존 자원을 교체하는 복사 대입	출처: 책 127쪽 / 원문 PDF 125쪽	43
지역 결과를 값으로 반환하는 벡터 덧셈	출처: 책 127-128쪽 / 원문 PDF 125-126쪽	44
배열 소유권을 가져오는 이동 생성자	출처: 책 129쪽 / 원문 PDF 127쪽	44
스레드와 큰 Vector의 자원 이전	출처: 책 131쪽 / 원문 PDF 129쪽	45
허수 접미사를 정의하는 리터럴 연산자	출처: 책 137쪽 / 원문 PDF 135쪽	47

제6장. 템플릿

코드	출처	학습 교재 쪽
타입 파라미터로 일반화한 Vector	출처: 책 142-143쪽 / 원문 PDF 139-140쪽	50
값 템플릿 인자를 받는 Buffer	출처: 책 146쪽 / 원문 PDF 143쪽	51
시퀀스와 누산기 타입을 분리한 sum	출처: 책 150쪽 / 원문 PDF 147쪽	52
데이터를 보유하는 함수 객체	출처: 책 151쪽 / 원문 PDF 148쪽	53
같은 위치에서 정의하는 람다	출처: 책 153쪽 / 원문 PDF 150쪽	54
일부 인자를 지정한 템플릿 별칭	출처: 책 160쪽 / 원문 PDF 157쪽	55
컴파일 시간 if로 구현 선택	출처: 책 160쪽 / 원문 PDF 157쪽	56

제7장. 컨셉과 제네릭 프로그래밍

코드	출처	학습 교재 쪽
시퀀스와 수의 관계를 지정하는 요구 절	출처: 책 166쪽 / 원문 PDF 163쪽	58
표현식의 유효성 확인	출처: 책 169쪽 / 원문 PDF 166쪽	59
구체적인 합산을 일반화한 accumulate	출처: 책 174쪽 / 원문 PDF 171쪽	61
파라미터 팩을 재귀적으로 처리	출처: 책 176쪽 / 원문 PDF 173쪽	62
빈 파라미터 팩 호출을 생성하지 않기	출처: 책 177쪽 / 원문 PDF 174쪽	62
오른쪽 접힘과 원문 결과	출처: 책 178쪽 / 원문 PDF 175쪽	63
왼쪽 접힘	출처: 책 178쪽 / 원문 PDF 175쪽	63

제8장. 라이브러리의 개요

코드	출처	학습 교재 쪽
기능을 제공하는 표준 헤더	출처: 책 186쪽 / 원문 PDF 183쪽	66
std 이름을 직접 한정하기	출처: 책 186쪽 / 원문 PDF 183쪽	66

제9장. 문자열과 정규 표현식

코드	출처	학습 교재 쪽
부분 문자열 복사와 교체	출처: 책 191쪽 / 원문 PDF 188쪽	68
서로 다른 저장 형태를 받는 문자열 뷰	출처: 책 195쪽 / 원문 PDF 192쪽	69
지역 문자열을 가리키는 뷰 반환의 위험	출처: 책 196쪽 / 원문 PDF 193쪽	70
로우 문자열로 기술한 원문 우편번호 패턴	출처: 책 197쪽 / 원문 PDF 194쪽	71
파일에서 전체 매치와 부분 매치 찾기	출처: 책 198-199쪽 / 원문 PDF 195-196쪽	72
정규 표현식 반복자로 단어 찾기	출처: 책 206쪽 / 원문 PDF 203쪽	74

제10장. 입력과 출력

코드	출처	학습 교재 쪽
한 행 전체를 읽는 getline	출처: 책 215쪽 / 원문 PDF 212쪽	76
입력 상태를 루프 조건으로 사용	출처: 책 215-216쪽 / 원문 PDF 212-213쪽	77
사용자 정의 타입의 입출력 연산	출처: 책 217-219쪽 / 원문 PDF 214-216쪽	78
정수 출력 진법을 바꾸는 조정자	출처: 책 219쪽 / 원문 PDF 216쪽	79
파일 스트림을 연 뒤 상태 확인	출처: 책 221쪽 / 원문 PDF 218쪽	80
문자열 스트림으로 일반적인 타입 변환	출처: 책 223쪽 / 원문 PDF 220쪽	81
C 스타일 입출력과의 동기화 설정	출처: 책 224쪽 / 원문 PDF 221쪽	81
디렉터리 순회와 파일 시스템 예외	출처: 책 226쪽 / 원문 PDF 223쪽	82

제11장. 컨테이너

코드	출처	학습 교재 쪽
vector의 구간 for 순회	출처: 책 235쪽 / 원문 PDF 232쪽	85
용량을 확보한 뒤 요소를 생성하는 push_back	출처: 책 237쪽 / 원문 PDF 234쪽	86
at()에 연결하여 구간 검사하는 Vec	출처: 책 239쪽 / 원문 PDF 236쪽	87
반복자로 전화번호부 선형 탐색	출처: 책 243쪽 / 원문 PDF 240쪽	88
map의 키로 전화번호 조회	출처: 책 245쪽 / 원문 PDF 242쪽	89

제12장. 알고리즘

코드	출처	학습 교재 쪽
정렬 후 중복 없이 새 list에 삽입	출처: 책 255쪽 / 원문 PDF 252쪽	92
반복자 타입 별칭을 사용한 제네릭 find_all	출처: 책 258쪽 / 원문 PDF 255쪽	93
파일 단어를 정렬하고 중복 없이 출력	출처: 책 262쪽 / 원문 PDF 259쪽	94
람다 술어를 전달한 find_if	출처: 책 264쪽 / 원문 PDF 261쪽	95
정렬의 실행 정책	출처: 책 273쪽 / 원문 PDF 270쪽	98

제13장. 유틸리티

코드	출처	학습 교재 쪽
내장 포인터와 unique_ptr의 해제 비교	출처: 책 278쪽 / 원문 PDF 275쪽	101
이동을 이용한 swap	출처: 책 282쪽 / 원문 PDF 279쪽	102
정수를 bitset 문자열로 출력	출처: 책 291쪽 / 원문 PDF 286쪽	104
tuple 생성과 타입 추론	출처: 책 294쪽 / 원문 PDF 289쪽	104
값이 있는 optional만 합산	출처: 책 299쪽 / 원문 PDF 294쪽	105
chrono로 작업 시간 측정	출처: 책 302쪽 / 원문 PDF 297쪽	107
람다로 멤버 함수 호출 연결	출처: 책 304쪽 / 원문 PDF 299쪽	107
포인터용 iterator_traits	출처: 책 309쪽 / 원문 PDF 304쪽	108
타입 술어의 표기를 단순화한 래퍼	출처: 책 311쪽 / 원문 PDF 306쪽	109

제14장. 수치

코드	출처	학습 교재 쪽
accumulate의 초기 값과 합산	출처: 책 318쪽 / 원문 PDF 313쪽	113
reduce와 병렬 실행 정책	출처: 책 320쪽 / 원문 PDF 315쪽	113
엔진과 분포를 묶은 Rand_int	출처: 책 322쪽 / 원문 PDF 317쪽	114
valarray의 배열 수준 산술	출처: 책 324쪽 / 원문 PDF 319쪽	115
수치 타입의 성질을 컴파일 시간에 검사	출처: 책 324쪽 / 원문 PDF 319쪽	116

제15장. 동시성

코드	출처	학습 교재 쪽
함수와 함수 객체를 스레드에서 실행	출처: 책 329쪽 / 원문 PDF 323쪽	118
참조 인자를 전달하는 두 방식	출처: 책 330-331쪽 / 원문 PDF 324-325쪽	119
별도 출력 위치로 결과를 돌려받기	출처: 책 332쪽 / 원문 PDF 326쪽	120
scoped_lock으로 공유 데이터 보호	출처: 책 333쪽 / 원문 PDF 327쪽	121
조건 변수로 메시지 큐를 기다리는 소비자	출처: 책 337쪽 / 원문 PDF 331쪽	122
promise로 값 또는 예외 전달	출처: 책 339-340쪽 / 원문 PDF 333-334쪽	123
packaged_task로 두 구간의 계산과 결과 연결	출처: 책 341쪽 / 원문 PDF 335쪽	124
async로 네 구간의 결과를 모으기	출처: 책 342쪽 / 원문 PDF 336쪽	125

제16장. 역사와 호환성

코드	출처	학습 교재 쪽
원문에서 제거 사례로 든 예외 명세	출처: 책 364쪽 / 원문 PDF 6쪽	132
malloc 결과의 암묵적 포인터 변환	출처: 책 369쪽 / 원문 PDF 11쪽	133
void* 변환이 초래할 수 있는 문제	출처: 책 369쪽 / 원문 PDF 11쪽	134
C 링크를 지정하는 선언	출처: 책 370쪽 / 원문 PDF 12쪽	134

주요 개념 복습

제1장. 기초

출처: 책 19-48쪽 / 원문 PDF 18-47쪽

- 컴파일과 링크의 역할, 소스 이식성의 의미를 다시 연결한다.
- 함수 선언의 인자·반환 타입과 오버로딩 선택을 함께 복습한다.
- 스코프는 이름의 가시성과, 수명은 객체의 생성·소멸과 연결한다.
- 배열의 연속성, 포인터의 주소, 참조의 바인딩을 구별한다.

제2장. 사용자 정의 타입

출처: 책 49-60쪽 / 원문 PDF 48-59쪽

- 관련 데이터의 구조화에서 표현·인터페이스 분리로 이어지는 Vector의 발전을 복습한다.
- 생성자, 초깃값 목록, operator[], size()가 각각 맡는 역할을 연결한다.
- 타입 필드와 union의 활성 값 사이 관계를 캡슐화해야 하는 이유를 확인한다.
- Color와 Traffic_light가 같은 red 이름을 가져도 독립된 타입이라는 점을 재확인한다.

제3장. 모듈화

출처: 책 61-89쪽 / 원문 PDF 60-88쪽

- 인터페이스를 공유하면서 구현을 독립 컴파일하는 파일 관계를 복습한다.
- 예외를 탐지하는 함수와 처리하는 함수의 역할을 분리해 본다.
- 생성자·불변 조건·RAII·예외 처리가 연결되는 흐름을 재확인한다.
- 값·참조·const 참조·이동과 구조화된 바인딩이 함수 경계에서 맡는 역할을 정리한다.

제4장. 클래스

출처: 책 91-118쪽 / 원문 PDF 89-116쪽

- complex의 표현·생성자·const 접근자·비멤버 연산의 관계를 복습한다.
- Vector의 자원 획득부터 스코프 종료·배열 해제까지 흐름을 다시 확인한다.
- Container→Vector_container와 Shape→Circle→Smiley의 관계를 원문 순서로 정리한다.
- 가상 소멸자와 unique_ptr가 각각 동적 타입의 소멸 선택과 소유권에 맡는 역할을 구별한다.

제5장. 필수적인 연산

출처: 책 119-139쪽 / 원문 PDF 117-137쪽

- 생성·복사·이동·대입·소멸을 한 집합으로 설계하는 이유를 재확인한다.
- 자원 핸들에 멤버별 복사를 적용했을 때의 공유와, 독립 배열 복사의 차이를 복습한다.
- 함수 결과의 값 반환, 이동, 복사 생략을 연결한다.
- 비교·반복자 범위·입출력·리터럴·swap·hash가 기존 코드의 관례와 맞아야 함을 정리한다.

제6장. 템플릿

출처: 책 141-162쪽 / 원문 PDF 138-159쪽

- 타입 파라미터, 값 파라미터, 요구 조건을 구분해 원문 Vector와 Buffer의 목적을 떠올린다.

- 시퀀스 요소 타입과 누산기 타입이 독립적이라는 관계를 sum()으로 복습한다.
- 함수 객체의 데이터와 람다 캡처의 관계를 다시 확인한다.
- 별칭은 표현을 단순화하고 if constexpr은 인스턴스화할 경우를 선택한다.

제7장. 컨셉과 제네릭 프로그래밍

출처: 책 163-182쪽 / 원문 PDF 160-179쪽

- Sequence·Number·Arithmetic이 각각 개별 성질과 두 타입 사이의 관계를 맡는다.
- 구문적으로 연산이 존재한다는 사실과 의미 있는 컨셉이라는 조건을 구별한다.
- 리프팅에서는 자료 구조·요소 타입·누산기·초깃값의 고정을 분리한다.
- 재귀의 종료 조건과 if constexpr, 접힘의 방향을 원문 코드와 연결한다.

제8장. 라이브러리의 개요

출처: 책 183-188쪽 / 원문 PDF 180-185쪽

- 컨테이너와 알고리즘을 비롯한 기본 기능은 표준 라이브러리가 제공한다.
- 표준 헤더와 std 네임스페이스를 함께 확인한다.
- C 헤더와 c 접두사 버전의 이름 제공 방식을 구분한다.

제9장. 문자열과 정규 표현식

출처: 책 189-208쪽 / 원문 PDF 186-205쪽

- substr의 복사와 replace의 교체를 원문 name 예로 구분한다.
- 임시 string을 줄이는 뷰의 장점과 소유하지 않는다는 제약을 함께 복습한다.
- 특수 문자·문자 클래스·반복·그룹을 서로 다른 구성 요소로 읽는다.
- 전체 매치·부분 매치·반복자 끝 표시를 원문 검색 코드와 연결한다.

제10장. 입력과 출력

출처: 책 209-232쪽 / 원문 PDF 206-229쪽

- ostream과 istream의 변환 방향을 값과 문자 시퀀스에 연결한다.
- 공백을 건너뛰는 >>, 한 행의 getline, 공백을 보존하는 get의 목적을 구별한다.
- 상태 검사·failbit 설정·clear로 회복하는 흐름을 원문 사례로 복습한다.
- 스트림은 파일과 문자열에도 같은 형식화 연산을 적용한다.
- path 표현과 실제 파일 상태, 디렉터리 순서와 정렬은 구별한다.

제11장. 컨테이너

출처: 책 233-252쪽 / 원문 PDF 230-249쪽

- vector의 연속 저장·복사·성장·reserve와 반복자 유효성의 관계를 복습한다.
- 구간 밖 접근을 at()·구간 for·시작/끝 반복자로 다루는 방식을 정리한다.
- list의 삽입·삭제와 vector의 짧은 시퀀스 효율을 함께 고려한다.
- map의 []는 없는 키를 삽입하며 unordered_map은 해시 품질과 순서 유무를 고려해야 한다.

제12장. 알고리즘

출처: 책 253-274쪽 / 원문 PDF 250-271쪽

- 반복자가 컨테이너 저장 구조와 알고리즘을 연결하는 공통 인터페이스임을 정리한다.
- find와 find_if, 일반 출력 반복자와 back_inserter의 조건을 구분한다.
- 입력·출력·순방향·양방향·임의 접근 반복자의 요구와 Range 계열을 연결한다.
- 술어의 비변경 조건과 병렬 계산의 독립성·경합 방지 조건을 복습한다.

제13장. 유틸리티

출처: 책 275-287, 290-314쪽 / 원문 PDF 272-309쪽

- 지역 객체·컨테이너·unique_ptr·shared_ptr 가운데 필요한 자원 관리 수준과 소유권을 구분한다.
- 구간의 길이와 포인터를 함께 다루는 span, 고정 크기 array, 비트 집합 bitset의 목적을 복습한다.
- pair·tuple의 위치·타입 접근과 이름 있는 struct의 가독성 차이를 정리한다.
- 할당자의 단편화 개선 사례와 chrono의 실제 성능 측정을 연결한다.
- 람다·mem_fn·function의 호출 연결과 iterator_traits·타입 술어·enable_if의 컴파일 시간 역할을 구분한다.

제14장. 수치

출처: 책 315-325쪽 / 원문 PDF 310-320쪽

- 합·내적·부분합·인접 차이의 목적과 연산 매개변수를 구분한다.
- 엔진은 값의 시퀀스를 만들고 분포는 필요한 범위와 분포로 매핑한다.
- 일반 저장 vector와 수치 계산 valarray의 선택 기준을 복습한다.
- 수치 타입의 표현 가능성은 numeric_limits로 확인한다.

제15장. 동시성

출처: 책 327-345쪽 / 원문 PDF 321-339쪽

- 분리된 태스크를 함수처럼 다루는 방식과 명시적인 공유 데이터 접근의 차이를 복습한다.
- scoped_lock·unique_lock·shared_lock의 역할과 보호 데이터의 관계를 정리한다.
- wait의 락 해제·재획득·조건 확인 순서를 설명한 원문 흐름을 재확인한다.
- 값 또는 예외가 promise에서 future로 전달되는 관계를 복습한다.
- packaged_task의 이동과 async의 실행 자원·성능 가정에 관한 원문 조건을 정리한다.

제16장. 역사와 호환성

출처: 책 347-370쪽 / 원문 PDF 1-12, 340-351쪽

- 문제 지향적인 설계와 언어·라이브러리의 공동 발전을 연결해 복습한다.
- 초기 C with Classes, C++98, C++11, C++14, C++17의 흐름은 원문의 연혁으로 확인한다.
- 이 장의 C++20과 코루틴에 대한 예정 표현은 원문 시점의 설명으로 읽는다.
- C에서 C++로 옮길 때 스타일, void* 변환, 키워드 충돌, 링크를 구별한다.
- 실행 결과가 없는 코드에는 출력이나 예상값을 만들어 붙이지 않는다.

개념 색인

개념	교재 쪽	개념	교재 쪽
=default	40	=delete	40
any	106	array	103
async	124	auto	10
back_inserter	92	bitset	103
C with Classes	128	C++11	130
C++14	130	C++17	130
C/C++ 호환성	132	capacity / reserve	85
chrono / duration_cast	106	complex	114
condition_variable / unique_lock	121	const	10
const 멤버 함수	31	constexpr	10
cout	76	directory_iterator	82
dynamic_cast	37	enable_if / SFINAE	109
enum class	20	explicit	41
extern "C"	134	failbit	77
filesystem	82	find / find_all	93
forward_list	88	fstream	80
function	108	future / promise	123
getline	76	gsl::span	102
hash	47	if constexpr	55
initializer_list	34	iostream	76
iterator_traits / 태그 디스패치	108	list	88
make_unique / make_shared	100	map	88
mem_fn	108	module / import	23
move / forward	101	mutex / scoped_lock	120
noexcept	25	nullptr	12
numeric_limits	115	optional	105
override	34	packaged_task	123
pair / tuple	104	path	82
precision	79	public / private	17
RAII	33	RAII / 자원	100
RAII의 역사	128	Ranges TS / 컨셉	96

개념	교재 쪽	개념	교재 쪽
reduce / 병렬 수치 알고리즘	113	regex	70
regex_iterator	73	Regular	60, 96
requires 표현식	59	rvalue 참조	44
shared_mutex / shared_lock	120	smatch	71
static_assert	27	static_cast	34
std 네임스페이스	66	std::move	44
STL	66	string	68
string_view	69	stringstream	80
struct	16	substr	68
swap	47	union	18
unique_ptr	38	unique_ptr / shared_ptr	100
unordered_map	89	valarray	115
variant	18	variant / visit	105
vector	85	void*	133
vtbl	36	WG21	129
값 반환	28	값 템플릿 인자	51
계약	27	구간 검사 / at	87
구조화된 바인딩	28	구체 타입	31
구현 상속	37	기본 멤버 초깃값	42
기본 생성자	31	난수 엔진과 분포	114
네임스페이스	24	덕 타이핑	64
데드락	120	동시성 / 메모리 모델	117
람다 어댑터	107	람다 표현식	53
로우 문자열 리터럴	70	리프팅	60
문자 클래스	72	반복자	46
반복자 타입	93	배치 연산 / emplace_back	90
변환 단위	22	별칭 템플릿	55
병렬 알고리즘 / 실행 정책	98	보초 / Sentinel	96
복사 대입	42	복사 생성자	42
분할 컴파일	22	불변 조건	26
사용자 정의 리터럴	47	사용자 정의 타입	16
생성자	17	소멸자	33
수치 알고리즘 / accumulate	112	수학 함수 / errno	111

개념	교재 쪽	개념	교재 쪽
순수 가상 함수	34	술어 / find_if	95
스코프	10	스트림 반복자	94
알고리즘 / 반개구간	92	예외	25
이동 생성자	44	인스턴스화	49
인자 포워딩	64	인터페이스 상속	37
입출력 연산자	78	자원 안전성	45
접힘 표현식	63	정적 타입	7
조정자	79	지연 매칭	72
지원 중단	131	짧은 문자열 최적화	69
참조	12	참조 인자 / ref	118
초기화	14	추상 타입	34
축소 변환	10	컨셉	58
컨테이너	85	컨테이너 알고리즘	98
컨테이너 요소	87	타입 술어	109
태스크 / thread / join	118	태스크 결과 반환	119
템플릿	49	템플릿 인자 추론	52
특수 멤버 함수	40	파라미터 팩	61
포인터	12	표준 라이브러리	65
표준 알고리즘	95	표준 헤더	66
할당자 / pmr	106	함수 객체	53
함수 오버로딩	8	핸들	17